



Geoprocessamento
com



pythonTM

ALEXANDRO GULARTE SCHAFER



Geoprocessamento com Python™



Dados Internacionais de Catalogação na Publicação (CIP)

G2929 Geoprocessamento com Python™/Schafer, A. G.
— Campina Grande: EPTEC, 2025.

118 f.: il. color.

Formato: PDF
Requisitos de sistema: Adobe Acrobat Reader
Modo de acesso: World Wide Web
Inclui bibliografia

ISBN: 978-65-01-85931-6

1. Geoinformática. 2. Análise espacial. 3. Python. I. Schafer, Alexandro Gularte. II.
Título.

CDU 681.3

Os capítulos ou materiais publicados são de inteira responsabilidade de seus autores.
As opiniões neles emitidas não exprimem, necessariamente, o ponto de vista do Editor responsável.
Sua reprodução parcial está autorizada desde que cite a fonte.



Todo o conteúdo deste livro está licenciado sob a Licença de Atribuição Creative Commons.
Atribuição-Não-Comercial-Não Derivativos 4.0 Internacional (CC BY-NC-ND 4.0).

2025 by Eptec
Copyright © Eptec
Copyright do texto © 2025 Os autores
Copyright da edição © 2025 Eptec
Direitos para esta edição cedidos à Eptec pelos autores.
Open access publication by Eptec

Crédito das imagens da capa
Freepik.com

Editoração, Revisão e Arte da Capa
Paulo Roberto Megna Francisco

Conselho Editorial
Claudiomir Silva Santos (IFSULMINAS)
Djail Santos (CCA-UFPB)
Dermeval Araújo Furtado (CTRN-UFCG)
Flávio Pereira de Oliveira (CCA-UFPB)
George do Nascimento Ribeiro (CDSA-UFCG)
Gypson Dutra Junqueira Ayres (CTRN-UFCG)
João Miguel de Moraes Neto (CTRN-UFCG)
José Nilton Silva (CTRN-UFCG)
José Wallace Barbosa do Nascimento (CTRN-UFCG)
Lúcia Helena Garófalo Chaves (CTRN-UFCG)
Luciano Marcelo Fallé Saboya (CTRN-UFCG)
Newton Carlos Santos (UFRN)
Paulo da Costa Medeiros (CDSA-UFCG)
Paulo Roberto Megna Francisco (CTRN-UFCG)
Raimundo Calixto Martins Rodrigues (DEAG-UEMA)
Soahd Arruda Rached Farias (CTRN-UFCG)
Virgínia Mirtes de Alcântara Silva (CTRN-UFCG)
Viviane Farias Silva (CSTR-UFCG)

Alexandro Gualarte Schafer

Geoprocessamento com Python™



1.a Edição
Campina Grande-PB
2025

REALIZAÇÃO



APOIO



SUMÁRIO

CAPÍTULO 1	8
GEOPROCESSAMENTO COM PYTHON™: FUNDAMENTOS CONCEITUAIS E CONTEXTO ATUAL	8
1.1 GEOTECNOLOGIAS, GEOINFORMÁTICA E GISCIENCE.....	9
1.2 PROGRAMAÇÃO NO CONTEXTO DO GEOPROCESSAMENTO.....	10
1.3 A LINGUAGEM PYTHON™ E SUA FILOSOFIA	11
1.4 PYTHON™ APLICADO AO GEOPROCESSAMENTO: CASOS DE USO	12
1.5 ECOSISTEMA DE BIBLIOTECAS PARA GEOPROCESSAMENTO	13
1.6 AMBIENTES DE DESENVOLVIMENTO EM PYTHON™	14
CAPÍTULO 2.....	15
PRIMEIRO CONTATO COM PYTHON™ NO GOOGLE COLAB®	15
2.1 O QUE É O GOOGLE COLAB® E POR QUE VAMOS USÁ-LO.....	16
2.2 CONHECENDO A TELA DO NOTEBOOK	16
2.3 SEU PRIMEIRO CÓDIGO EM PYTHON™	17
2.4 INDENTAÇÃO E SÍMBOLOS BÁSICOS EM PYTHON	19
2.5 VARIÁVEIS E TIPOS DE DADOS.....	20
2.6 ESTRUTURAS DE DADOS BÁSICAS: LISTAS, TUPLAS E DICIONÁRIOS	22
2.6.1 LISTAS	22
2.6.2 TUPLAS.....	24
2.6.3 DICIONÁRIOS	24
2.7 ESTRUTURAS DE REPETIÇÃO	25
2.8 FUNÇÕES: AGRUPANDO PASSOS EM BLOCOS REUTILIZÁVEIS.....	27
2.9 IMPORTANDO BIBLIOTECAS: AMPLIANDO O PODER DO PYTHON™	28
CAPÍTULO 3.....	30
ROTINA DE TRABALHO E ORGANIZAÇÃO DOS NOTEBOOKS NO GOOGLE COLAB®	30
3.1 COMO OS NOTEBOOKS DO LIVRO SÃO ORGANIZADOS	30
3.2 O BLOCO INICIAL DE TODO NOTEBOOK: PASTAS, DADOS E VERIFICAÇÃO	31
3.3 BIBLIOTECAS NO COLAB®: INSTALAR QUANDO NECESSÁRIO E IMPORTAR	32
3.4 ONDE SALVAR RESULTADOS E COMO RESOLVER ERROS COMUNS.....	32
CAPÍTULO 4.....	34
FUNDAMENTOS DE DADOS VETORIAIS COM GEOPANDAS.....	34
4.1 PREPARAÇÃO DO NOTEBOOK	34
4.2 O MODELO VETORIAL E O QUE É UM GEODATAFRAME	35

4.3 GEOMETRIAS BÁSICAS: PONTO, LINHA E POLÍGONO	38
4.4 OPERAÇÕES GEOMÉTRICAS ESSENCIAIS: MEDIDAS E RELAÇÕES ESPACIAIS (ENXUGADA)	39
4.5 LENDO UM GEODATAFRAME COMO TABELA: INSPEÇÃO E FILTROS BÁSICOS	43
CAPÍTULO 5.....	45
VISUALIZAÇÃO DE DADOS ESPACIAIS	45
5.1 MAPAS ESTÁTICOS COM GEOPANDAS E MATPLOTLIB	46
5.2 ADICIONANDO MAPAS BASE COM CONTEXTILY	55
5.3 MAPAS INTERATIVOS COM FOLIUM	57
5.4 QUANDO USAR CADA FERRAMENTA?	59
CAPÍTULO 6.....	61
GEOMETRIAS E OPERAÇÕES ESPACIAIS	61
6.1 PREPARANDO O CONJUNTO MÍNIMO DE GEOMETRIAS	61
6.2 PROPRIEDADES GEOMÉTRICAS EM CRS MÉTRICO	64
6.3 OPERAÇÕES BASEADAS EM DISTÂNCIA: BUFFER	65
6.4 OPERAÇÕES DE SOBREPOSIÇÃO: CLIP E OVERLAY	66
6.5 AGREGAÇÃO ESPACIAL: DISSOLVE	70
6.6 JUNÇÃO ESPACIAL: COMBINANDO TABELAS POR LOCALIZAÇÃO (SJOIN)	71
6.7 GEOMETRIAS INVÁLIDAS: DIAGNÓSTICO E CORREÇÃO	71
CAPÍTULO 7.....	75
FUNDAMENTOS E DIAGNÓSTICO DE DADOS RASTER.....	75
7.1 ABRINDO O RASTER E INSPECIONANDO METADADOS	76
7.2 LENDO A BANDA E FAZENDO UM “MAPA RÁPIDO”	77
7.3 ESTATÍSTICAS INICIAIS E DETECÇÃO DE VALORES SUSPEITOS	79
7.4 HISTOGRAMA: CONFIRMANDO O PADRÃO NUMÉRICO	80
7.5 NODATA, NAN E A CORREÇÃO EM MEMÓRIA	80
7.6 SALVANDO UM GEOTIFF “LIMPO” COM NODATA EXPLÍCITO	82
7.7 REABRINDO E VALIDANDO A VERSÃO CORRIGIDA	82
7.8 ESTATÍSTICAS FINAIS E COBERTURA VÁLIDA	85
CAPÍTULO 8.....	87
PROCESSAMENTO E TRANSFORMAÇÃO DE DADOS RASTER.....	87
8.1 ABRINDO OS DADOS DO REPOSITÓRIO.....	88
8.2 RECORTE ESPACIAL DE RASTERS	91
8.2.1 RECORTE POR BOUNDING BOX.....	91
8.2.2 RECORTE POR POLÍGONO (MÁSCARA PELA BACIA).....	92
8.3 SALVANDO O RASTER RECORTADO PELA BACIA.....	93
8.4 REPROJEÇÃO DE RASTERS E ESCOLHA DO MÉTODO DE REAMOSTRAGEM	95

8.5 RASTERS MULTIBANDA: CONCEITO E PRIMEIROS CONTATOS.....	98
CAPÍTULO 9.....	103
INTEGRAÇÃO RASTER + VETOR: ESTATÍSTICA ZONAL.....	103
9.1 DADOS UTILIZADOS E PREPARAÇÃO INICIAL.....	103
9.2 ESTATÍSTICA ZONAL: ENTENDENDO O PROCESSO EM UMA ÚNICA SUB BACIA	106
9.3 AUTOMATIZANDO A ESTATÍSTICA ZONAL PARA TODAS AS SUB BACIAS	109
9.4 INTERPRETANDO OS RESULTADOS DA ESTATÍSTICA ZONAL	112
CAPÍTULO 10	114
ORGANIZAÇÃO, CUIDADOS ESSENCIAIS E PRÓXIMOS PASSOS	114
10.1 ORGANIZAÇÃO E FLUXO DE TRABALHO EM PROJETOS GEOESPACIAIS.....	114
10.2 CUIDADOS ESSENCIAIS COM RASTERS E VETORES	115
10.3 CAMINHOS PARA SEGUIR ALÉM DESTES LIVROS	116
CURRICULUM DO AUTOR	118

CAPÍTULO 1

GEOPROCESSAMENTO COM PYTHON™: FUNDAMENTOS CONCEITUAIS E CONTEXTO ATUAL

O uso de dados geoespaciais cresceu de forma impressionante nas últimas décadas. Mapas digitais, imagens de satélite, dados de sensores, trajetórias de veículos e registros georreferenciados passaram a fazer parte do cotidiano de governos, empresas, pesquisadores e da população em geral. Essa expansão não se limita à produção de mapas visualmente atrativos: representa uma mudança profunda na forma como observamos, analisamos e tomamos decisões sobre o território.

A chamada Tecnologia Geoespacial reúne um conjunto amplo de métodos e ferramentas voltados à representação e à análise de fenômenos no espaço e no tempo. Nesse conjunto estão a cartografia digital, os sistemas de navegação por satélite, o sensoriamento remoto, os sistemas de informação geográfica (SIG), os bancos de dados espaciais e as aplicações de mapeamento na *web*. Hoje, essas tecnologias são centrais em atividades como monitorar desastres naturais, acompanhar mudanças de uso e cobertura da terra, planejar redes de transporte, gerir recursos hídricos e avaliar impactos ambientais.

Paralelamente, cresceu a demanda por análises estruturadas e reproduzíveis baseadas em informação espacial, o que tornou cada vez mais importante a integração entre dados geográficos e processamento computacional. É nesse contexto que este livro se insere: apresenta, de forma introdutória e prática, como utilizar Python™ para organizar, manipular e analisar dados geoespaciais, integrando fundamentos de geoprocessamento e geoinformática em fluxos de trabalho claros e aplicáveis.

Ao longo dos próximos capítulos, vamos construir um percurso gradual e bem guiado: começamos pelas ideias fundamentais — o que são dados espaciais, por que programar e quais ferramentas estão envolvidas — e avançamos para a prática com dados vetoriais e matriciais, sempre orientados por problemas reais do geoprocessamento.

Este primeiro capítulo tem, portanto, um papel mais conceitual: situar o leitor no campo das geotecnologias e mostrar por que o Python™ se tornou uma linguagem central nesse

contexto, funcionando como uma ponte entre o entender o dado e o fazer acontecer, por meio de fluxos de trabalho claros, reproduzíveis e fáceis de manter.

1.1 GEOTECNOLOGIAS, GEOINFORMÁTICA E GISCIENCE

Quando falamos em geotecnologias, não estamos nos referindo a uma ferramenta isolada, mas a um conjunto de práticas, conceitos e instrumentos voltados à representação, ao processamento e à análise do espaço geográfico. Ao longo das últimas décadas, a cartografia tradicional — baseada em papel e instrumentos analógicos — evoluiu para uma cartografia digital integrada a bancos de dados, sensores, modelos numéricos e algoritmos. Hoje, o uso cotidiano de GPS em smartphones, o acesso a imagens de satélite em plataformas abertas e a disponibilidade de dados geográficos em serviços web fazem parte do cenário comum de trabalho.

Dentro desse universo, a Geoinformática ocupa um papel central. Ela trata da aquisição, armazenamento, processamento, análise e disseminação de informação geográfica, combinando cartografia, geodésia, ciência da computação e estatística. Em termos bem práticos, quando você organiza um conjunto de arquivos, recorta dados por uma área de interesse, padroniza projeções, integra camadas diferentes e gera produtos finais (mapas, tabelas, relatórios, painéis), você está trabalhando no domínio da Geoinformática. É a disciplina que dá forma aos processos: do dado bruto ao resultado utilizável.

A GIScience (*Geographic Information Science*) pode ser vista como o “lado científico” da informação geográfica. Enquanto a Geoinformática enfatiza ferramentas e processos, a GIScience discute modelos e conceitos: como representar entidades e fenômenos no espaço e no tempo; como lidar com diferentes escalas; quais estruturas são adequadas para fenômenos contínuos (como altitude, temperatura ou precipitação) e discretos (como lotes, pontos de amostragem, linhas de infraestrutura); como definir relações espaciais (vizinhança, conectividade, proximidade); e como projetar métodos que extraiam padrões de maneira consistente. Muitas operações comuns — *buffers*, interseções, modelos de vizinhança, medidas de autocorrelação e clusters espaciais — têm raízes conceituais discutidas nesse campo.

Nos últimos anos, essa discussão se aproximou fortemente da Ciência de Dados, impulsionada por dois fatores: (1) a abundância de dados com localização (sensoriamento remoto em alta resolução, redes de sensores, registros administrativos, bases cadastrais detalhadas, dados de mobilidade) e (2) o avanço das técnicas computacionais para trabalhar com grandes volumes de informação. Esse encontro levou ao uso do termo Ciência de Dados

Espacial: a aplicação sistemática de métodos de ciência de dados em conjuntos que possuem referência espacial, com desafios e cuidados próprios (como dependência espacial, efeitos de escala, agregações e heterogeneidade do território).

A partir desse cenário, também se popularizou a GeoAI (*Geospatial Artificial Intelligence*), que integra algoritmos de aprendizado de máquina e *deep learning* com dados geográficos, especialmente imagens, redes e séries espaço-temporais georreferenciadas. Aplicações típicas incluem segmentação de uso e cobertura da terra, detecção automatizada de mudanças, classificação de feições em imagens, previsão em redes de transporte e identificação de padrões anômalos em séries espaço-temporais.

Este livro não tem como objetivo aprofundar teoria de *GIScience*, nem se tornar um manual de GeoAI. Em vez disso, ele se apoia nesse pano de fundo para sustentar um ponto central: trabalhar com informação geográfica exige método, e método significa organizar dados e operações em um fluxo coerente. É aí que o Python™ entra como ferramenta de integração e de execução: ele ajuda a transformar operações “soltas” em um processo controlado, documentado e repetível.

1.2 PROGRAMAÇÃO NO CONTEXTO DO GEOPROCESSAMENTO

Softwares de SIG com interface gráfica tornaram muitas tarefas acessíveis: carregar camadas, aplicar buffer, interseção, dissolver polígonos, calcular áreas e produzir mapas temáticos pode ser feito com poucos cliques. Então por que programar?

A ideia não é “substituir o SIG”. O objetivo é ganhar controle, repetição e consistência.

Ao programar, você descreve o caminho da análise passo a passo. O que antes era uma sequência de cliques vira um roteiro explícito: quais arquivos entram, que parâmetros são usados, o que é filtrado, qual projeção é adotada, o que é salvo e com que nome. O código passa a funcionar como uma documentação viva do método — algo valioso quando você precisa justificar resultados, revisar decisões ou repetir a análise meses depois.

Em geoprocessamento real, raramente existe “um arquivo e pronto”. É comum lidar com dezenas de municípios, várias datas, múltiplas bacias, centenas de cenas ou muitas camadas para integrar. Repetir manualmente aumenta tempo e chance de erro. Com programação, você transforma uma sequência de tarefas em um script: muda a entrada, roda de novo e obtém o mesmo tipo de saída com consistência.

Um notebook ou script também pode ser compartilhado, revisado por pares e executado novamente por outra pessoa (ou por você mesmo em outro computador). Em ambientes

acadêmicos, institucionais e profissionais, isso faz diferença: a análise deixa de ser “o que alguém fez no computador” e passa a ser um procedimento verificável.

Além disso, um fluxo bem escrito tende a ser mais fácil de manter e atualizar quando surgem novos dados, novas regras, um novo recorte territorial ou uma mudança no padrão dos arquivos. Em vez de refazer tudo do zero, você ajusta uma etapa e o restante continua funcionando.

Na prática, a abordagem mais eficiente costuma ser híbrida: usar o SIG gráfico para exploração visual, checagens rápidas e edições pontuais, e usar programação para padronizar, automatizar, documentar e integrar processos. Este livro se concentra nesse segundo eixo: usar Python™ como ponte entre o entendimento do dado e a execução de um fluxo robusto.

1.3 A LINGUAGEM PYTHON™ E SUA FILOSOFIA

O Python™ se tornou uma linguagem muito usada em geoprocessamento por reunir características que, juntas, fazem diferença no dia a dia. Sua sintaxe tende a ser direta e fácil de acompanhar, o que reduz a barreira de entrada e favorece a construção de fluxos de trabalho que não dependem de “mágicas” ou passos escondidos. Em geoprocessamento, onde é comum encadear várias operações, a legibilidade do código é parte do que torna o processo mais confiável.

Além disso, Python™ funciona muito bem em ambientes em que executamos etapas aos poucos, conferimos resultados e ajustamos parâmetros — especialmente em notebooks. Isso combina com o modo como análises espaciais geralmente são conduzidas: aplicar uma transformação, visualizar no mapa, checar números, salvar a saída e seguir para a próxima etapa.

Outro ponto decisivo é o ecossistema de bibliotecas, que resolve partes específicas do problema e permite montar um fluxo robusto com componentes já consolidados: ler e escrever dados, tratar geometrias, reprojeter coordenadas, manipular matrizes, produzir mapas, automatizar tarefas e conectar bancos de dados. Em vez de reinventar ferramentas básicas, o usuário combina peças confiáveis para construir um processo completo.

Existe ainda uma filosofia associada ao Python™, frequentemente chamada de “Zen do Python”, que valoriza clareza, simplicidade e explicitação. No contexto deste livro, isso se traduz em um compromisso prático: escrever código que pareça um roteiro do que está sendo feito, e não uma sequência obscura de comandos — exatamente o que ajuda a construir fluxos claros, reproduzíveis e fáceis de manter.

1.4 PYTHON™ APLICADO AO GEOPROCESSAMENTO: CASOS DE USO

Em projetos geoespaciais, o Python™ costuma funcionar como uma “cola” que conecta etapas e transforma procedimentos isolados em um fluxo de trabalho completo. Em vez de pensar apenas em uma lista de tarefas, é mais útil enxergar um roteiro típico, no qual o processo vai do dado bruto ao produto final com passos claros e repetíveis.

Tudo começa pela entrada dos dados: carregamos camadas vetoriais (como limites, rios, estradas e pontos), matriciais (como MDE, imagens de satélite e mapas contínuos) e, quando necessário, tabelas auxiliares (CSV, planilhas ou consultas em bancos de dados). Em seguida, vem a fase de checagens e padronização, em que confirmamos o CRS, verificamos se as camadas estão alinhadas, corrigimos problemas de geometria, padronizamos nomes de colunas, avaliamos valores ausentes e organizamos diretórios e arquivos. Essa etapa é parte do “entender o dado”: antes de analisar, é preciso confiar no material de entrada.

Com a base pronta, avançamos para o processamento e as operações espaciais, que formam o núcleo do “fazer acontecer”. Aqui entram recortes, interseções, *buffers*, dissoluções, junções espaciais, reprojeções, estatísticas por áreas, reclassificações, cálculo de índices em *rasters* e geração de produtos derivados, como declividade, NDVI, máscaras e grades — sempre com parâmetros explícitos e execução repetível. Ao longo do caminho, a visualização e a validação são indispensáveis: mapas não servem apenas para “embelezar”, mas para detectar erros e confirmar se o resultado faz sentido, e por isso um fluxo didático e robusto alterna processamento e inspeção visual.

Por fim, o trabalho se materializa em saídas e integração: exportamos novos arquivos (*GeoPackage*, GeoJSON, GeoTIFF), tabelas, mapas estáticos, relatórios ou camadas prontas para uso em um SIG. Em cenários mais amplos, conectamos esse fluxo a bancos de dados, serviços web ou pipelines maiores. Dentro desse panorama, aparecem também usos ligados à Ciência de Dados e à GeoAI — como extrair variáveis geográficas para treinar modelos, automatizar classificação de imagens e detectar mudanças —, o que ajuda a explicar por que o Python™ se consolidou como uma linguagem central: ele permite que o geoprocessamento converse com outras partes do ecossistema computacional sem trocar de ambiente. O ponto central é que Python™ não é apenas “um jeito diferente de fazer *buffer*”; é um meio de organizar e executar o fluxo do começo ao fim, com rastreabilidade e consistência.

1.5 ECOSSISTEMA DE BIBLIOTECAS PARA GEOPROCESSAMENTO

Python, por si só, fornece a base da linguagem. O que o transforma em uma plataforma realmente poderosa para geoprocessamento é o conjunto de bibliotecas que fazem o trabalho pesado, cada uma com um papel bem definido. É útil enxergar esse ecossistema como peças que se encaixam em um fluxo de trabalho, em vez de uma lista de ferramentas desconectadas.

Para dados vetoriais, a GeoPandas permite trabalhar com tabelas (*GeoDataFrames*) que combinam atributos e geometrias, enquanto a Shapely oferece operações geométricas como *buffers*, interseções, distâncias e uniões. A leitura e escrita de formatos como *Shapefile* e *GeoPackage* é sustentada por bibliotecas como Fiona, normalmente usada “por baixo” da GeoPandas. Para dados *raster*, a Rasterio dá acesso à leitura, escrita e operações com *rasters* (como GeoTIFF), com forte integração ao GDAL, e a NumPy entra como base para tratar os valores como matrizes numéricas eficientes, sobre as quais realizamos cálculos e transformações.

Além disso, o PyProj é referência para transformações de coordenadas e conversões entre sistemas de referência. Na visualização, o Matplotlib funciona como base para gráficos e mapas estáticos. A Contextily facilita a adição de mapas base (*tiles*) e a Folium permite construir mapas interativos no navegador. Para análises espaciais mais avançadas, bibliotecas como PySAL reúnem ferramentas de vizinhança, autocorrelação e modelos espaciais.

Nem todas essas bibliotecas serão usadas com a mesma profundidade ao longo do livro. A proposta é selecionar um subconjunto que permita construir fluxos completos com dados espaciais sem dispersar o leitor. Por isso, as bibliotecas aparecem gradualmente e sempre associadas a uma necessidade concreta do fluxo: ler, conferir, processar, visualizar e exportar.

Com esse panorama, você já tem o contexto necessário para entender o que está por trás do geoprocessamento moderno e por que o Python™ se tornou uma linguagem tão presente nesse tipo de trabalho. Mais do que “aprender comandos”, a proposta do livro é mostrar como transformar dados geoespaciais em um fluxo de trabalho organizado: ler, conferir, processar, visualizar e exportar resultados de forma clara, reproduzível e fácil de manter.

A partir daqui o caminho deixa de ser apenas conceitual e passa a ser prático. No próximo capítulo, vamos entrar no ambiente em que todo esse fluxo vai acontecer ao longo do livro: o Google Colab®. Você vai aprender como funciona um notebook, como executar células de código com segurança e quais são os elementos mínimos de Python™ necessários para acompanhar os exemplos dos capítulos seguintes. A ideia é que, ao final do Capítulo 2, você

esteja pronto para abrir os notebooks do livro, rodar os primeiros códigos e avançar com confiança para os dados vetoriais e *rasters*.

1.6 AMBIENTES DE DESENVOLVIMENTO EM PYTHON™

Para utilizar Python™, precisamos de um ambiente onde possamos escrever código, executá-lo e inspecionar resultados. Existem diferentes opções, e cada uma atende a necessidades distintas, variando entre praticidade, recursos e exigência de instalação.

Notebooks Jupyter (locais ou em nuvem) permitem misturar texto, código, tabelas e mapas no mesmo documento, tornando o processo mais interativo. O Google Colaboratory® (Colab®) oferece uma experiência semelhante, mas em nuvem e sem necessidade de instalação local. Para projetos maiores, editores e IDEs como VS Code e PyCharm trazem recursos avançados de organização, depuração e manutenção. Também existem consoles em SIGs, como o console Python™ do QGIS®, úteis para automatizar tarefas diretamente dentro do software.

Neste livro, adotaremos principalmente o Google Colab® como ambiente padrão, por ser gratuito, acessível e adequado para fins didáticos. A escolha reforça o objetivo central: notebooks funcionam como um “caderno executável”, em que raciocínio e procedimento ficam registrados no mesmo lugar, favorecendo reprodutibilidade — você consegue executar novamente, ajustar parâmetros, refazer com outros dados e compartilhar o processo com clareza.

Os detalhes práticos (como abrir os notebooks do livro, instalar bibliotecas e organizar diretórios) serão tratados no capítulo sobre ambiente. Por enquanto, vale guardar a ideia que vai orientar o restante do texto: Python™ não é apenas código, mas um modo de transformar tarefas geoespaciais em um fluxo de trabalho confiável, repetível e sustentável.

CAPÍTULO 2

PRIMEIRO CONTATO COM PYTHON™ NO GOOGLE COLAB®

Abrir no Colab

https://colab.research.google.com/github/Alexandrogschafer/geoprocessamento-python-livro/blob/main/notebooks/geo_python_capitulo2.ipynb

Depois do panorama conceitual do capítulo anterior, chegou a hora de colocar a mão na massa. A partir daqui a proposta do livro é que você aprenda Python™ do jeito mais natural para quem está começando: lendo uma explicação curta, executando um bloco de código e observando o resultado imediatamente no notebook.

Neste capítulo, você vai entender como funciona o Google Colab® e o formato de notebook, que será usado em todos os exemplos do livro. Vamos ver a diferença entre células de texto (onde está a explicação) e células de código (onde o Python™ é executado), e como alternar entre ler, testar e ajustar comandos com segurança.

Ao longo do caminho, você vai rodar seus primeiros comandos em Python™ mesmo que nunca tenha programado antes. Em vez de aprofundar teoria, vamos introduzir apenas o essencial para acompanhar os capítulos seguintes: variáveis (para guardar valores), listas e dicionários (para organizar informações), repetições simples (para automatizar pequenas tarefas), funções (para reaproveitar código) e importação de bibliotecas (para ampliar o que o Python™ consegue fazer).

Os exemplos deste capítulo usam situações simples (como cálculos e pequenas coleções de dados) e, sempre que fizer sentido, adotam nomes e contextos que aparecem com frequência em projetos de geoprocessamento. A parte de trabalhar diretamente com dados geográficos, tabelas reais, gráficos e mapas entra a partir dos próximos capítulos, quando começaremos a usar bibliotecas como Pandas, GeoPandas e Matplotlib.

O objetivo é que, ao final do capítulo, você esteja confortável para abrir um notebook do livro, reconhecer rapidamente onde fica a explicação e onde você deve digitar, executar as células com segurança e entender o mínimo de Python™ necessário para seguir os exemplos que virão. Você não precisa de nenhuma experiência prévia em programação: a proposta aqui é começar do zero, com calma, e construir uma base sólida para avançar.

2.1 O QUE É O GOOGLE COLAB® E POR QUE VAMOS USÁ-LO

O Google Colab®, ou simplesmente Colab®, é um serviço gratuito do Google® que permite escrever e executar código Python™ diretamente no navegador, sem precisar instalar programas ou configurar o ambiente no computador. Ele funciona no formato de notebook, reunindo em um mesmo lugar o texto explicativo, os blocos de código e os resultados gerados — como tabelas, gráficos e mapas.

A escolha do Colab® para este livro está ligada, acima de tudo, à praticidade para quem está começando. Ao eliminar a etapa de instalação do Python™ e das bibliotecas, o Colab® reduz uma das maiores barreiras iniciais e permite que você comece a trabalhar imediatamente, usando apenas uma conta Google®. Além disso, o formato de notebook favorece o aprendizado e a documentação: cada etapa da análise fica registrada junto com a explicação e o código, o que facilita acompanhar, revisar e reproduzir os exemplos ao longo do livro.

Para apoiar quem nunca utilizou esse ambiente, o livro disponibiliza um tutorial introdutório sobre o Google Colab®, com um passo a passo da interface, da criação e configuração de notebooks e do uso das funções básicas necessárias para executar e acompanhar os exemplos. Esse material complementar está disponível em: [https://github.com/Alexandrogschafer/geoprocessamento-python-livro/blob/main/material adicional/Tutorial google colab.pdf](https://github.com/Alexandrogschafer/geoprocessamento-python-livro/blob/main/material%20adicional/Tutorial%20google%20colab.pdf)

Nos capítulos seguintes, vamos assumir o Google Colab® como ambiente de trabalho. Ainda assim, os códigos podem ser adaptados para outras opções, como um Jupyter Notebook local ou o VS Code, caso você já tenha mais experiência ou prefira outra configuração. Para acessar o Colab, basta abrir o navegador, entrar em <https://colab.research.google.com> e fazer login com uma conta Google. A tela inicial oferece opções para criar um novo notebook ou abrir arquivos existentes; você não precisa dominar tudo isso agora, pois os passos serão apresentados gradualmente ao longo do capítulo.

2.2 CONHECENDO A TELA DO NOTEBOOK

Quando você abre um notebook no Colab®, vê algo parecido com uma página em branco dividida em blocos (Figura 2.1). Esses blocos são chamados de células e é neles que você alterna entre ler a explicação e executar o código, mantendo tudo organizado no mesmo documento.

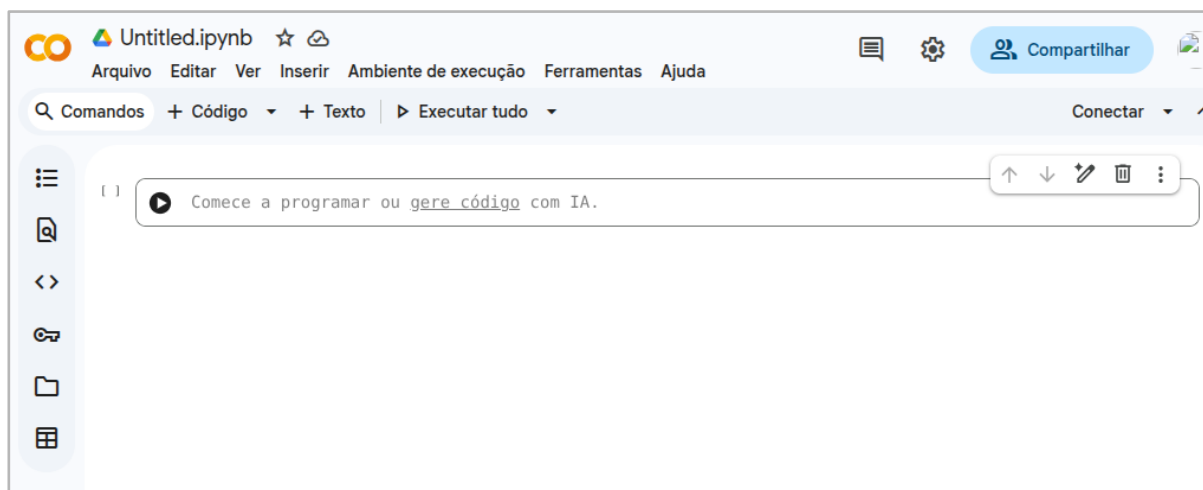


Figura 2.1 — Interface inicial de um notebook no Google Colab®.

Existem dois tipos principais de células. As células de texto (em Markdown) são usadas para títulos, instruções, comentários e explicações. Já as células de código são o espaço onde você digita comandos em Python™. Para executar uma célula de código, basta clicar dentro dela e pressionar Shift + Enter. O Colab® roda exatamente o que está naquela célula e mostra o resultado logo abaixo, que pode aparecer como texto, números, pequenas tabelas e gráficos simples — e, mais adiante no livro, também como mapas, quando começarmos a trabalhar com bibliotecas geoespaciais.

Ao longo deste livro, os notebooks seguem sempre a mesma lógica: primeiro vem um trecho de texto explicando o que será feito e por quê; em seguida, logo abaixo, aparece o bloco de código correspondente, que você pode executar e, se quiser, modificar para testar variações. Se algo der errado — como um erro de digitação ou um comando inválido — o Colab® exibirá uma mensagem de erro. Isso faz parte do processo de aprendizagem e, aos poucos, você vai aprender a reconhecer e interpretar os erros mais comuns para corrigir o caminho com mais segurança.

2.3 SEU PRIMEIRO CÓDIGO EM PYTHON™

Vamos começar com um exemplo simples, que cabe em qualquer contexto. Em uma célula de código do Colab®, escreva:

```
print("Geoprocessamento com Python!")
```

A Figura 2.2 ilustra esse momento inicial: o código foi digitado em uma célula ainda não executada. Note que, até esse ponto, nenhum resultado aparece na tela; a célula apenas contém o comando que será interpretado pelo Python™.

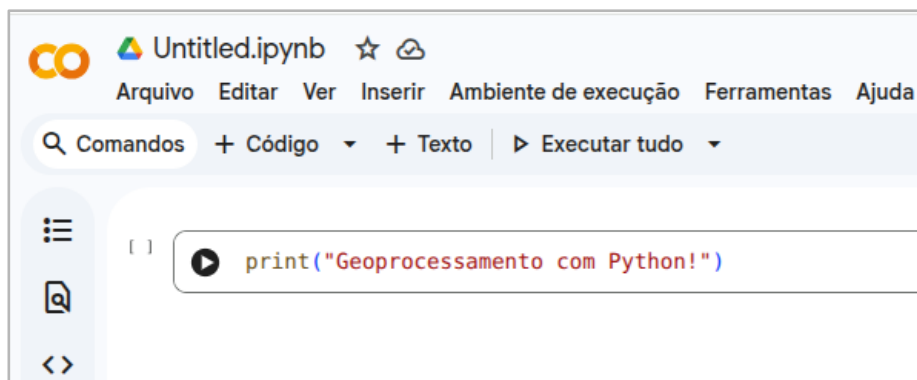


Figura 2.2 — Primeira célula de código em um notebook do Google Colab® antes da execução.

Para executar o código, pressione Shift + Enter no teclado. Esse atalho executa a célula atual e, em seguida, avança automaticamente para a próxima. Se tudo ocorrer corretamente, o texto será exibido logo abaixo da célula executada, como mostrado na Figura 2.3.

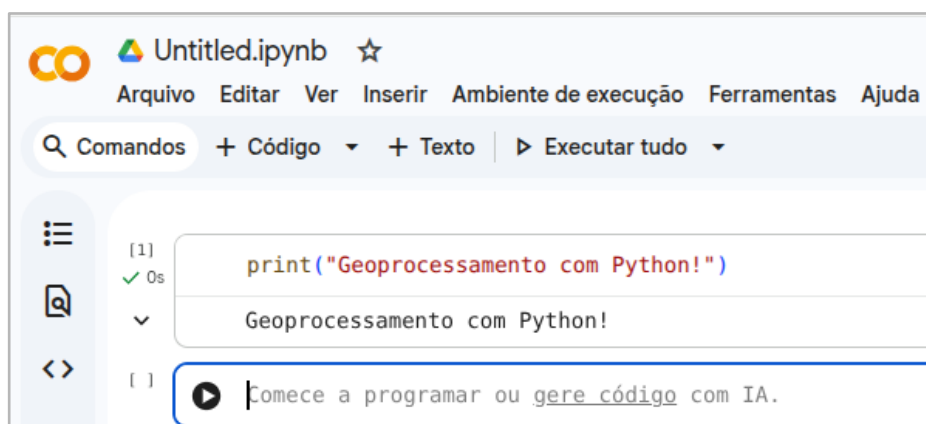


Figura 2.3 — Resultado da execução da primeira célula de código no Google Colab®.

Esse pequeno exemplo já ilustra a ideia fundamental do notebook: você escreve um comando em Python™, o Colab® executa esse comando e o resultado aparece imediatamente. Essa lógica — escrever, executar e observar o resultado — vai se repetir ao longo de todo o livro.

A partir disso, podemos avançar para algo um pouco mais “numérico”. Em uma nova célula, experimente:

```
2 + 2
```

Execute com Shift + Enter. O resultado deve ser 4. Python™ também pode fazer contas mais complexas; por exemplo:

```
(50 + 30) / 4
```

Ao executar, você verá o resultado da expressão (20).

Esses primeiros exemplos podem parecer simples, mas cumprem um papel importante: eles mostram que o notebook é um espaço interativo, no qual você pode testar ideias aos poucos e explorar comandos sem medo. Não há problema nenhum em experimentar, errar e ajustar — esse processo faz parte do aprendizado e também do uso cotidiano do Python™. Nos próximos tópicos, vamos dar nome aos elementos básicos que aparecerão em todos os programas, construindo gradualmente a base necessária para você entender e escrever códigos com mais segurança.

2.4 INDENTAÇÃO E SÍMBOLOS BÁSICOS EM PYTHON

Antes de avançarmos para programas um pouco mais elaborados, vale entender dois aspectos fundamentais do Python™: a indentação e alguns símbolos básicos que aparecem o tempo todo no código. Esses elementos ajudam o Python™ a interpretar corretamente o que você escreveu e, ao mesmo tempo, tornam o programa mais legível para quem está lendo.

Em Python™, a indentação — isto é, o uso de espaços no início das linhas — não é apenas uma questão de organização visual. Ela faz parte da própria sintaxe da linguagem. Na prática, o Python™ usa esses espaços para identificar quais linhas pertencem a um mesmo bloco de código. Mais adiante, quando trabalharmos com repetições, condições e funções, você verá trechos “entrando” um pouco para a direita: isso indica que aquelas instruções fazem parte da mesma estrutura.

Neste início, a maior parte dos exemplos será escrita de forma linear, sem blocos indentados, para facilitar o acompanhamento. Ainda assim, é importante saber desde já que erros de indentação são comuns quando estamos começando. Se o Python™ encontrar espaços a mais, a menos ou mal posicionados, ele exibirá uma mensagem de erro. Isso não significa que você “não sabe programar”; significa apenas que o bloco precisa ser ajustado — algo absolutamente normal no processo de aprendizagem.

Além da indentação, alguns símbolos aparecem com muita frequência. O sinal de igual (=) é usado para atribuir valores a variáveis. Diferentemente da matemática tradicional, ele não representa igualdade; ele significa “guardar” um valor em um nome. Os operadores +, -, * e / são usados para soma, subtração, multiplicação e divisão, como você já viu nos exemplos numéricos.

Os parênteses (()) também aparecem o tempo todo. Eles servem para agrupar expressões e deixar clara a ordem dos cálculos, e também para chamar funções, como no comando `print()`. Textos (strings) são escritos entre aspas simples ou duplas e precisam sempre começar e terminar corretamente para que o Python™ reconheça aquele conteúdo como texto.

Outro símbolo importante é o #, que indica um comentário: tudo o que aparece depois dele na mesma linha é ignorado pelo Python™. Comentários são úteis para explicar o que o código faz, registrar observações ou deixar lembretes para você mesmo. Ao longo do livro, vamos usá-los com frequência para tornar os exemplos mais claros.

Por enquanto, não é necessário memorizar regras ou detalhes técnicos. O mais importante é reconhecer esses elementos quando eles aparecem e entender, de forma geral, o papel de cada um. Com a prática, a indentação e esses símbolos vão se tornar naturais, quase automáticos, à medida que você escreve e executa seus próprios códigos.

2.5 VARIÁVEIS E TIPOS DE DADOS

Uma variável é um nome que damos a um valor para poder reutilizá-lo depois. Em vez de repetir o mesmo número (ou texto) várias vezes no código, você guarda esse valor em um nome e passa a trabalhar com ele de forma mais clara. Por exemplo:

```
populacao = 121143
```

```
area_km2 = 4095
```

Aqui, criamos duas variáveis: `populacao` guarda o número 121143 e `area_km2` guarda o número 4095. A partir disso, podemos usar essas variáveis em uma conta e guardar o resultado em uma terceira variável:

```
densidade = populacao / area_km2  
densidade
```

Saída: 29.583150183150185

Ao executar, o notebook mostra automaticamente o valor da última expressão da célula. Se quisermos deixar a saída mais amigável, podemos usar o comando `print`:

```
print("Densidade populacional:", densidade, "hab/km²")  
Saída: Densidade populacional: 29.583150183150185 hab/km²
```

Além do conceito de variável, é importante perceber que Python™ lida com diferentes tipos de dados. Neste início, vamos usar principalmente números e textos. Números inteiros representam valores sem casas decimais, números decimais representam valores com casas decimais e textos são sequências de caracteres escritas entre aspas.

Veja um exemplo simples com textos:

```
cidade = "Brasília"  
estado = "DF"  
print("Cidade:", cidade, "-", estado)  
Saída esperada: Cidade: Brasília - DF
```

Você não precisa decorar nomes técnicos como `int`, `float` ou `str` agora. O mais importante, neste momento, é entender que variáveis servem para guardar valores, que esses valores podem ser reutilizados em contas, textos e funções, e que escolher nomes claros — como `area_km2`, `populacao` e `cidade` — torna o código mais fácil de ler e de manter.

2.6 ESTRUTURAS DE DADOS BÁSICAS: LISTAS, TUPLAS E DICIONÁRIOS

Até aqui, cada variável que usamos guardava apenas um valor. Em muitas situações, porém, precisamos trabalhar com conjuntos de informações relacionadas: uma lista de cidades, um grupo de arquivos, uma sequência de anos ou um conjunto de classes temáticas. Para isso, o Python™ oferece estruturas que guardam vários valores de uma vez.

As três mais comuns neste início são listas, tuplas e dicionários. Listas e tuplas organizam valores em sequência (primeiro, segundo, terceiro...), enquanto dicionários organizam informações como um mapeamento do tipo chave → valor. A diferença prática é simples: em listas e tuplas você acessa um item pela posição. Em dicionários você acessa pela chave. Neste capítulo, vamos usar mais listas e dicionários. Tuplas aparecem aqui apenas como referência: elas se parecem com listas, mas são mais “fixas”, isto é, normalmente não são usadas para adicionar/remover itens ao longo do código.

2.6.1 LISTAS

Uma lista é criada com colchetes ([]) e permite armazenar vários valores em uma única variável. Por exemplo, podemos guardar nomes de cidades e anos assim:

```
idades = ["Porto Alegre", "São Paulo", "Salvador", "Manaus"]  
anos = [2000, 2010, 2020]
```

Se você executar a célula e depois digitar o nome de uma dessas variáveis, o notebook mostra o conteúdo da lista:

```
idades
```

```
Saída: ['Porto Alegre', 'São Paulo', 'Salvador', 'Manaus']
```

Cada elemento ocupa uma posição específica, chamada de índice. Em Python™, a contagem começa em zero, e não em um. Isso significa que o primeiro elemento está na posição 0, o segundo na posição 1 e assim por diante:


```
idades[0]
```

Saída: 'Porto Alegre'

```
idades[2]
```

Saída: 'Salvador'

Também é possível adicionar novos elementos a uma lista já existente. O método `append()` insere um novo valor ao final da lista:

```
idades.append("Goiânia")
```

```
idades
```

Saída: ['Porto Alegre', 'São Paulo', 'Salvador', 'Manaus', 'Goiânia']

Outro uso muito comum é percorrer todos os elementos, um a um, para repetir uma ação. Isso é feito com o comando `for`, que veremos com mais calma na próxima seção. Por enquanto, observe a ideia:

```
for nome in cidades:
```

```
    print("Analisando dados de:", nome)
```

Saída:

Analisando dados de: Porto Alegre

Analisando dados de: São Paulo

Analisando dados de: Salvador

Analisando dados de: Manaus

Analisando dados de: Goiânia

No contexto do geoprocessamento, listas aparecem o tempo todo: para iterar sobre municípios, percorrer camadas (limites, hidrografia, rodovias), organizar múltiplas bandas de uma imagem ou montar sequências de anos em análises temporais.

2.6.2 TUPLAS

Uma tupla é muito parecida com uma lista, mas é criada com parênteses (). Na prática, ela costuma ser usada quando queremos representar um conjunto de valores que não deve ser alterado ao longo do código.

```
coordenada = (-54.10, -31.33)
```

```
coordenada
```

Saída esperada: (-54.1, -31.33)

Por enquanto, basta reconhecer que tuplas existem e têm esse “comportamento fixo”. No restante do livro, quando precisarmos de sequências alteráveis, as listas serão a escolha mais comum.

2.6.3 DICIONÁRIOS

Enquanto listas e tuplas organizam valores em sequência, um dicionário organiza informações como um mapeamento do tipo chave → valor. Em vez de acessar um item pela posição, acessamos pela chave, que funciona como uma etiqueta única.

Podemos pensar em um dicionário como uma pequena tabela em que cada chave aponta para um valor correspondente. No exemplo abaixo, guardamos áreas aproximadas (em km²) associadas a nomes de capitais:

```
area_mun = {"Porto Alegre": 496, "São Paulo": 1521, "Salvador": 693}
```

Se você digitar o nome da variável, o notebook mostra o conteúdo do dicionário:

```
area_mun
```

Saída: {'Porto Alegre': 496, 'São Paulo': 1521, 'Salvador': 693}

Para acessar a área de Salvador, usamos a chave entre colchetes:

```
area_mun["Salvador"]
```

Saída: 693

Também é possível adicionar novos pares a qualquer momento, criando uma nova chave:

```
area_mun["Goiânia"] = 739
area_mun
```

Saída: {'Porto Alegre': 496, 'São Paulo': 1521, 'Salvador': 693, 'Goiânia': 739}

Mais adiante, ao trabalharmos com dados categóricos (classes numéricas em tabelas e *rasters*), será comum usar dicionários para relacionar códigos a nomes legíveis. Um exemplo típico seria:

```
classes = {1: "Corpo d'água", 2: "Área não vegetada", 3: "Formação florestal", 4: "Formação
campestre"}
classes
```

Saída:

```
{1: "Corpo d'água", 2: 'Área não vegetada', 3: 'Formação florestal', 4: 'Formação campestre'}
```

Esse tipo de estrutura permite guardar internamente os códigos (1, 2, 3, 4), mas exibir nomes claros para interpretação, construção de legendas e comunicação de resultados — algo que aparece com frequência em geoprocessamento.

2.7 ESTRUTURAS DE REPETIÇÃO

Imagine que você tem uma lista de municípios e quer gerar um resultado para cada um deles — por exemplo, calcular um indicador, recortar um *raster* ou produzir um mapa. Fazer isso manualmente, um por um, seria trabalhoso e sujeito a erros. Para resolver esse tipo de tarefa, Python™ oferece estruturas de repetição, que permitem executar o mesmo conjunto de instruções várias vezes de maneira automática.

Em Python™ existem diferentes formas de repetição, mas, neste livro, vamos trabalhar principalmente com o laço `for`. Ele é o mais comum, costuma ser o mais legível e é suficiente

para a maioria das situações práticas que vamos encontrar no geoprocessamento. Por esse motivo, não vamos introduzir agora outras variações (como while, break e continue) nem entrar em teoria de laços: a ideia é aprender o padrão que você realmente vai usar com frequência.

O for é particularmente útil quando temos uma coleção de itens — como uma lista — e queremos aplicar a mesma ação a cada elemento. Por exemplo:

```
idades = ["Porto Alegre", "São Paulo", "Salvador", "Manaus"]  
for cidade in cidades:  
    print("Gerando mapa para:", cidade)
```

Saída:

```
Gerando mapa para: Porto Alegre  
Gerando mapa para: São Paulo  
Gerando mapa para: Salvador  
Gerando mapa para: Manaus
```

Ao executar, o Python™ repete o bloco print(...) para cada item da lista, um de cada vez. O mesmo vale para outras coleções, como anos de referência:

```
anos = [2000, 2010, 2020]  
for ano in anos:  
    print("Processando dados do ano:", ano)
```

Saída:

```
Processando dados do ano: 2000  
Processando dados do ano: 2010  
Processando dados do ano: 2020
```

No geoprocessamento, estruturas de repetição como o for são fundamentais para automatizar tarefas repetitivas: aplicar o mesmo procedimento a vários arquivos matriciais, gerar mapas em série (um mapa por município, por ano ou por classe) e calcular estatísticas para diversas unidades espaciais (por exemplo, todos os municípios de uma determinada

região). A ideia central é simples: você descreve a sequência de passos uma única vez, e o for se encarrega de repetir esses passos para cada item da lista.

2.8 FUNÇÕES: AGRUPANDO PASSOS EM BLOCOS REUTILIZÁVEIS

A medida que os notebooks ficam maiores, o código pode se tornar difícil de acompanhar se tudo for escrito como um único bloco contínuo. Para manter a organização, usamos funções, que são blocos de código com um nome próprio. Em termos práticos, uma função reúne uma sequência de passos que você pode reaproveitar sempre que precisar, sem ter que copiar e colar o mesmo trecho várias vezes.

Uma função pode receber entradas (chamadas de parâmetros), executar um conjunto de comandos e, quando fizer sentido, devolver um resultado. No exemplo abaixo, criamos uma função para calcular densidade populacional a partir de população e área:

```
def calcular_densidade(populacao, area_km2):
```

```
    densidade = populacao / area_km2
```

```
    return densidade
```

Depois de definida, a função pode ser usada quantas vezes forem necessárias:

```
dens_poa = calcular_densidade(1484941, 496)
```

```
print("Densidade populacional de Porto Alegre:", dens_poa, "hab/km2")
```

Saída:

```
Densidade populacional de Porto Alegre: 2993.8326612903224 hab/km2
```

Agora imagine que você queira repetir o mesmo cálculo para várias capitais. Ao invés de print (escrever a conta para cada uma, podemos organizar os dados em um dicionário e percorrê-los com um for. No exemplo a seguir, cada município tem dois valores associados — população e área — e esses dois valores aparecem agrupados entre parênteses.

```
municipios = {
```

```
    "Porto Alegre": (1484941, 496), # população, área em km2
```

```
    "São Paulo": (11316191, 1521),
```

```
"Salvador": (2675656, 693),  
"Manaus": (2020301, 11401)  
}
```

```
for nome, (pop, area) in municipios.items():  
    dens = calcular_densidade(pop, area)  
    print("Densidade de", nome, ":", dens, "hab/km2")
```

Saída:

Densidade de Porto Alegre : 2993.8326612903224 hab/km²

Densidade de São Paulo : 7441.072320841552 hab/km²

Densidade de Salvador : 3860.109668831169 hab/km²

Densidade de Manaus : 177.2019998245768 hab/km²

No contexto deste livro, funções serão úteis para encapsular rotinas que se repetem, como abrir dados sempre a partir de um mesmo diretório, padronizar procedimentos de plotagem para gerar mapas com a mesma aparência ou concentrar trechos mais longos em um único lugar. Embora você não precise criar funções sofisticadas agora, é importante entender que elas ajudam a organizar o raciocínio, evitando repetição desnecessária e facilitando a manutenção do código.

2.9 IMPORTANDO BIBLIOTECAS: AMPLIANDO O PODER DO PYTHON™

O Python™, por si só, já permite realizar muitas tarefas. Mas, na prática, o que amplia de verdade o seu potencial é o uso de bibliotecas: conjuntos de códigos prontos, desenvolvidos por outras pessoas, que você reutiliza em vez de “reinventar a roda”.

Para usar uma biblioteca, você precisa importá-la. Veja um exemplo simples com a biblioteca `math`, que traz funções matemáticas prontas:

```
import math  
math.sqrt(16)
```

Saída: 4.0

Ao longo deste livro, algumas bibliotecas aparecerão com muita frequência. Por isso, é comum importá-las usando apelidos curtos, que deixam o código mais limpo e fácil de ler:

```
import pandas as pd  
import geopandas as gpd  
import matplotlib.pyplot as plt
```

Nesse padrão, *pd* será usado para trabalhar com tabelas, *gpd* para lidar com dados vetoriais espaciais e *plt* para criar gráficos e mapas. Esses apelidos não são obrigatórios, mas são amplamente usados na comunidade Python™ e ajudam a reconhecer rapidamente o papel de cada biblioteca no código.

O fluxo geral será sempre o mesmo: primeiro importamos as bibliotecas necessárias no início do notebook e, em seguida, utilizamos seus recursos ao longo das células. A partir dos próximos capítulos, essas bibliotecas passam a aparecer de forma constante, agora aplicadas a dados geográficos reais e análises passo a passo.

CAPÍTULO 3

ROTINA DE TRABALHO E ORGANIZAÇÃO DOS NOTEBOOKS NO GOOGLE COLAB®

Neste capítulo, vamos estabelecer a rotina de trabalho que será repetida ao longo de todo o livro. A ideia aqui não é fazer análises geoespaciais ainda, mas garantir que você saiba como iniciar um notebook do livro no Google Colab®, onde organizar os arquivos e como manter o fluxo de trabalho consistente.

No Capítulo 2, você teve o primeiro contato com o Python™ e com a lógica dos notebooks (texto + código + resultado). Agora damos um passo importante para evitar travas nos capítulos seguintes: definir um padrão simples de organização que funcione bem no Colab®, um ambiente em que tudo começa “do zero” a cada sessão.

A partir deste ponto, cada capítulo do livro terá um notebook próprio. Esses notebooks são independentes, mas seguem sempre a mesma estrutura: primeiro preparamos o ambiente, depois trazemos apenas os dados necessários para aquele capítulo, executamos a análise passo a passo e salvamos os resultados. O objetivo é que você se acostume com esse padrão e consiga se concentrar no que realmente importa: entender os dados, o código e os mapas que vamos produzir mais adiante.

Ao final do capítulo, você estará confortável para abrir qualquer notebook do livro, executar as células iniciais na ordem correta, reconhecer onde ficam os dados e onde salvar os resultados — e também resolver os erros mais comuns que aparecem quando algo é executado fora de sequência.

3.1 COMO OS NOTEBOOKS DO LIVRO SÃO ORGANIZADOS

Cada capítulo do livro possui um notebook próprio. Esses notebooks são independentes entre si, mas seguem sempre a mesma lógica de organização. Em vez de depender de arquivos já existentes no ambiente, cada notebook cria suas próprias pastas e traz apenas os dados necessários para aquele capítulo.

De forma geral, todo notebook do livro começa criando duas pastas locais:
dados/: armazena os arquivos utilizados como entrada na análise;
resultados/: armazena mapas, tabelas, figuras e *rasters* gerados ao longo do capítulo.

Essa organização simples ajuda a manter o projeto legível e evita misturar arquivos de entrada com resultados intermediários ou finais.

3.2 O BLOCO INICIAL DE TODO NOTEBOOK: PASTAS, DADOS E VERIFICAÇÃO

Sempre que um notebook é aberto no Google Colab®, o ambiente começa vazio. Por isso, adotamos uma rotina padrão que será repetida em todos os capítulos práticos:

1. Criar as pastas dados/ e resultados/;
2. Baixar os arquivos necessários a partir do repositório do livro ou de outras fontes indicadas;
3. Verificar se os arquivos estão disponíveis antes de iniciar a análise;
4. Instalar (quando necessário) e importar as bibliotecas utilizadas.

Nos capítulos seguintes, essa rotina aparecerá logo no início do notebook, com código específico para os dados utilizados naquele capítulo. Aqui, o objetivo é apenas entender o padrão geral.

Para criar as pastas locais no notebook, utilizamos o módulo `pathlib`, que facilita o trabalho com caminhos de arquivos:

```
from pathlib import Path
```

```
dados_dir = Path("dados")
```

```
dados_dir.mkdir(exist_ok=True)
```

```
resultados_dir = Path("resultados")
```

```
resultados_dir.mkdir(exist_ok=True)
```

O uso de `Path` evita erros comuns de digitação e torna o código mais legível, especialmente quando lidamos com vários arquivos ao longo do capítulo.

3.3 BIBLIOTECAS NO COLAB®: INSTALAR QUANDO NECESSÁRIO E IMPORTAR

O Google Colab® já vem com diversas bibliotecas Python™ instaladas, mas não é recomendável depender do ambiente padrão, pois versões e dependências podem mudar ao longo do tempo. Por isso, sempre que um notebook exige bibliotecas que não vêm garantidas no Colab®, incluímos explicitamente uma célula de instalação no início do capítulo.

Nos capítulos práticos de geoprocessamento, isso normalmente envolve bibliotecas como GeoPandas, Rasterio e Matplotlib. Após a instalação, as bibliotecas são importadas e utilizadas ao longo do notebook.

Caso ocorra algum erro ao importar bibliotecas após a instalação, o procedimento recomendado é reiniciar o ambiente de execução e executar novamente as células desde o início. Essa prática resolve a maioria dos problemas iniciais no Colab®.

3.4 ONDE SALVAR RESULTADOS E COMO RESOLVER ERROS COMUNS

Todos os arquivos criados ou baixados durante uma sessão do Google Colab® ficam armazenados temporariamente. Ao encerrar a sessão, esses arquivos são apagados. Por isso, sempre que um resultado for importante, ele deve ser baixado manualmente ou salvo em outro local.

Alguns erros são comuns quando se trabalha nesse ambiente. Entre os mais frequentes estão:

`ModuleNotFoundError`, indicando que uma biblioteca necessária não está disponível na sessão;

`FileNotFoundError`, quando o código tenta acessar um arquivo que ainda não foi baixado ou cujo caminho está incorreto;

execução fora de ordem, como tentar ler dados antes de preparar o ambiente.

Na maioria dos casos, esses problemas são resolvidos voltando ao início do notebook e executando as células na sequência correta. Ao longo do livro, sempre que um novo tipo de erro puder surgir, ele será explicado no contexto em que aparece.

Com essa organização, todos os capítulos do livro seguem uma estrutura previsível: preparar o ambiente, trazer os dados necessários, realizar a análise e salvar os resultados. Essa

previsibilidade é intencional e faz parte da proposta didática do livro, permitindo que você se concentre no raciocínio geoespacial e na interpretação dos resultados, e não na configuração do ambiente.

CAPÍTULO 4

FUNDAMENTOS DE DADOS VETORIAIS COM GEOPANDAS

Abrir no Colab

https://colab.research.google.com/github/Alexandrogschafer/geoprocessamento-python-livro/blob/main/notebooks/geo_python_capitulo4.ipynb

Nos capítulos anteriores, você conheceu a linguagem Python™, o ambiente de trabalho no Google Colab® e a rotina básica dos notebooks do livro. A partir deste capítulo, começamos a trabalhar diretamente com dados geoespaciais, introduzindo o modelo vetorial e as ferramentas que permitem manipular esse tipo de informação em Python™.

O objetivo aqui é construir uma base sólida: entender como dados vetoriais são organizados, como o GeoPandas estrutura essas informações em tabelas, como as geometrias são representadas e quais operações básicas podem ser realizadas sobre elas. Ao longo do capítulo, trabalharemos com dois tipos de exemplo complementares: (1) um arquivo vetorial real, contendo as sub-bacias da área de estudo, para aprender a inspecionar e manipular *GeoDataFrames*; e (2) geometrias simples criadas em código, para compreender o funcionamento do modelo vetorial sem depender de dados externos.

4.1 PREPARAÇÃO DO NOTEBOOK

Antes de começar, vamos preparar o notebook seguindo a rotina padrão do livro: criar uma pasta local para os dados, baixar o arquivo necessário do repositório e confirmar que ele está disponível no ambiente. Essa pequena etapa evita erros de caminho e garante que todo mundo execute exatamente o mesmo material.

```
from pathlib import Path
# Pasta padrão para dados
dados_dir = Path("dados")
dados_dir.mkdir(exist_ok=True)
```

```

# URL RAW do GeoPackage das subbacias
url_subbacias = (
    "https://raw.githubusercontent.com/"
    "Alexandrogschafer/geoprocessamento-python-livro/main/"
    "dados/subbacias.gpkg"
)

subbacias_path = dados_dir / "subbacias.gpkg"

# Download do arquivo
!wget -q -O "{subbacias_path}" "{url_subbacias}"

if not subbacias_path.exists():
    raise FileNotFoundError(
        "Falha no download do arquivo de subbacias. Verifique a URL e tente novamente."
    )

print("Arquivo das subbacias disponível:", subbacias_path.exists())

#Saída: Arquivo das subbacias disponível: True

```

4.2 O MODELO VETORIAL E O QUE É UM GEODATAFRAME

O modelo vetorial representa feições geográficas discretas — objetos que conseguimos identificar no espaço como pontos, linhas e polígonos. Um ponto pode ser uma ocorrência ou um endereço; uma linha pode representar uma rua ou um rio; e um polígono descreve uma área fechada, como um lote, um bairro, um município ou uma bacia hidrográfica.

No GeoPandas, dados vetoriais são organizados em uma estrutura chamada GeoDataFrame. A forma mais direta de entendê-la, no início, é pensar em uma tabela: cada linha é uma feição, cada coluna é um atributo e existe uma coluna especial chamada geometry, onde fica a geometria associada àquela feição. É essa combinação (atributos + geometria) que torna

possível trabalhar com dados espaciais em Python™ de modo prático: você inspeciona colunas, filtra registros, cria novos campos e salva resultados sem perder o componente espacial.

Para enxergar essa ideia na prática, vamos carregar um dado real: um *GeoPackage* com as sub bacias da área de estudo. Nesta etapa, o objetivo ainda não é analisar as sub bacias em detalhe, mas fazer as checagens iniciais mais importantes: verificar a estrutura da tabela, a quantidade de feições, o tipo de geometria e o CRS.

```
import geopandas as gpd
gdf_sub = gpd.read_file(subbacias_path)
gdf_sub.head()
```

	nome	id	geometry
0	bacia arroio tábua	1	MULTIPOLYGON (((778104.925 6538133.882, 778254...
1	bacia alto bagé direita	4	MULTIPOLYGON (((784404.925 6539333.882, 784464...
2	bacia alto bagé esquerda	3	MULTIPOLYGON (((779604.925 6540233.882, 779724...
3	bacia arroio gontan	2	MULTIPOLYGON (((772404.925 6534773.882, 772464...

Ao executar, repare que os atributos aparecem como colunas comuns e que a coluna *geometry* armazena as geometrias. Como cada linha corresponde a uma feição, vale conferir quantas feições existem na camada:

```
len(gdf_sub)
#Saída: 4
```

Em seguida, uma checagem simples ajuda a confirmar o tipo de geometria presente (ponto, linha, polígono ou versões “multi”):

```
gdf_sub.geom_type.unique()
#Saída: array(['MultiPolygon'], dtype=object)
```

Outro passo essencial é conferir o sistema de referência de coordenadas (CRS). Ele define em que sistema espacial as coordenadas estão expressas e influencia diretamente cálculos de área, distância e sobreposição:

```
gdf_sub.crs
```

```
#Saída:
```

```
<Projected CRS: EPSG:31981> Name: SIRGAS 2000 / UTM zone 21S Axis Info [cartesian]: - E[east]:  
Easting (metre) - N[north]: Northing (metre). Area of Use: - name: Brazil - between 60°W and  
54°W, northern and southern hemispheres. In remainder of South America - between 60°W and  
54°W, southern hemisphere, onshore and offshore. - bounds: (-60.0, -44.82, -54.0, 4.51) Coordinate  
Operation: - name: UTM zone 21S - method: Transverse Mercator Datum: Sistema de Referencia  
Geocentrico para las AmericaS 2000 - Ellipsoid: GRS 1980 - Prime Meridian: Greenwich
```

Neste momento, não vamos calcular áreas das sub bacias ainda. Primeiro, precisamos garantir um CRS adequado para medidas (em metros), o que será tratado mais adiante no livro. Por enquanto, o objetivo é criar o hábito de sempre checar o CRS ao abrir um dado vetorial.

Por fim, antes de qualquer operação, é útil produzir um primeiro mapa apenas para confirmar que a camada foi lida corretamente. Aqui ainda não estamos preocupados com estética nem com mapas “prontos”; a ideia é só visualizar as feições de forma rápida e segura.

```
ax = gdf_sub.plot(edgecolor="black", facecolor="none", linewidth=1.5)  
ax.set_title("Subbacias — visualização inicial")  
ax.set_axis_off()
```



Figura 4.1 — Sub bacias em um *GeoDataFrame* (primeiro contato).

Com essa camada aberta e checada, agora podemos voltar para um exemplo controlado e criar geometrias do zero. Isso facilita entender, sem distrações, como pontos, linhas e polígonos são construídos e manipulados no GeoPandas.

4.3 GEOMETRIAS BÁSICAS: PONTO, LINHA E POLÍGONO

Até aqui, você já viu um GeoDataFrame vindo de um arquivo real (as sub bacias) e percebeu a estrutura central do GeoPandas: colunas com atributos e uma coluna especial chamada geometry. Agora vale dar um passo importante para consolidar o modelo vetorial: criar geometrias do zero, diretamente em código. Essa abordagem é útil porque nos permite entender o que é, de fato, um ponto, uma linha e um polígono — sem depender de dados externos.

No GeoPandas, as feições vetoriais são representadas por objetos geométricos definidos pela biblioteca Shapely. Os tipos mais comuns são ponto (uma localização), linha (uma feição linear formada por uma sequência de coordenadas) e polígono (uma área fechada delimitada por um contorno). Em bases reais, também aparecem tipos “multi” (como MultiPolygon), usados quando uma mesma feição é composta por várias partes. Outro detalhe é que polígonos podem ter “buracos” (anéis internos), mas por enquanto vamos ficar no essencial.

Vamos criar um ponto, uma linha e um polígono e organizá-los em um GeoDataFrame.

```
import matplotlib.pyplot as plt
from shapely.geometry import Point, LineString, Polygon

# 1) Ponto
ponto = Point(2, 4)

# 2) Linha
linha = LineString([(0, 0), (2, 2), (4, 1)])

# 3) Polígono (deslocado no eixo x para não ficar sobreposto)
poligono = Polygon([(6, 0), (11, 0), (12, 3), (8, 4), (6, 2)])

# Organiza em um GeoDataFrame
gdf = gpd.GeoDataFrame(
    {"tipo": ["Ponto", "Linha", "Polígono"]},
    geometry=[ponto, linha, poligono]
)

gdf
```

#Saída:

	tipo	geometry
0	Ponto	POINT (2 4)
1	Linha	LINESTRING (0 0, 2 2, 4 1)
2	Polígono	POLYGON ((6 0, 11 0, 12 3, 8 4, 6 2, 6 0))

Ao executar, você verá a estrutura tabular do GeoDataFrame e a coluna geometry contendo as geometrias (em formato textual). O próximo passo é visualizar essas feições em um mapa simples:

```
ax = gdf.plot(edgecolor="black", facecolor="none")
ax.set_title("Geometrias vetoriais básicas")
ax.set_axis_off()
plt.show()
```

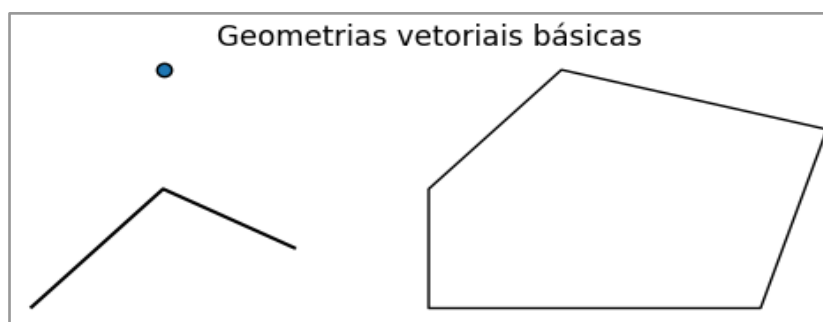


Figura 4.2 — Geometrias vetoriais básicas (ponto, linha e polígono).

A partir daqui, sempre que você carregar um dado vetorial real (como as subbacias), ele seguirá exatamente essa lógica. O que muda são as coordenadas, o CRS, os atributos e a quantidade de feições — não a estrutura do modelo.

4.4 OPERAÇÕES GEOMÉTRICAS ESSENCIAIS: MEDIDAS E RELAÇÕES ESPACIAIS (ENXUGADA)

Uma das vantagens de trabalhar com geometrias como objetos em Python™ é poder calcular propriedades e testar relações espaciais diretamente pela coluna geometry. Nesta seção, vamos aplicar operações simples no GeoDataFrame gdf que acabamos de criar.

Como as coordenadas aqui são artificiais, área e comprimento estão em “unidades do plano”. Em dados reais, esses cálculos só fazem sentido quando a camada está em um CRS projetado (em metros), o que veremos mais adiante.

Começamos com quatro propriedades comuns: área, comprimento, centroide e envelope (o retângulo envolvente, também chamado de *bounding box*).

```
gdf["area"] = gdf.geometry.area
gdf["comprimento"] = gdf.geometry.length
gdf["centroide"] = gdf.geometry.centroid
gdf["bbox"] = gdf.geometry.envelope
gdf
```

#Saída:

	tipo	geometry	area	comprimento	centroide	bbox
0	Ponto	POINT (2 4)	0.0	0.000000	POINT (2 4)	POINT (2 4)
1	Linha	LINESTRING (0 0, 2 2, 4 1)	0.0	5.064495	POINT (1.88304 1.22076)	POLYGON ((0 0, 4 0, 4 2, 0 2, 0 0))
2	Polígono	POLYGON ((6 0, 11 0, 12 3, 8 4, 6 2, 6 0))	18.5	17.113810	POINT (8.85586 1.75676)	POLYGON ((6 0, 12 0, 12 4, 6 4, 6 0))

Repare que centroide e bbox também são geometrias: o GeoPandas permite manter colunas geométricas auxiliares além da geometria principal.

Se quiser confirmar qual coluna é a geometria ativa (a usada por padrão em operações como `plot()`), faça:

```
gdf.geometry.name
#Saída: geometry
```

Uma forma simples de “mexer” na geometria como objeto é extrair uma informação do polígono: o número de coordenadas do contorno externo. Como esse cálculo só faz sentido para *Polygon*, aplicamos apenas nesse caso. Note que o contorno fecha repetindo a primeira coordenada no final, então esse número pode aparecer “um a mais” do que o número de vértices distintos.

```
gdf["n_vertices"] = gdf.geometry.apply(
    lambda geom: len(geom.exterior.coords) if geom.geom_type == "Polygon" else None
```

)

```
gdf[["tipo", "n_vertices"]]
```

#Saída:

	tipo	n_vertices
0	Ponto	NaN
1	Linha	NaN
2	Polígono	6.0

Além de medidas (área, comprimento), outra família de operações muito usada em geoprocessamento são as relações topológicas. Em vez de devolver um número, elas devolvem True/False para perguntas do tipo:

contains (contém?): o ponto está dentro da geometria (não vale estar apenas na borda);

intersects (intersecta?): existe qualquer contato/interseção entre as geometrias (inclusive tocar na borda);

touches (toca?): encosta exatamente na borda, mas não entra no interior.

Para enxergar isso com clareza, vamos criar dois pontos de referência:

Ponto A (dentro): está no interior do polígono;

Ponto B (borda): cai exatamente em um vértice do polígono (ou seja, na borda).

```
from shapely.geometry import Point
```

```
p_dentro = Point(7, 2) # Ponto A (dentro do polígono)
```

```
p_borda = Point(6, 0) # Ponto B (um vértice do polígono)
```

Antes de testar as relações, vamos plotar as geometrias e marcar os dois pontos no mapa.

```
ax = gdf.plot(edgecolor="black", facecolor="none")
```

```
# Plota os pontos (A em vermelho, B em azul)
```

```
gpd.GeoSeries([p_dentro]).plot(ax=ax, color="red")
```

```
gpd.GeoSeries([p_borda]).plot(ax=ax, color="blue")
```

```
# Adiciona rótulos próximos aos pontos
```

```
ax.annotate("Ponto A (dentro)", xy=(p_dentro.x, p_dentro.y), xytext=(5, 5),
```

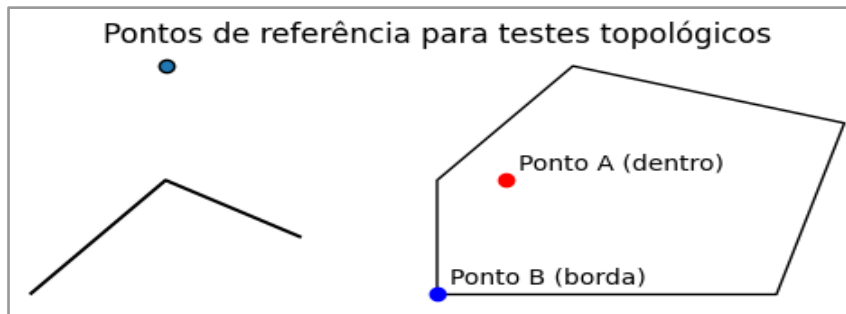
```
textcoords="offset points")
```

```
ax.annotate("Ponto B (borda)", xy=(p_borda.x, p_borda.y), xytext=(5, 5),
           textcoords="offset points")
```

```
ax.set_title("Pontos de referência para testes topológicos")
```

```
ax.set_axis_off()
```

```
plt.show()
```



Agora aplicamos as relações topológicas. Note que *contains* e *intersects* serão avaliados usando o Ponto A (dentro), enquanto *touches* será avaliado usando o Ponto B (borda) — assim conseguimos ver claramente a diferença entre “estar dentro” e “apenas tocar”.

```
gdf["contem_A"] = gdf.contains(p_dentro)
gdf["intersecta_A"] = gdf.intersects(p_dentro)
gdf["toca_B"] = gdf.touches(p_borda)
gdf[["tipo", "contem_A", "intersecta_A", "toca_B"]]
```

#Saída:

	tipo	contem_A	intersecta_A	toca_B
0	Ponto	False	False	False
1	Linha	False	False	False
2	Polígono	True	True	True

4.5 LENDO UM GEODATAFRAME COMO TABELA: INSPEÇÃO E FILTROS BÁSICOS

A pesar de lidarmos com espaço, o GeoDataFrame continua sendo uma tabela. Por isso, faz parte do fluxo checar rapidamente estrutura, campos e tipos antes de avançar.

Uma visão “enxuta” do GeoDataFrame ajuda a focar no essencial:

```
gdf_base = gdf[["tipo", "geometry"]].copy()
gdf_base.head()
```

#Saída:

	tipo	geometry
0	Ponto	POINT (2 4)
1	Linha	LINESTRING (0 0, 2 2, 4 1)
2	Polígono	POLYGON ((6 0, 11 0, 12 3, 8 4, 6 2, 6 0))

Para listar campos e inspecionar a estrutura:

```
gdf_base.columns
gdf_base.info()
```

#Saída:

```
<class 'geopandas.geodataframe.GeoDataFrame'>
```

```
RangeIndex: 3 entries, 0 to 2
```

```
Data columns (total 2 columns):
```

```
#   Column  Non-Null Count  Dtype
```

```
---  -
```

```
0  tipo    3 non-null   object
```

```
1  geometry 3 non-null   geometry
```

```
dtypes: geometry(1), object(1)
```

```
memory usage: 180.0+ bytes
```

E, como a geometria é o coração do GeoDataFrame, também vale checar os tipos geométricos presentes:

```
gdf_base.geom_type.unique()
```

```
#Saída:array(['Point', 'LineString', 'Polygon'], dtype=object)
```

Por fim, filtros tabulares são parte do dia a dia. Por exemplo, manter apenas o polígono:

```
gdf_base[gdf_base["tipo"] == "Polígono"]
```

```
#Saída:
```

```
gdf_base[gdf_base["tipo"] == "Polígono"]
```

	tipo	geometry
2	Polígono	POLYGON ((6 0, 11 0, 12 3, 8 4, 6 2, 6 0))

O resultado continua sendo um GeoDataFrame — isto é, você filtra como tabela e ainda pode mapear o subconjunto filtrado quando quiser.

CAPÍTULO 5

VISUALIZAÇÃO DE DADOS ESPACIAIS

Abrir no Colab

https://colab.research.google.com/github/Alexandrogschafer/geoprocessamento-python-livro/blob/main/notebooks/geo_python_capitulo5.ipynb

Ao longo de um projeto de geoprocessamento, os mapas aparecem em diferentes momentos e cumprem funções distintas. Eles não servem apenas para “embelezar” o resultado final: são ferramentas de diagnóstico, apoio à análise e comunicação.

No início do trabalho, a visualização ajuda a responder perguntas simples, mas decisivas: os dados foram lidos corretamente? As feições aparecem na região esperada? Camadas diferentes (limites, rios, estradas e pontos) se sobrepõem de forma coerente? Um mapa rápido costuma revelar, em segundos, problemas que passariam despercebidos em uma tabela — como CRS incompatível, geometrias deslocadas ou camadas invertidas.

Na etapa de análise, os mapas permitem acompanhar o efeito das transformações aplicadas às camadas ao longo do fluxo de trabalho. Depois de operações como recortes, dissoluções, junções espaciais ou reprojeções, é natural “bater o olho” e confirmar se o resultado faz sentido.

Por fim, na etapa de comunicação, a visualização cartográfica sintetiza a análise em produtos claros e interpretáveis: mapas para relatórios técnicos, figuras para artigos, slides para apresentações ou mapas interativos para exploração no navegador. Nesse momento, escolhas como paleta de cores, legendas e títulos influenciam diretamente a compreensão do público.

Ao longo deste capítulo, trabalharemos com GeoPandas e Matplotlib para mapas estáticos, Contextily para adição de mapas base e Folium para construção de mapas interativos. Essas ferramentas se complementam e fazem parte de um mesmo fluxo reprodutível, em que leitura, análise e visualização são descritas em código e podem ser refeitas sempre que necessário.

5.1 MAPAS ESTÁTICOS COM GEOPANDAS E MATPLOTLIB

O ponto de partida para a visualização cartográfica em Python™ é o GeoPandas. Ele organiza atributos e geometrias em um GeoDataFrame e, para desenhar mapas, utiliza o Matplotlib internamente. Isso permite começar com visualizações rápidas, usadas como checagem, e evoluir gradualmente para mapas mais controlados, com ajuste de cores, legendas, títulos e rótulos.

Nesta seção, vamos trabalhar com um *GeoPackage* disponível no repositório do livro, que contém duas camadas:

BR_UF: polígonos dos estados brasileiros
 capitais_br: pontos correspondentes às capitais

Esse conjunto é didaticamente interessante porque combina polígonos e pontos em um espaço familiar, facilitando a interpretação visual.

Começamos importando as bibliotecas necessárias e fazendo o download do *GeoPackage*, caso ele ainda não exista no diretório local.

```
import geopandas as gpd
from pathlib import Path
import fiona
import matplotlib.pyplot as plt

url_gpkg = (
    "https://raw.githubusercontent.com/"
    "Alexandrogschafer/geoprocessamento-python-livro/main/"
    "dados/brasil_estados_capitais.gpkg"
)

dados_dir = Path("dados")
dados_dir.mkdir(exist_ok=True)
gpkg_path = dados_dir / "brasil_estados_capitais.gpkg"
if not gpkg_path.exists():
    !wget -q -O "{gpkg_path}" "{url_gpkg}"
```

Antes de carregar as camadas, verificamos quais estão disponíveis no arquivo:


```
fiona.listlayers(gpkg_path)
```

Em seguida, lemos as duas camadas:

```
gdf_uf = gpd.read_file(gpkg_path, layer="BR_UF")
gdf_capitais = gpd.read_file(gpkg_path, layer="capitais_br")
```

Uma inspeção rápida da tabela de atributos ajuda a entender quais informações estão associadas às geometrias:

```
gdf_uf.head()
```

#Saída:

	CD_UF	NM_UF	SIGLA_UF	NM_REGIAO	AREA_KM2	geometry
0	12	Acre	AC	Norte	164173.429	MULTIPOLYGON (((-68.79282 -10.99957, -68.79367 ...
1	13	Amazonas	AM	Norte	1559255.881	MULTIPOLYGON (((-56.76292 -3.23221, -56.76789 ...
2	15	Pará	PA	Norte	1245870.704	MULTIPOLYGON (((-48.97548 -0.19834, -48.97487 ...
3	16	Amapá	AP	Norte	142470.762	MULTIPOLYGON (((-51.04561 -0.05088, -51.05422 ...
4	17	Tocantins	TO	Norte	277423.627	MULTIPOLYGON (((-48.2483 -13.19239, -48.24844 ...

A camada inclui colunas como CD_UF, NM_UF, SIGLA_UF, NM_REGIAO, AREA_KM2 e a coluna geometry.

Essa etapa não é apenas “burocrática”: conhecer os nomes das colunas é essencial para criar mapas temáticos e rótulos corretamente.

A primeira plotagem tem um objetivo simples: confirmar que os dados foram lidos corretamente e aparecem na posição esperada.

```
fig, ax = plt.subplots(figsize=(6, 6))
gdf_uf.plot(ax=ax)
ax.set_title("Estados brasileiros — checagem inicial")
ax.set_axis_off()
plt.show()
```



Figura 5.1 — Plotagem básica dos estados brasileiros.

Nesse tipo de mapa, não buscamos estética refinada. Ele serve como diagnóstico rápido: se os estados aparecem deslocados, invertidos ou fragmentados, algo está errado na leitura ou no sistema de referência.

Com a leitura confirmada, pequenos ajustes visuais já melhoram bastante a legibilidade. Um contorno mais definido ajuda a separar as feições, especialmente em mapas com muitos polígonos.

```
fig, ax = plt.subplots(figsize=(6, 6))  
gdf_uf.plot(ax=ax, facecolor="white", edgecolor="black", linewidth=0.8)  
ax.set_title("Estados brasileiros")  
ax.set_axis_off()  
plt.show()
```



Figura 5.2 — Estados com contorno reforçado.

Esse padrão — preenchimento claro e contorno escuro — será mantido ao longo do capítulo, pois funciona bem tanto em tela quanto em material impresso.

Uma das grandes vantagens do GeoPandas é a criação direta de mapas temáticos a partir de colunas da tabela de atributos. Como exemplo, vamos colorir os estados de acordo com a região (NM_REGIAO).

```
fig, ax = plt.subplots(figsize=(8, 8))
gdf_uf.plot(ax=ax, column="NM_REGIAO", cmap="Pastel2", legend=True,
            edgecolor="black", linewidth=0.5)
ax.set_title("Brasil — Estados por região")
ax.set_axis_off()
plt.show()
```

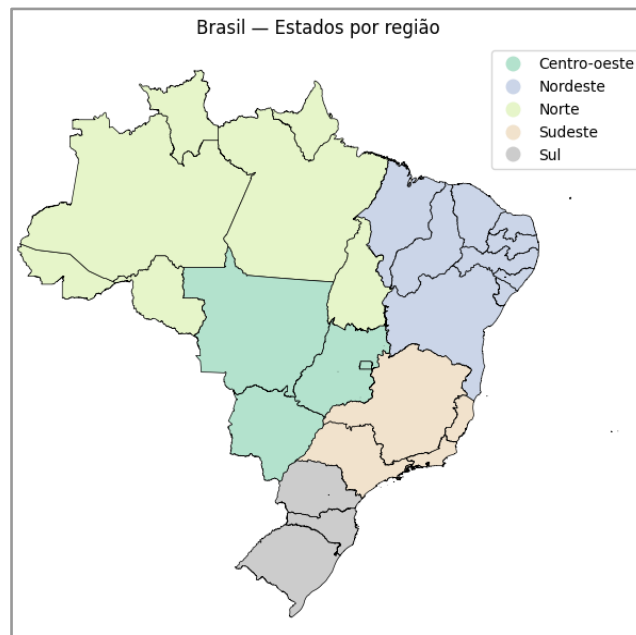


Figura 5.3 — Mapa temático dos estados brasileiros por região.

Nesse tipo de mapa, a legenda passa a ser um elemento central da interpretação, pois conecta a simbologia gráfica aos valores do atributo.

Como o método `.plot()` retorna um objeto `ax`, é possível ajustar elementos do eixo quando necessário, como `grade` e rótulos. Esse tipo de ajuste é útil quando o objetivo é explorar coordenadas ou discutir posicionamento espacial. Aqui, mantemos os eixos visíveis de propósito.

```
fig, ax = plt.subplots(figsize=(6, 6))
gdf_uf.plot(ax=ax, facecolor="none", edgecolor="black")
ax.grid(True, linestyle="-", alpha=0.5)
ax.tick_params(axis="both", labelsize=7)
ax.set_title("Estados brasileiros com grade")
plt.show()
```



Figura 5.4 — Estados com grade e rótulos de coordenadas.

Em mapas cartográficos “de apresentação”, a grade costuma ser dispensável. Aqui, ela aparece apenas para ilustrar que o controle do eixo está disponível quando necessário.

Para produzir figuras mais consistentes, o uso explícito de `plt.subplots()` é recomendado. Isso garante controle sobre tamanho e título.

```
fig, ax = plt.subplots(figsize=(10, 8))
gdf_uf.plot(ax=ax, facecolor="white", edgecolor="black", linewidth=0.8)
ax.set_title("Brasil — Estados")
ax.set_axis_off()
plt.show()
```



Figura 5.5 — Figura com tamanho e título definidos.

Uma forma simples de adicionar rótulos é usar pontos de referência derivados da própria geometria. Aqui, utilizamos os centroides dos estados para posicionar as siglas.

```
fig, ax = plt.subplots(figsize=(10, 8))
gdf_uf.plot(ax=ax, facecolor="white", edgecolor="black", linewidth=0.8)
for x, y, label in zip(gdf_uf.geometry.centroid.x, gdf_uf.geometry.centroid.y,
    gdf_uf["SIGLA_UF"]):
    ax.annotate(text=label, xy=(x, y), xytext=(0, -2), textcoords="offset points",
        fontsize=9)
ax.set_title("Brasil — Estados rotulados")
ax.set_axis_off()
plt.show()
```



Figura 5.6 — Estados rotulados por sigla.

Em geometrias muito recortadas, o centroide pode cair fora do polígono. Para o Brasil, essa abordagem é suficiente no contexto didático, mas existem alternativas mais robustas para casos complexos.

Para compor mapas com múltiplas camadas, basta desenhá-las no mesmo eixo. A ordem das chamadas define o que aparece “por cima”.

```
fig, ax = plt.subplots(figsize=(10, 8))
gdf_uf.plot(ax=ax, facecolor="white", edgecolor="gray", linewidth=0.6)
gdf_capitais.plot(ax=ax, color="purple", markersize=60)
ax.set_title("Brasil — Estados e Capitais")
ax.set_axis_off()
plt.show()
```

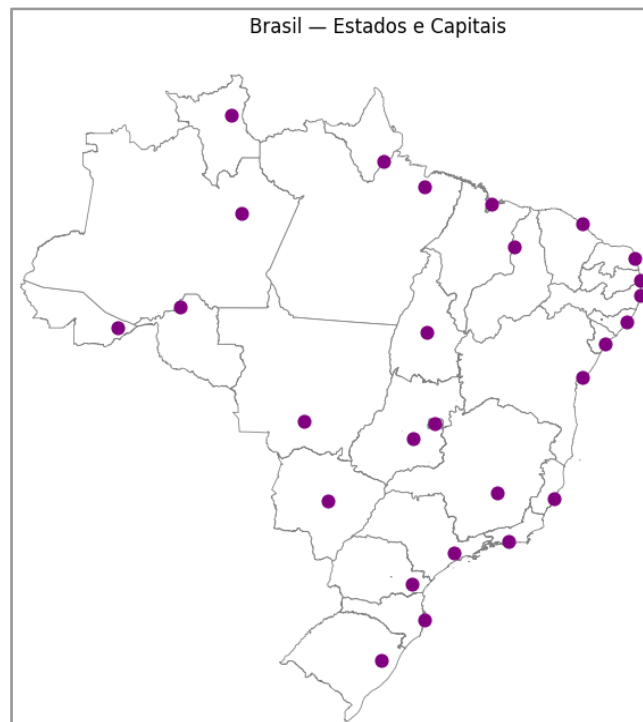


Figura 5.7 — Estados e capitais no mesmo mapa.

O mesmo procedimento de rotulagem pode ser aplicado a pontos. Neste exemplo, posicionamos o nome da capital próximo a cada marcador.

```
fig, ax = plt.subplots(figsize=(10, 8))
gdf_uf.plot(ax=ax, facecolor="white", edgecolor="gray", linewidth=0.6)
gdf_capitais.plot(ax=ax, color="purple", markersize=35)
for x, y, label in zip(gdf_capitais.geometry.x, gdf_capitais.geometry.y,
    gdf_capitais["capital"]):
    ax.annotate(text=label, xy=(x, y), xytext=(3, 3), textcoords="offset points",
        fontsize=7)
ax.set_title("Brasil — Estados e Capitais rotulados")
ax.set_axis_off()
plt.show()
```

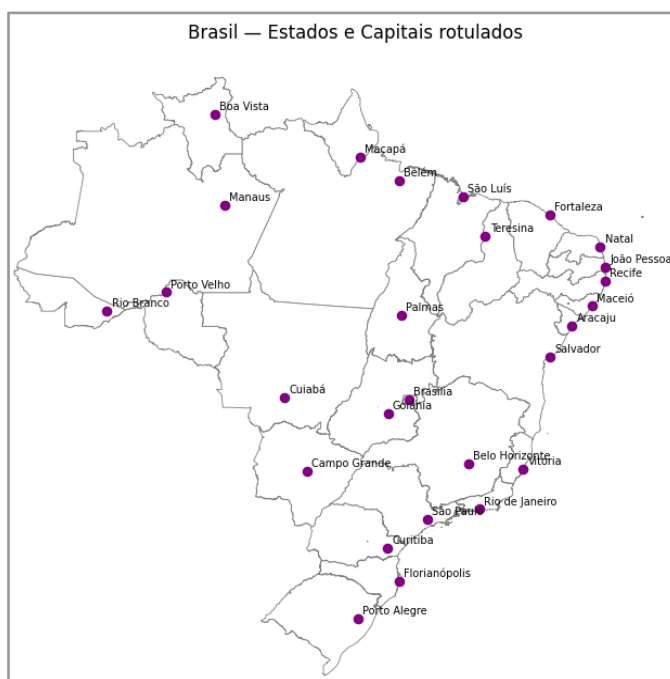



Figura 5.8 — Estados e capitais com rótulos.

Em mapas nacionais, a densidade de rótulos pode prejudicar a leitura em algumas regiões. Ainda assim, o exemplo é útil para entender o método, que se torna muito mais legível em mapas regionais ou locais.

5.2 ADICIONANDO MAPAS BASE COM CONTEXTILY

Em muitos casos, desenhar apenas a camada vetorial não é suficiente para interpretar um mapa com confiança. Um mapa base funciona como pano de fundo e fornece referência imediata do mundo real: costa, grandes feições, distribuição espacial e “sensação de lugar”. Isso facilita tanto a checagem de posicionamento quanto a comunicação com quem não está acostumado com limites administrativos “soltos” no espaço.

No ecossistema Python™, a biblioteca Contextily resolve essa tarefa de forma direta: ela adiciona tiles (mapas base) em um eixo do Matplotlib, mantendo o mesmo fluxo que já usamos com GeoPandas.

No Google Colab®, instale e importe:

```
!pip install contextily --quiet
```

```
import contextily as ctx
```

Antes de adicionar um mapa base, há um detalhe técnico essencial: a maioria dos provedores de tiles trabalha em Web Mercator (EPSG:3857). Isso significa que, para o fundo e as feições coincidirem, as camadas vetoriais também precisam estar nesse CRS.

Uma forma simples de verificar o CRS atual é:

```
print("CRS estados:", gdf_uf.crs)
print("CRS capitais:", gdf_capitais.crs)
```

Em seguida, reprojecemos para EPSG:3857:

```
gdf_uf_3857 = gdf_uf.to_crs(epsg=3857)
gdf_capitais_3857 = gdf_capitais.to_crs(epsg=3857)
```

Se você tentar adicionar o mapa base mantendo as camadas em outro CRS, o resultado típico é um mapa “desencaixado”, com o fundo em um lugar e as feições em outro.

Com as camadas reprojecadas, o procedimento é previsível: criamos figura e eixo, desenhemos estados e capitais e, por fim, chamamos `ctx.add_basemap()`.

Como estamos trabalhando com o Brasil inteiro, vale fixar a extensão com os limites da camada. Isso ajuda o Contextily a buscar tiles na região correta e evita zoom inadequado.

```
fig, ax = plt.subplots(figsize=(10, 8))
# Estados (base vetorial)
gdf_uf_3857.plot(ax=ax, facecolor="none", edgecolor="black", linewidth=0.8)
# Capitais (sobreposição)
gdf_capitais_3857.plot(ax=ax, color="purple", markersize=35)
# Ajusta extensão pela camada de estados
xmin, ymin, xmax, ymax = gdf_uf_3857.total_bounds
ax.set_xlim(xmin, xmax)
ax.set_ylim(ymin, ymax)
# Mapa base (OpenStreetMap padrão)
ctx.add_basemap(ax, source=ctx.providers.OpenStreetMap.Mapnik, zoom=4)
ax.set_title("Brasil — Estados e Capitais (OSM + Contextily)")
ax.set_axis_off()
plt.show()
```

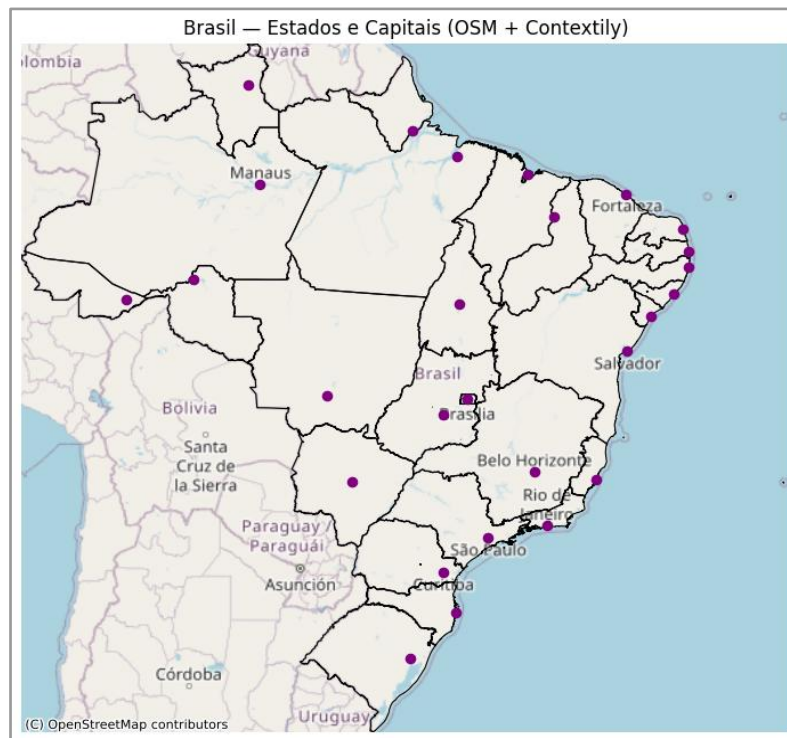


Figura 5.9 — Estados e capitais sobre mapa base do OpenStreetMap (Contextily).

O parâmetro zoom controla o nível de detalhe do fundo: quanto maior, mais detalhado (e mais pesado) fica o carregamento. Para o Brasil inteiro, valores na faixa de 4 a 5 costumam ser suficientes; se estiver lento, reduza o zoom.

5.3 MAPAS INTERATIVOS COM FOLIUM

Quando a interatividade é importante — zoom, pan, clique para ver informações e controle de camadas — uma opção prática é o Folium, que integra Python™ ao Leaflet.js e permite gerar mapas web diretamente no notebook. Isso é especialmente útil em atividades didáticas e para compartilhar resultados em HTML.

No Colab[®], instale e importe:

```
!pip install folium --quiet
import folium
```

Como o Folium trabalha com coordenadas em latitude/longitude, é recomendável garantir que as camadas estejam em EPSG:4326:

```
gdf_uf_4326 = gdf_uf.to_crs(epsg=4326)
gdf_capitais_4326 = gdf_capitais.to_crs(epsg=4326)
```

Um mapa básico pode ser criado definindo um centro e um nível inicial de zoom. Para o Brasil, um zoom em torno de 4 funciona bem.

```
m = folium.Map(location=[-14.2, -51.9], zoom_start=4)
m
```



Figura 5.10 — Mapa interativo básico (Folium).

Agora vamos montar um mapa mais completo: estados como polígonos, capitais como pontos com pop-up e um controle para ligar/desligar camadas.

```
m = folium.Map(location=[-14.2, -51.9], zoom_start=4)
# Estados (polígonos)
folium.GeoJson(gdf_uf_4326.to_json(), name="Estados", style_function=lambda feat: {
    "fillColor": "transparent", "color": "black", "weight": 1}).add_to(m)

# Capitais (pontos) com pop-up
fg_capitais = folium.FeatureGroup(name="Capitais")
col_nome = "capital"
for _, row in gdf_capitais_4326.iterrows():
```

```

folium.CircleMarker(location=[row.geometry.y, row.geometry.x], radius=4,
                    popup=row[col_nome], color="purple", fill=True, fill_color="purple",
                    fill_opacity=0.8).add_to(fg_capitais)
fg_capitais.add_to(m)
# Controle de camadas
folium.LayerControl(collapsed=False).add_to(m)
m

```

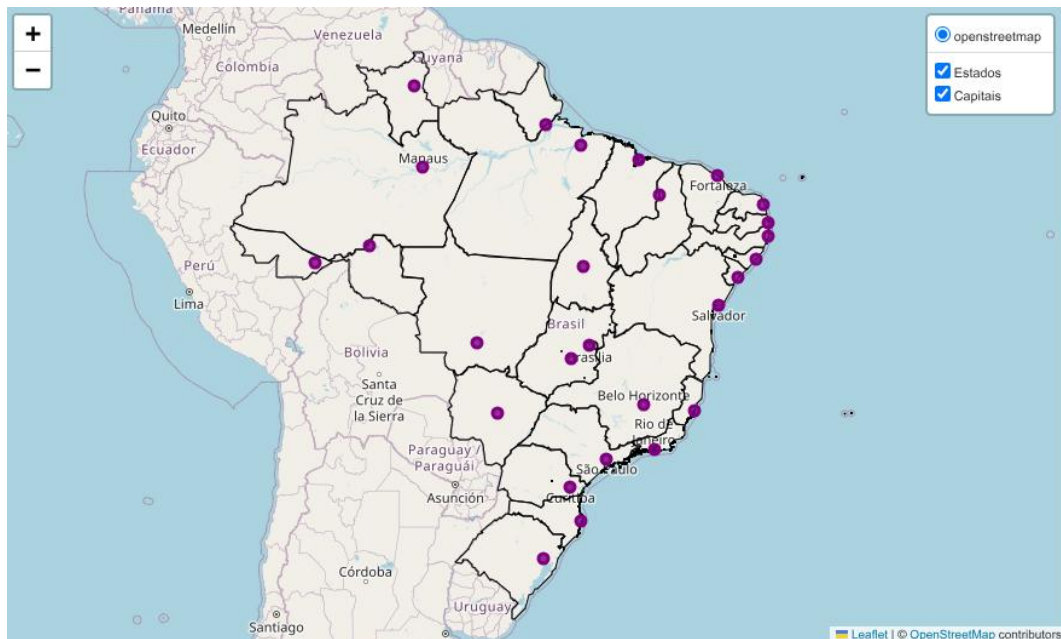


Figura 5.11 — Mapa interativo com estados, capitais, pop-ups e controle de camadas (Folium).

Para compartilhar o resultado fora do notebook, basta exportar para um arquivo HTML:

```
m.save("mapa_interativo_brasil.html")
```

Esse arquivo pode ser aberto em qualquer navegador e também pode ser anexado em relatórios ou publicado em páginas web.

5.4 QUANDO USAR CADA FERRAMENTA?

Depois de conhecer as principais opções de visualização cartográfica em Python™, vale consolidar em que situações cada uma delas costuma ser mais indicada no fluxo de trabalho.

No dia a dia, o método `gdf.plot()` do GeoPandas é a forma mais rápida de produzir mapas de checagem e exploração. Ele aparece o tempo todo durante a preparação dos dados porque permite confirmar, em segundos, se a leitura deu certo, se o CRS está coerente e se uma operação intermediária (recorte, dissolução, junção espacial ou reprojeção) produziu o efeito esperado. Em outras palavras, é o “mapa de diagnóstico” do notebook.

Quando o objetivo passa a ser uma figura de entrega — para relatório, slides, apostila ou artigo — o caminho natural é trabalhar com GeoPandas sobre um eixo do Matplotlib. A lógica do desenho continua simples, mas você ganha controle sobre os elementos que fazem diferença na apresentação: tamanho da figura, títulos, espessuras de contorno, legendas, rótulos e sobreposição de camadas. É esse nível de controle que transforma um mapa exploratório em uma figura pronta para documentação.

O Contextily entra em cena quando você precisa de um pano de fundo reconhecível. Em vez de exibir apenas polígonos e pontos em um espaço “abstrato”, você apoia suas feições em um mapa base (como o *OpenStreetMap*) ou em um fundo neutro, o que melhora a interpretação espacial e facilita a comunicação com públicos não técnicos. O cuidado essencial é que os tiles são servidos em Web Mercator, então as camadas precisam estar em EPSG:3857 para que o encaixe seja correto.

Por fim, quando a necessidade é interatividade, o Folium é a escolha mais direta. Ele permite navegar com zoom e pan, abrir pop-ups ao clicar em feições, ligar e desligar camadas e exportar o resultado como um arquivo HTML. Isso torna o mapa compartilhável fora do notebook, em qualquer navegador, e cria um caminho simples para demonstrar resultados de forma explorável.

CAPÍTULO 6

GEOMETRIAS E OPERAÇÕES ESPACIAIS

Abrir no Colab

https://colab.research.google.com/github/Alexandrogschafer/geoprocessamento-python-livro/blob/main/notebooks/geo_python_capitulo6.ipynb

As operações espaciais são o núcleo do geoprocessamento vetorial. É por meio delas que conseguimos medir áreas e comprimentos, criar zonas de influência, recortar camadas por áreas de interesse, combinar informações de diferentes fontes e transferir atributos por localização. Neste capítulo, o foco não é “o dado em si”, e sim o comportamento das operações: vamos trabalhar com um conjunto mínimo e controlado de geometrias e, a partir dele, executar as operações mais usadas no dia a dia com GeoPandas e Shapely.

A ideia é simples: manter um cenário pequeno, visualmente claro e reutilizável. Assim, em vez de inventar um novo exemplo para cada operação, você aprende o padrão e depois aplica o mesmo raciocínio em camadas reais (municípios, bacias, bairros, estradas, pontos de interesse etc.) sem mudar a lógica.

6.1 PREPARANDO O CONJUNTO MÍNIMO DE GEOMETRIAS

Nesta etapa, vamos criar um pequeno conjunto de geometrias que será reutilizado em todo o capítulo. Ele foi escolhido para ser o “mínimo que funciona”:

- 1 ponto (referência pontual);
- 1 linha (elemento linear);
- 2 polígonos parcialmente sobrepostos (necessários para recorte, interseção, união, diferença e dissolve).

Vamos criar as geometrias em EPSG:4326 (lon/lat) apenas para manter a intuição espacial e, em seguida, reprojeter tudo para um CRS métrico (em metros). A partir daí, todas as operações do capítulo serão feitas na versão em metros.

```
import geopandas as gpd
import matplotlib.pyplot as plt
from shapely.geometry import Point, LineString, Polygon

# 1) Geometrias em lon/lat (EPSG:4326)
# Ponto de referência
ponto = Point(-54.10, -31.33)
# Linha (trecho linear qualquer)
linha = LineString([(-54.08, -31.31), (-54.18, -31.36), (-54.26, -31.40)])
# Polígono A
poligono_A = Polygon([(-54.12, -31.30), (-54.22, -31.30),
                      (-54.22, -31.38), (-54.12, -31.38), (-54.12, -31.30)])

# Polígono B (parcialmente sobreposto ao A)
poligono_B = Polygon([(-54.18, -31.32), (-54.28, -31.32),
                      (-54.28, -31.42), (-54.18, -31.42), (-54.18, -31.32)])

# 2) Organização em GeoDataFrame
gdf = gpd.GeoDataFrame(
    {"nome": ["Ponto A", "Linha", "Área A", "Área B"],
     "grupo": ["ref", "linear", "A", "A"]},
    crs="EPSG:4326",
).set_geometry([ponto, linha, poligono_A, poligono_B])

gdf
```


	nome	grupo	geometry
0	Ponto A	ref	POINT (-54.1 -31.33)
1	Linha	linear	LINESTRING (-54.08 -31.31, -54.18 -31.36, -54....
2	Área A	A	POLYGON ((-54.12 -31.3, -54.22 -31.3, -54.22 -...
3	Área B	A	POLYGON ((-54.18 -31.32, -54.28 -31.32, -54.28...

Agora reprojetamos para um CRS em metros. Aqui usamos SIRGAS 2000 / UTM zona 22S (EPSG:31982), que é um CRS projetado e permite interpretar áreas e distâncias em unidades físicas.

```
gdf_m = gdf.to_crs("EPSG:31982")
gdf_m.crs
```

#Saída:

```
<Projected CRS: EPSG:31982> Name: SIRGAS 2000 / UTM zone 22S Axis Info [cartesian]: - E[east]:
Easting (metre) - N[north]: Northing (metre) Area of Use: - name: Brazil - between 54°W and
48°W, northern and southern hemispheres, onshore and offshore. In remainder of South America
- between 54°W and 48°W, southern hemisphere, onshore and offshore. - bounds: (-54.0, -54.18, -
47.99, 7.04) Coordinate Operation: - name: UTM zone 22S - method: Transverse Mercator Datum:
Sistema de Referencia Geocentrico para las AmericaS 2000 - Ellipsoid: GRS 1980 - Prime Meridian:
Greenwich
```

Antes de avançar, fazemos um mapa simples para confirmar que o cenário está “do jeito certo”: linha e polígonos visíveis, e sobreposição clara entre Área A e Área B.

```
fig, ax = plt.subplots(figsize=(6, 6))
# Plota linha e polígonos
gdf_m[gdf_m.geom_type != "Point"].plot(ax=ax, edgecolor="black", facecolor="none",
    linewidth=2)
# Plota o ponto por cima
gdf_m[gdf_m.geom_type == "Point"].plot(ax=ax, color="red", markersize=60)

# Rótulo do ponto
```

```

p = gdf_m.loc[gdf_m["nome"] == "Ponto A", geometry].iloc[0]
ax.annotate("Ponto A",xy=(p.x, p.y),xytext=(6, 6), textcoords="offset points")
ax.set_title("Geometrias (em metros)")
ax.set_axis_off()
plt.show()

```

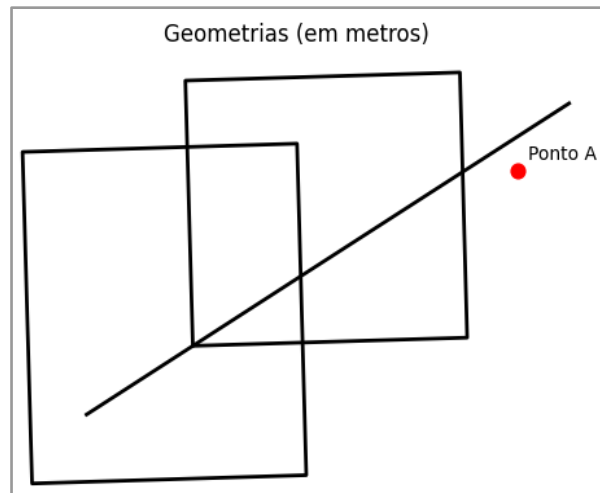


Figura 6.1 — Conjunto mínimo de geometrias do capítulo (em CRS métrico).

6.2 PROPRIEDADES GEOMÉTRICAS EM CRS MÉTRICO

Com o GeoDataFrame em CRS métrico, as propriedades geométricas passam a ter interpretação direta:

- área em m^2 (e conversões comuns, como hectares);
- comprimento em metros (para linhas) e perímetro em metros (para polígonos);
- centroide e envelope como geometrias derivadas úteis para inspeção e apoio a análises.

Vamos calcular algumas propriedades e guardar o resultado em colunas, apenas para visualizar com clareza.

```

gdf_m = gdf_m.copy()

gdf_m["area_m2"] = gdf_m.area
gdf_m["area_ha"] = gdf_m.area / 10_000 # conversão comum: 1 ha = 10.000 m²

```

```
gdf_m["comprimento_m"] = gdf_m.length
```

```
gdf_m["centroide"] = gdf_m.centroid
```

```
gdf_m["envelope"] = gdf_m.envelope
```

```
gdf_m["tipo"] = gdf_m.geom_type
```

```
gdf_m[["nome", "tipo", "area_m2", "area_ha", "comprimento_m"]]
```

	nome	tipo	area_m2	area_ha	comprimento_m
0	Ponto A	Point	0.000000e+00	0.000000	0.000000
1	Linha	LineString	0.000000e+00	0.000000	19836.073212
2	Área A	Polygon	8.453246e+07	8453.245621	36799.359928
3	Área B	Polygon	1.056414e+08	10564.141068	41233.370149

Neste ponto, vale lembrar a leitura correta:

Ponto: área = 0 e comprimento = 0

Linha: área = 0 e comprimento = “tamanho do traçado”

Polígono: área > 0 e comprimento = “perímetro”

Para confirmar rapidamente os tipos presentes no conjunto:

```
gdf_m.geom_type.unique()
```

```
#Saída: array(['Point', 'LineString', 'Polygon'], dtype=object)
```

Com as propriedades básicas em mãos, passamos para o objetivo principal do capítulo: operações espaciais, isto é, operações que criam novas geometrias (*buffer*, recorte, união) ou transferem atributos por localização (junção espacial).

6.3 OPERAÇÕES BASEADAS EM DISTÂNCIA: BUFFER

A operação *buffer* cria uma zona ao redor de uma geometria a uma distância definida. Ela aparece sempre que queremos representar áreas de influência, faixas de proteção ou proximidade.

Como estamos em CRS métrico, a distância do *buffer* é interpretada em metros. Vamos criar um *buffer* de 5.000 m ao redor do Ponto A.

```

# Seleciona o ponto
gdf_ponto = gdf_m[gdf_m["nome"] == "Ponto A"]
# Buffer de 5000 m (resultado é uma GeoSeries)
buffer_5000 = gdf_ponto.geometry.buffer(5000)
# Visualização: cenário + buffer
fig, ax = plt.subplots(figsize=(6, 6))
gdf_m[gdf_m.geom_type != "Point"].plot(ax=ax,
    edgecolor="black",facecolor="none",linewidth=2)
buffer_5000.plot(ax=ax, alpha=0.3)
gdf_ponto.plot(ax=ax, color="red", markersize=60)
p = gdf_ponto.geometry.iloc[0]
ax.annotate("Ponto A", xy=(p.x, p.y), xytext=(6, 6), textcoords="offset points")
ax.set_title("Buffer de 5.000 m ao redor do Ponto A")
ax.set_axis_off()
plt.show()

```

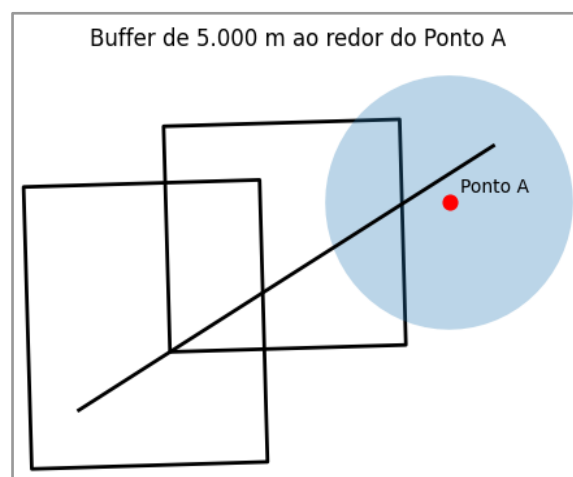


Figura 6.2 — Buffer de 5.000 m ao redor de um ponto.

6.4 OPERAÇÕES DE SOBREPOSIÇÃO: CLIP E OVERLAY

Quando duas áreas se sobrepõem, duas famílias de operação aparecem o tempo todo:

- clip: recortar uma camada usando outra como máscara;
- overlay: combinar explicitamente geometrias (interseção, união, diferença).

Vamos trabalhar apenas com os polígonos do conjunto base, separando Área A e Área B para evitar resultados ambíguos.

```
gdf_pol = gdf_m[gdf_m.geom_type == "Polygon"].copy()
area_a = gdf_pol[gdf_pol["nome"] == "Área A"]
area_b = gdf_pol[gdf_pol["nome"] == "Área B"]
```

Antes de operar, um mapa rápido para enxergar a sobreposição.

```
fig, ax = plt.subplots(figsize=(6, 6))
area_a.plot(ax=ax, edgecolor="black", facecolor="none", linewidth=2)
area_b.plot(ax=ax, edgecolor="red", facecolor="none", linewidth=2)
ax.set_title("Área A (preto) e Área B (vermelho) — sobreposição")
ax.set_axis_off()
plt.show()
```

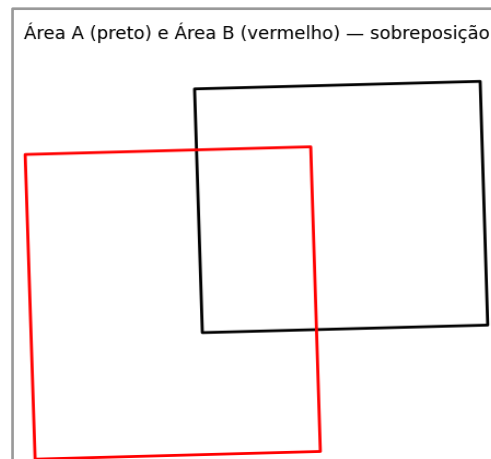


Figura 6.3 — Sobreposição entre dois polígonos.

O clip “corta” uma camada usando outra como máscara. Aqui, vamos recortar Área A pela Área B.

```
recorte = gpd.clip(area_a, area_b)
fig, ax = plt.subplots(figsize=(6, 6))
area_a.plot(ax=ax, edgecolor="black", facecolor="none", linewidth=2)
area_b.plot(ax=ax, edgecolor="red", facecolor="none", linewidth=2)
recorte.plot(ax=ax, alpha=0.5)
ax.set_title("clip: Área A recortada pela Área B")
ax.set_axis_off()
plt.show()
```

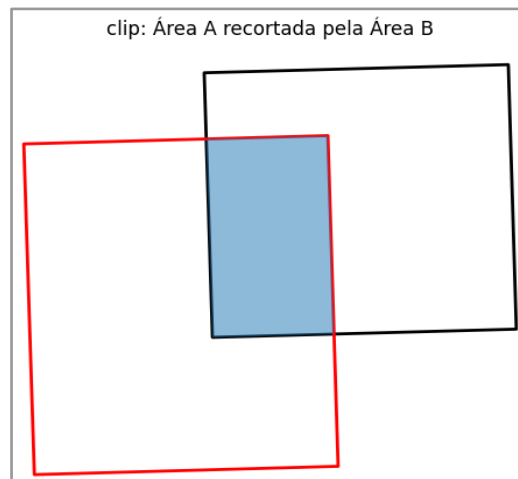


Figura 6.4 — Resultado do clip.

O overlay permite controlar explicitamente a operação. Vamos calcular:

intersection ($A \cap B$): apenas a parte em comum;

union ($A \cup B$): combinação completa;

difference ($A - B$) e difference ($B - A$): partes exclusivas de cada um.

```
inter = gpd.overlay(area_a, area_b, how="intersection")
uni = gpd.overlay(area_a, area_b, how="union")
dif_a = gpd.overlay(area_a, area_b, how="difference") # A - B
dif_b = gpd.overlay(area_b, area_a, how="difference") # B - A
```

Visualizando a interseção com contexto:

```
fig, ax = plt.subplots(figsize=(6, 6))
area_a.plot(ax=ax, edgecolor="black", facecolor="none", linewidth=2)
area_b.plot(ax=ax, edgecolor="red", facecolor="none", linewidth=2)
inter.plot(ax=ax, alpha=0.5)
ax.set_title("overlay (intersection): região em comum ( $A \cap B$ )")
ax.set_axis_off()
plt.show()
```

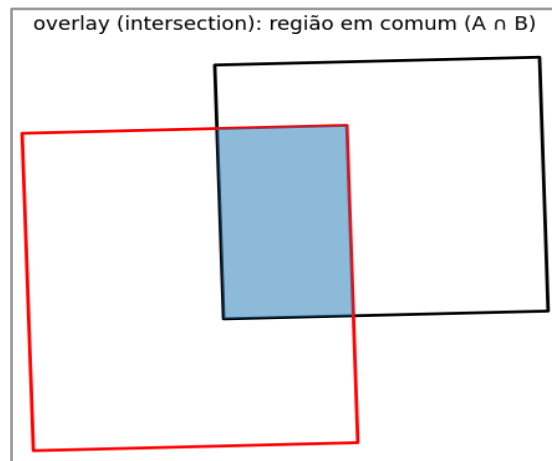


Figura 6.5 — Interseção ($A \cap B$).

O `overlay` com `how="intersection"` devolve apenas a região compartilhada pelos dois polígonos.

Se você quiser comparar rapidamente as duas diferenças:

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5))
dif_a.plot(ax=ax1, alpha=0.5)
ax1.set_title("overlay (difference): Área A – Área B")
ax1.set_axis_off()
dif_b.plot(ax=ax2, alpha=0.5)
ax2.set_title("overlay (difference): Área B – Área A")
ax2.set_axis_off()
plt.show()
```

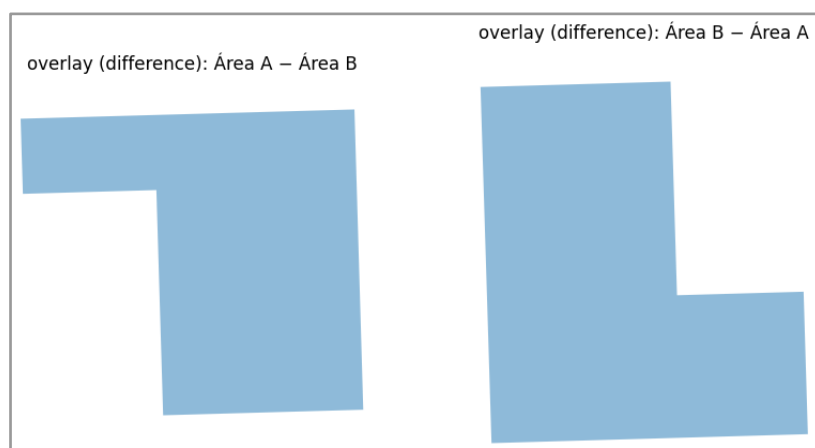


Figura 6.6: Diferenças entre dois polígonos usando `overlay (difference)`.

6.5 AGREGAÇÃO ESPACIAL: DISSOLVE

A operação dissolve agrupa feições por um atributo e remove fronteiras internas, produzindo uma geometria unificada por grupo. Esse padrão aparece quando passamos de um nível detalhado para um nível mais agregado (por exemplo, unir várias áreas de uma mesma classe).

No conjunto base, a coluna grupo foi definida para que Área A e Área B pertençam ao mesmo grupo "A".

```
gdf_diss = gdf_m.dissolve(by="grupo")
fig, ax = plt.subplots(figsize=(6, 6))
gdf_diss.plot(ax=ax, edgecolor="black", facecolor="none", linewidth=2)
ax.set_title("dissolve: geometrias agrupadas por 'grupo'")
ax.set_axis_off()
plt.show()
```

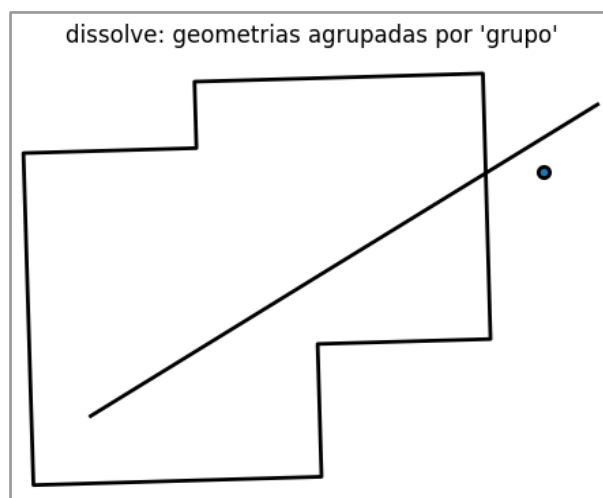


Figura 6.7 — Resultado do dissolve.

As áreas do grupo "A" são unificadas, removendo a fronteira interna entre Área A e Área B.

6.6 JUNCTÃO ESPACIAL: COMBINANDO TABELAS POR LOCALIZAÇÃO (SJOIN)

A junção espacial funciona como uma “junção por localização”. Em vez de combinar tabelas por um campo (como em SQL), ela combina com base em uma relação espacial: dentro, contém, intersecta etc.

Aqui, vamos responder: em qual polígono o Ponto A está contido?

```
gdf_pts = gdf_m[gdf_m.geom_type == "Point"].copy()
gdf_pol = gdf_m[gdf_m.geom_type == "Polygon"].copy()
# Conferência de CRS (deve ser igual)
gdf_pts.crs == gdf_pol.crs
```

Agora fazemos a junção. Usamos `within` para perguntar se o ponto está dentro do polígono.

```
juncao = gpd.sjoin(gdf_pts, gdf_pol, how="left", predicate="within")
cols = ["nome_left", "grupo_left", "nome_right", "grupo_right"]
juncao[cols].rename(columns={"nome_left": "ponto", "grupo_left": "grupo_ponto",
                             "nome_right": "area", "grupo_right": "grupo_area",})
```

	ponto	grupo_ponto	area	grupo_area
0	Ponto A	ref	NaN	NaN

O resultado mantém o ponto e adiciona as colunas do polígono correspondente. Se um ponto não cair dentro de nenhum polígono, as colunas vindas do polígono aparecem como NaN — e isso não é erro: é o comportamento esperado.

6.7 GEOMETRIAS INVÁLIDAS: DIAGNÓSTICO E CORREÇÃO

Em dados reais, é comum encontrar polígonos inválidos — por exemplo, quando as bordas se cruzam (auto interseção). Isso não é apenas “um detalhe do desenho”: geometrias inválidas podem fazer operações como `overlay()`, `clip()` e até `buffer()`

falharem ou retornarem resultados estranhos. Por isso, vale aprender um fluxo simples de diagnosticar → visualizar → corrigir → confirmar.

Aqui, vamos criar intencionalmente um polígono inválido (com auto interseção) e comparar com um polígono válido, apenas para deixar a diferença clara no mapa.

```
import geopandas as gpd
import matplotlib.pyplot as plt
from shapely.geometry import Polygon

# Polígono inválido (auto-interseção)
poligono_invalido = Polygon([(-54.21, -31.36), (-54.25, -31.40),
                              (-54.25, -31.30), (-54.21, -31.38), (-54.21, -31.36)])

# Polígono válido (retângulo simples, para comparação)
poligono_valido = Polygon([(-54.27, -31.35), (-54.30, -31.35),
                              (-54.30, -31.40), (-54.27, -31.40), (-54.27, -31.35)])

gdf_pol = gpd.GeoDataFrame({"nome": ["Área inválida", "Área válida"]},
                             geometry=[poligono_invalido, poligono_valido], crs="EPSG:4326")

gdf_pol
```

	nome	geometry
0	Área inválida	POLYGON ((-54.21 -31.36, -54.25 -31.4, -54.25 ...
1	Área válida	POLYGON ((-54.27 -31.35, -54.3 -31.35, -54.3 -...

Antes de qualquer correção, fazemos um mapa rápido. Mesmo sem medir nada, o polígono inválido costuma “parecer estranho”, porque suas bordas se cruzam.

```
ax = gdf_pol.plot(edgecolor="black", facecolor="none", linewidth=2, figsize=(6, 6))
ax.set_title("Exemplo de geometria inválida (auto-interseção) vs válida")
ax.set_axis_off()
plt.show()
```

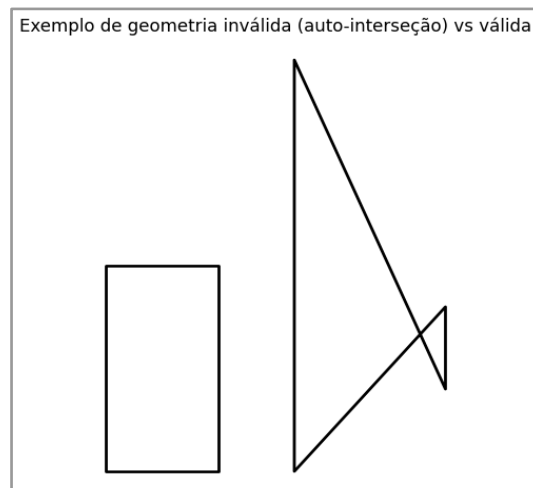


Figura 6.8 — Dois polígonos: um inválido (auto-interseção) e um válido (retângulo).

O GeoPandas permite verificar a validade com `is_valid`, que devolve `True/False` para cada feição.

```
gdf_pol[["nome"]].assign(valida=gdf_pol.is_valid)
```

	nome	valida
0	Área inválida	False
1	Área válida	True

Um recurso muito usado é aplicar `buffer(0)`: apesar do nome, esse “buffer de distância zero” frequentemente força o Shapely a reconstruir a geometria de forma consistente, removendo a auto interseção quando isso é possível.

```
gdf_corr = gdf_pol.copy()
gdf_corr["geometry"] = gdf_corr.geometry.buffer(0)
```

Agora comparamos antes e depois lado a lado. Para o leitor enxergar claramente a diferença, mantemos a mesma escala e o mesmo estilo.

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
gdf_pol.plot(ax=ax1, edgecolor="black", facecolor="none", linewidth=2)
ax1.set_title("Antes da correção")
ax1.set_axis_off()
gdf_corr.plot(ax=ax2, edgecolor="black", facecolor="none", linewidth=2)
```

```
ax2.set_title("Depois do buffer(0)")
ax2.set_axis_off()
plt.show()
```

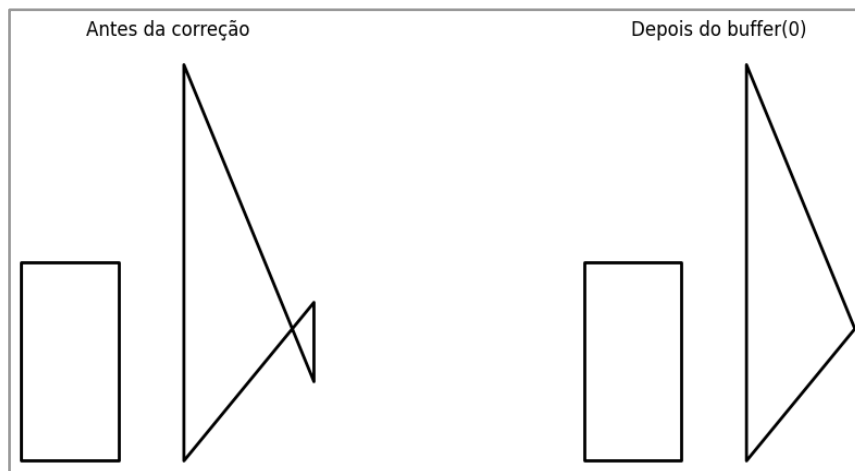


Figura 6.8 — Comparação visual: geometria original (inválida) e geometria após `buffer(0)` (corrigida).

Por fim, repetimos o teste de validade.

```
gdf_pol[["nome"]].assign(valida=gdf_pol.is_valid).assign(valida_corr=gdf_corr.is_valid)
```

	nome	valida	valida_corr
0	Área inválida	False	True
1	Área válida	True	True

Observação importante: `buffer(0)` é um atalho útil, mas não é uma “garantia universal”. Dependendo do erro, ele pode alterar detalhes da geometria. A regra prática é sempre a mesma: corrigir, visualizar e confirmar antes de seguir com análises.

FUNDAMENTOS E DIAGNÓSTICO DE DADOS RASTER

Abrir no Colab

https://colab.research.google.com/github/Alexandrogschafer/geoprocessamento-python-livro/blob/main/notebooks/geo_python_capitulo7.ipynb

Nos capítulos anteriores, trabalhamos com dados vetoriais e aprendemos a visualizar e manipular pontos, linhas e polígonos. Neste capítulo, entramos em um outro tipo de dado fundamental no geoprocessamento: o modelo *raster*.

Sempre que lidamos com imagens de satélite, modelos digitais de elevação, mapas de temperatura ou precipitação, estamos trabalhando com *rasters*. Diferentemente do vetor, que representa objetos discretos (objetos “separados” no espaço), o *raster* descreve o território como uma grade regular de células (*pixels*), em que cada célula armazena um valor numérico. Cada *pixel* corresponde a uma pequena área do terreno e guarda uma informação associada àquela posição.

Antes de qualquer processamento — como recortes, máscaras ou cálculos — é essencial diagnosticar o *raster*: abrir o arquivo, inspecionar metadados, checar valores, visualizar rapidamente e identificar problemas comuns. Vamos trabalhar com um Modelo Digital de Elevação (MDE) real, disponibilizado no repositório do livro.

Seguimos o mesmo padrão adotado nos outros capítulos: criamos uma pasta local de dados e baixamos o arquivo diretamente do GitHub usando a URL “raw”.

```
from pathlib import Path
import rasterio as rio
import matplotlib.pyplot as plt
import numpy as np

# URL "raw" do MDE no repositório do livro
url_mde = (
    "https://raw.githubusercontent.com/"
```

```
"Alexandrogschafer/geoprocessamento-python-livro/main/"
"dados/mde.tif"
)
```

```
# Pasta de dados
```

```
dados_dir = Path("dados")
```

```
dados_dir.mkdir(exist_ok=True)
```

```
# Caminho local do arquivo
```

```
mde_path = dados_dir / "mde.tif"
```

```
# Download do arquivo, se necessário
```

```
if not mde_path.exists():
```

```
    !wget -q -O "{mde_path}" "{url_mde}"
```

7.1 ABRINDO O RASTER E INSPECIONANDO METADADOS

Com o arquivo salvo localmente, abrimos o *raster* com o Rasterio. A partir do objeto *src*, conseguimos acessar tanto os valores quanto os metadados que descrevem como a matriz se posiciona no espaço.

Para manter o código mais seguro (e evitar arquivos abertos sem necessidade), vamos usar o padrão com *with*, que fecha o *raster* automaticamente ao final do bloco.

```
with rio.open(mde_path) as src:
```

```
    profile = src.profile
```

```
    print(profile)
```

O *profile* reúne os metadados principais do *raster*. Em um diagnóstico inicial, vale focar nos campos mais importantes:

driver: formato do arquivo (aqui, GeoTIFF).

count: número de bandas (MDE costuma ter 1).

dtype: tipo do dado (altitude costuma ser contínua, float32).

crs: sistema de referência.

transform: matriz afim que relaciona linha/coluna com coordenadas.

width/height: dimensões (colunas/linhas).

nodata: valor NoData declarado (quando existe).

Além do profile, imprimir alguns atributos diretamente reforça um hábito essencial: sempre conferir CRS, dimensões, resolução e NoData antes de avançar.

with rio.open(mde_path) as src:

```
print("CRS:", src.crs)
```

```
print("Número de bandas:", src.count)
```

```
print("Dimensões (linhas, colunas):", src.height, src.width)
```

```
print("Resolução espacial:", src.res)
```

```
print("Valor NoData:", src.nodata)
```

#Saída:

CRS: EPSG:31981

Número de bandas: 1

Dimensões (linhas, colunas): 739 754

Resolução espacial: (30.0, 30.0)

Valor NoData: None

No nosso MDE, o CRS é EPSG:31981 (SIRGAS 2000 / UTM 21S), o que é ótimo porque está em metros — facilitando análises métricas (distância, área, declividade etc.). A resolução é de 30 m, ou seja, cada *pixel* representa um quadrado de 30 × 30 m no terreno. Repare também que o arquivo não declara um NoData explicitamente (None), algo relativamente comum em dados reais e que exige atenção na etapa de diagnóstico.

7.2 LENDO A BANDA E FAZENDO UM “MAPA RÁPIDO”

Como este MDE possui apenas uma banda, lemos diretamente a banda 1. A partir daqui os valores passam a ser um array NumPy: uma matriz em que cada elemento corresponde a um *pixel*.

with rio.open(mde_path) as src:

```
mde_raw = src.read(1) # array NumPy
```

```
print("Shape:", mde_raw.shape)
```

#Saída: Shape: (739, 754)

Antes de entrar em estatísticas, vale fazer um mapa rápido para confirmar visualmente que o *raster* foi carregado corretamente e que “parece” um MDE (sem faixas estranhas, deslocamentos ou bordas muito suspeitas).

```
plt.figure(figsize=(6, 6))  
plt.imshow(mde_raw, cmap="cividis")  
plt.title("MDE — visualização rápida (arquivo original)")  
plt.axis("off")  
plt.show()
```

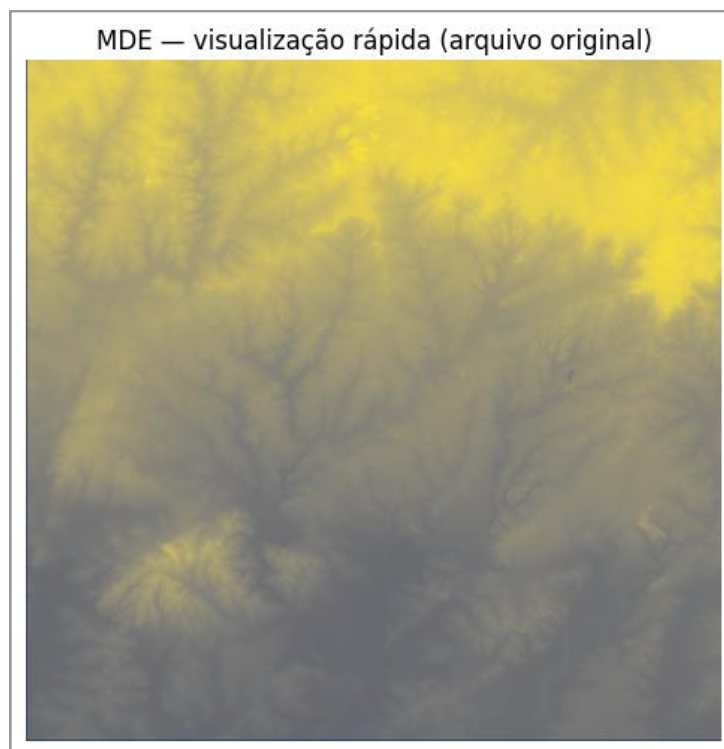


Figura 7.1: Visualização inicial do Modelo Digital de Elevação (MDE).

Esse primeiro mapa não é para análise detalhada: é uma checagem visual de sanidade. A análise numérica vem a seguir.

7.3 ESTATÍSTICAS INICIAIS E DETECÇÃO DE VALORES SUSPEITOS

Com os valores em memória, calculamos mínimo, máximo e média. Isso ajuda a identificar rapidamente se os números estão dentro de uma faixa plausível.

```
mde_min = np.nanmin(mde_raw)
mde_max = np.nanmax(mde_raw)
mde_mean = np.nanmean(mde_raw)

mde_min, mde_max, mde_mean
#Saída: (np.float32(0.0), np.float32(389.69382), np.float32(247.5976))
```

O máximo e a média tendem a ser plausíveis para um MDE. O que costuma chamar atenção é quando o mínimo aparece como 0, porque em modelos de elevação isso raramente representa uma altitude real. Na prática, esse valor pode ser usado como “preenchimento” em áreas sem informação, especialmente quando o arquivo não declara NoData.

Antes de corrigir, é importante verificar se esses zeros aparecem em quantidade relevante.

```
qtd_zeros = np.sum(mde_raw == 0)
perc_zeros = (qtd_zeros / mde_raw.size) * 100
print("Pixels com valor 0:", qtd_zeros)
print(f"Percentual de zeros: {perc_zeros:.2f}%")
```

```
#Saída:
Pixels com valor 0: 1492
Percentual de zeros: 0.27%
```

Mesmo quando esse percentual é pequeno, ele pode distorcer estatísticas sensíveis, como o valor mínimo, e afetar análises que dependem de extremos ou intervalos.

7.4 HISTOGRAMA: CONFIRMANDO O PADRÃO NUMÉRICO

O histograma é uma ferramenta simples para enxergar o comportamento numérico do *raster*. Aqui, o objetivo é identificar padrões artificiais — por exemplo, um pequeno pico isolado perto de zero.

```
plt.figure(figsize=(6, 4))
plt.hist(mde_raw.flatten(), bins=50)
plt.xlabel("Altitude (m)")
plt.ylabel("Frequência")
plt.title("Distribuição dos valores do MDE — arquivo original")
plt.show()
```

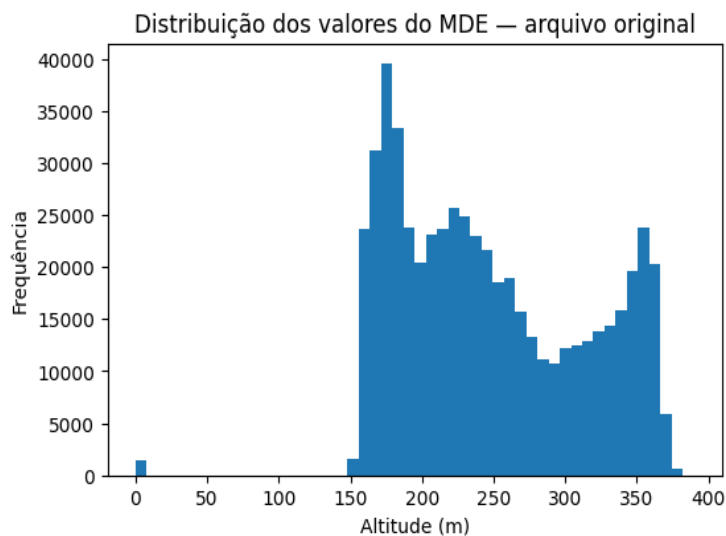


Figura 7.2: Histograma dos valores do MDE (arquivo original).

Se houver um acúmulo separado perto de 0, isso sugere fortemente que esses *pixels* não representam altitude real, mas ausência de dado codificada como um valor numérico comum.

7.5 NODATA, NAN E A CORREÇÃO EM MEMÓRIA

Em *rasters*, três situações aparecem com frequência:

NoData: valor inválido declarado no metadado do arquivo (`src.nodata`).

NaN: representação de ausência de dado em memória (NumPy).

Valores numéricos comuns: como 0, que pode ser válido ou ser usado como preenchimento, dependendo do dataset.

Vamos checar se o arquivo declara NoData e se já existem NaNs no array.

```
with rio.open(mde_path) as src:
    print("NoData (metadado):", src.nodata)
print("Quantidade de NaN no array:", np.isnan(mde_raw).sum())
```

#Saída:

NoData (metadado): None

Quantidade de NaN no array: 0

No nosso caso, o arquivo não declara NoData e o array não tem NaN. Isso significa que todos os *pixels* estão sendo tratados como válidos. Como o diagnóstico (mínimo e histograma) sugere que 0 está sendo usado como preenchimento neste arquivo, vamos tratá-lo como inválido em memória.

Em outros *rasters*, 0 pode ser um valor válido. Aqui, a decisão é guiada pelo diagnóstico numérico.

```
mde_clean = mde_raw.astype("float32").copy()
mde_clean[mde_clean == 0] = np.nan
print("Mínimo (ignorando zeros):", np.nanmin(mde_clean))
print("Máximo (ignorando zeros):", np.nanmax(mde_clean))
```

#Saída:

Mínimo (ignorando zeros): 153.25703

Máximo (ignorando zeros): 389.69382

Se o mínimo passa para uma faixa coerente (por exemplo, acima de 100 m, dependendo da área), isso confirma que aqueles zeros não eram altitudes reais.

7.6 SALVANDO UM GEOTIFF “LIMPO” COM NODATA EXPLÍCITO

Em memória, NaN é excelente para representar ausência de dado. Em arquivo, é mais seguro salvar um NoData numérico (por exemplo, -9999.0), amplamente aceito por softwares GIS.

```
mde_clean_path = dados_dir / "mde_corrigido.tif"
```

```
nodata_saida = -9999.0
```

```
with rio.open(mde_path) as src:
```

```
    profile_out = src.profile.copy()
```

```
    profile_out.update(dtype=rio.float32, count=1, nodata=nodata_saida)
```

```
mde_to_save = np.where(np.isnan(mde_clean), nodata_saida, mde_clean).astype("float32")
```

```
with rio.open(mde_clean_path, "w", **profile_out) as dst:
```

```
    dst.write(mde_to_save, 1)
```

```
print("Arquivo corrigido salvo em:", mde_clean_path)
```

7.7 REABRINDO E VALIDANDO A VERSÃO CORRIGIDA

Apartir daqui, trabalharemos sempre com o arquivo corrigido. Ao reabrir, convertemos o NoData numérico para NaN em memória.

```
with rio.open(mde_clean_path) as src:
```

```
    mde = src.read(1).astype("float32")
```

```
    mde[mde == src.nodata] = np.nan
```

Repetimos as visualizações como checagem final: mapa e histograma devem refletir apenas valores válidos.

```
plt.figure(figsize=(6, 6))  
plt.imshow(mde, cmap="cividis")  
plt.colorbar(label="Altitude (m)")  
plt.title("MDE — corrigido")  
plt.axis("off")  
plt.show()
```

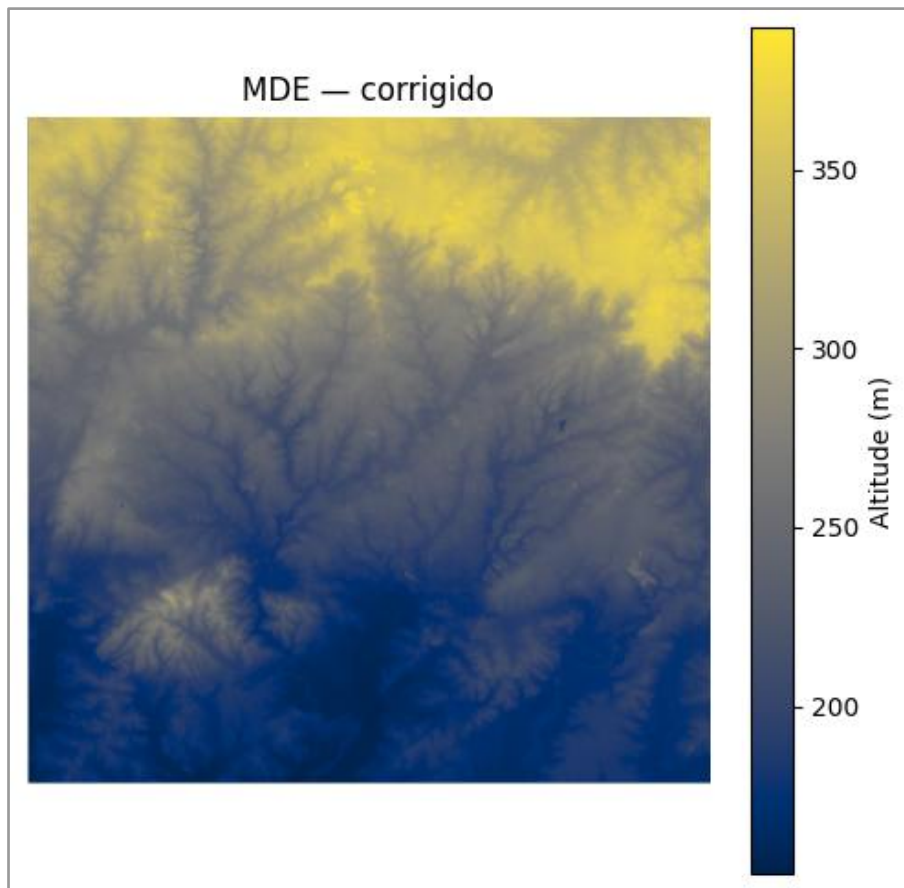


Figura 7.3: Visualização do MDE após correção do NoData.

```
vals = mde[~np.isnan(mde)]  
plt.figure(figsize=(6, 4))  
plt.hist(vals, bins=50)  
plt.xlabel("Altitude (m)")  
plt.ylabel("Frequência")  
plt.title("Distribuição dos valores do MDE — corrigido")  
plt.show()
```

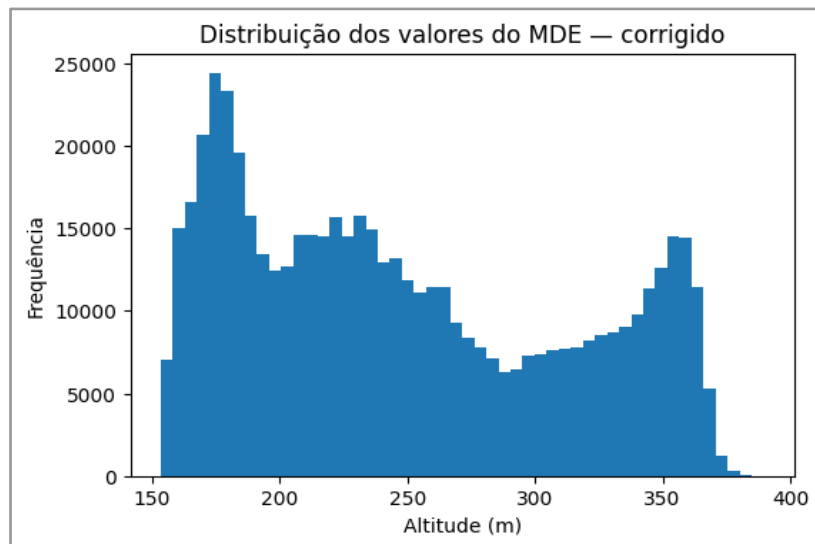


Figura 7.4: Histograma dos valores do MDE após correção.

Por fim, confirmamos se o GeoTIFF foi gravado com os metadados esperados.

```
with rio.open(mde_clean_path) as src:
    print("Driver:", src.profile["driver"])
    print("Dimensões (largura x altura):", src.width, "x", src.height)
    print("Número de bandas:", src.count)
    print("CRS:", src.crs)
    print("Resolução espacial:", src.res)
    print("NoData (arquivo):", src.nodata)
    print("Transform:", src.transform)
```

#Saída:

```
Driver: GTiff
Dimensões (largura x altura): 754 x 739
Número de bandas: 1
CRS: EPSG:31981
Resolução espacial: (30.0, 30.0)
NoData (arquivo): -9999.0
Transform: | 30.00, 0.00, 767430.00|
| 0.00,-30.00, 6543030.00|
| 0.00, 0.00, 1.00|
```

7.8 ESTATÍSTICAS FINAIS E COBERTURA VÁLIDA

Agora calculamos estatísticas ignorando *pixels* inválidos (NaN em memória).

```
print("Mínimo (válidos):", np.nanmin(mde))
print("Máximo (válidos):", np.nanmax(mde))
print("Média (válidos):", np.nanmean(mde))
print("Desvio padrão (válidos):", np.nanstd(mde))
```

#Saída:

Mínimo (válidos): 153.25703

Máximo (válidos): 389.69382

Média (válidos): 248.26234

Desvio padrão (válidos): 64.28669

Também vale checar a cobertura: quantos *pixels* são válidos.

```
qtd_validos = np.sum(~np.isnan(mde))
qtd_total = mde.size
perc_validos = (qtd_validos / qtd_total) * 100
print("Pixels válidos:", qtd_validos)
print("Pixels totais:", qtd_total)
print(f"Percentual válido: {perc_validos:.2f}%")
```

#Saída: Pixels válidos: 555714

Pixels totais: 557206

Percentual válido: 99.73%

Para fechar o diagnóstico, conferimos a extensão espacial do *raster* (útil para recortes no próximo capítulo).

```
with rio.open(mde_clean_path) as src:
    print("Bounds (xmin, ymin, xmax, ymax):", src.bounds)
```

#Saída:

Bounds (xmin, ymin, xmax, ymax): BoundingBox(left=767430.0, bottom=6520860.0, right=790050.0, top=6543030.0)

Com isso, encerramos o diagnóstico do MDE: o *raster* foi lido corretamente, os metadados foram conferidos, valores suspeitos foram identificados e tratados, e o arquivo corrigido foi salvo com NoData explícito. A partir do próximo capítulo, trabalharemos sempre com *mde_corrigido.tif*, garantindo estatísticas e visualizações coerentes durante recortes, máscaras e derivados.

PROCESSAMENTO E TRANSFORMAÇÃO DE DADOS RASTER

Abrir no Colab

https://colab.research.google.com/github/Alexandrogschafer/geoprocessamento-python-livro/blob/main/notebooks/geo_python_capitulo8.ipynb

No Capítulo 7, o foco foi compreender o *raster* como estrutura de dados: uma matriz de valores associada a uma posição no espaço. Aprendemos a inspecionar metadados, interpretar CRS, identificar valores inválidos e visualizar corretamente um arquivo *raster*. Esse diagnóstico é indispensável — mas, em projetos reais, ele é apenas o começo. O *raster* quase nunca é usado “como vem”: ele precisa ser transformado.

Transformar um *raster* significa adaptá-lo ao problema que estamos estudando. Isso pode envolver reduzir a extensão espacial, aplicar máscaras, ajustar valores, salvar versões derivadas e, quando necessário, reprojetar para integrar com outras bases. Neste capítulo, vamos praticar essas operações de forma controlada, sempre seguindo a lógica didática do livro: entender a operação → aplicar o código → ver o resultado no mapa → conferir algo simples (metadados/valores) para validar.

Vamos trabalhar com três arquivos reais do repositório do livro:

mde_corrigido.tif: Modelo Digital de Elevação (MDE) já corrigido (altitude).

bacia_arroio_bage.gpkg: polígono da bacia do Arroio Bagé (vetor).

sentinel2_20250220.tif: GeoTIFF multibanda (quatro bandas empilhadas) para primeiros contatos com *rasters* multibanda.

O MDE e a bacia estão em EPSG:31981, o que simplifica recortes e medições em metros. Já o *raster* Sentinel está em EPSG:4326; por isso, quando formos recortá-lo pela bacia, precisaremos reprojetar o vetor para o CRS do *raster*.

8.1 ABRINDO OS DADOS DO REPOSITÓRIO

Para garantir reprodutibilidade tanto no ambiente local quanto no Google Colab®, vamos baixar os dados diretamente do GitHub usando links “raw”. Começamos importando bibliotecas e criando a pasta dados/.

```
from pathlib import Path
import geopandas as gpd
import rasterio as rio
import numpy as np
import matplotlib.pyplot as plt
```

```
from rasterio.plot import show
```

Agora definimos as URLs e os caminhos locais:

```
# URLs "raw" dos arquivos no repositório do livro
url_mde = (
    "https://raw.githubusercontent.com/"
    "Alexandrogschafer/geoprocessamento-python-livro/main/"
    "dados/mde_corrigido.tif")
```

```
url_bacia = (
    "https://raw.githubusercontent.com/"
    "Alexandrogschafer/geoprocessamento-python-livro/main/"
    "dados/bacia_arroio_bage.gpkg")
```

```
url_sentinel = (
    "https://raw.githubusercontent.com/"
    "Alexandrogschafer/geoprocessamento-python-livro/main/"
    "dados/sentinel2_20250220.tif")
```

```
# Pasta de dados
dados_dir = Path("dados")
```

```
dados_dir.mkdir(exist_ok=True)

# Caminhos locais
mde_path = dados_dir / "mde_corrigido.tif"
bacia_path = dados_dir / "bacia_arroio_bage.gpkg"
sentinel_path = dados_dir / "sentinel2_20250220.tif"
```

Fazemos o download (se necessário). No Colab®, wget é uma forma prática e estável:

```
if not mde_path.exists():
    !wget -q -O "{mde_path}" "{url_mde}"

if not bacia_path.exists():
    !wget -q -O "{bacia_path}" "{url_bacia}"

if not sentinel_path.exists():
    !wget -q -O "{sentinel_path}" "{url_sentinel}"

print("Arquivos baixados em:", dados_dir.resolve())
print("MDE:", mde_path.exists(), "| Bacia:", bacia_path.exists(), "| Sentinel:",
sentinel_path.exists())
```

Com os arquivos disponíveis, abrimos o vetor e verificamos o CRS do *raster*.

```
gdf_bacia = gpd.read_file(bacia_path)
with rio.open(mde_path) as src:
    print("CRS do vetor :", gdf_bacia.crs)
    print("CRS do raster:", src.crs)

#Saída:
CRS do vetor : EPSG:31981
CRS do raster: EPSG:31981
```

Como ambos estão em EPSG:31981, não precisamos reprojetar nada para trabalhar com o MDE e a bacia. Antes de processar, vale visualizar as camadas juntas. Esse primeiro mapa é uma checagem visual: ele confirma que o *raster* foi lido corretamente e que o polígono está na área esperada.

```
with rio.open(mde_path) as src:
    mde = src.read(1).astype("float32")
    if src.nodata is not None:
        mde[mde == src.nodata] = np.nan

fig, ax = plt.subplots(1, 1, figsize=(8, 7))
show(src, ax=ax)
gdf_bacia.boundary.plot(ax=ax, linewidth=2, color="black")
ax.set_title("MDE corrigido e contorno da bacia do Arroio Bagé")
plt.show()
```

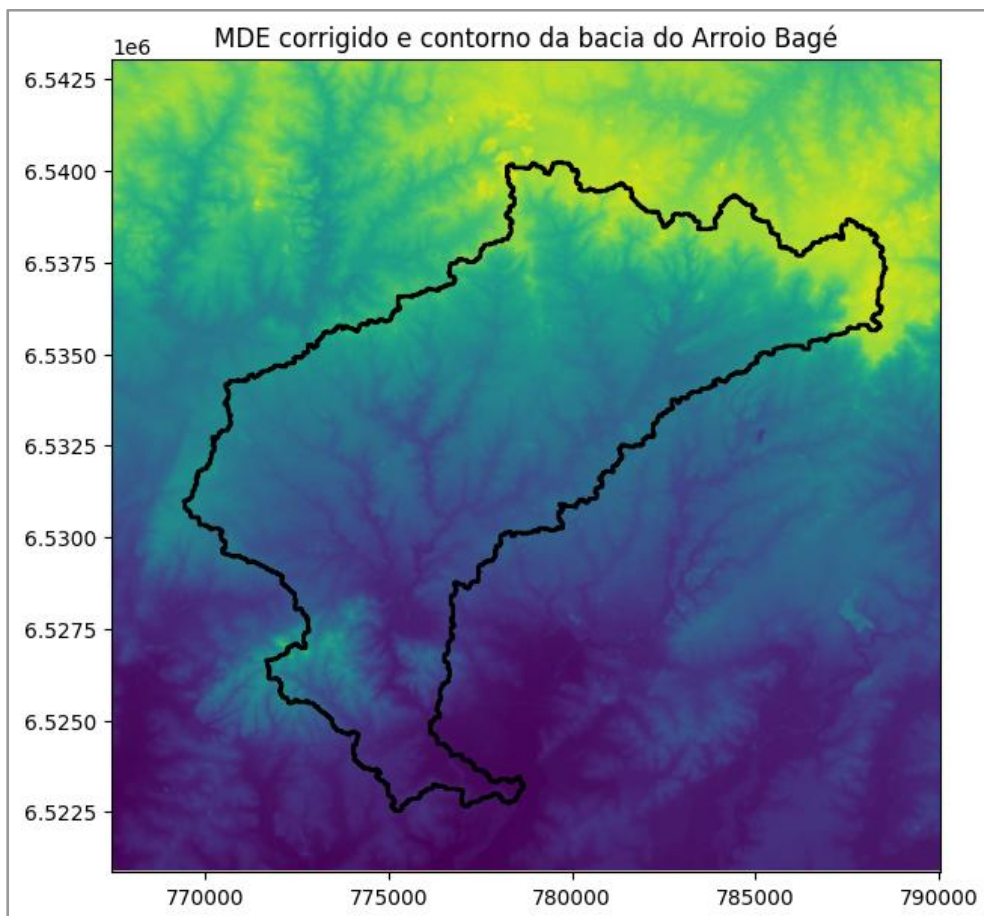


Figura 8.1: MDE corrigido e contorno da bacia: verificação inicial de sobreposição.

8.2 RECORTE ESPACIAL DE RASTERS

Uma das operações mais comuns em geoprocessamento é reduzir o *raster* à área que realmente interessa. Trabalhar com o arquivo inteiro é possível, mas geralmente é desnecessário: aumenta o tempo de processamento, pesa a visualização e torna o fluxo menos objetivo.

Nesta seção, vamos recortar o *mde_corrigido.tif* de duas maneiras:

1. Recorte por caixa delimitadora (bounding box): rápido e útil para reduzir a área de trabalho.
2. Recorte por polígono (máscara pela bacia): recorte preciso, mantendo apenas *pixels* dentro da bacia.

8.2.1 RECORTE POR BOUNDING BOX

A caixa delimitadora é o menor retângulo capaz de conter a bacia. O GeoPandas fornece esses limites com *total_bounds*. A partir deles, o Rasterio cria uma janela (*window*) e lê apenas aquela parte do *raster*.

```
from rasterio.windows import from_bounds
minx, miny, maxx, maxy = gdf_bacia.total_bounds
with rio.open(mde_path) as src:
    win = from_bounds(minx, miny, maxx, maxy, transform=src.transform)
    mde_bbox = src.read(1, window=win).astype("float32")
    transform_bbox = src.window_transform(win)
    fig, ax = plt.subplots(1, 1, figsize=(8, 7))
    show(mde_bbox, transform=transform_bbox, ax=ax)
    gdf_bacia.boundary.plot(ax=ax, linewidth=2, color="black")
    ax.set_title("MDE recortado por bounding box")
    plt.show()
```

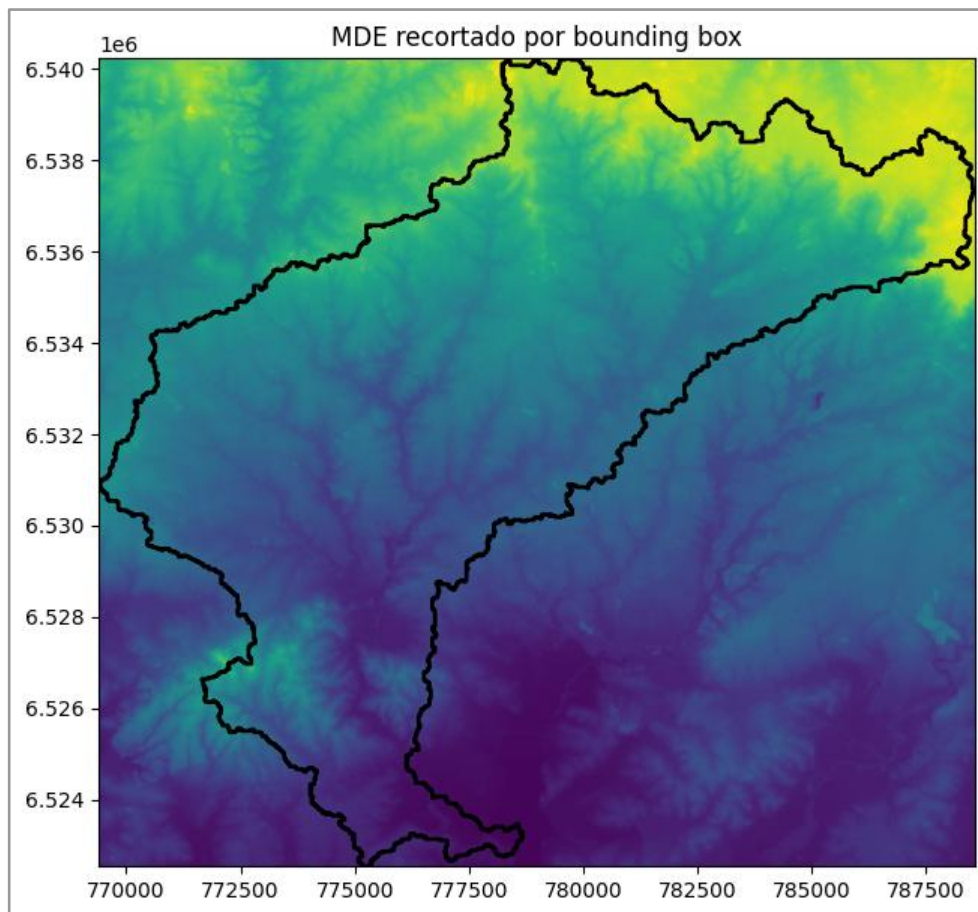


Figura 8.2 — Recorte do MDE por caixa delimitadora (bounding box) da bacia.

O resultado é um *raster* menor e retangular. Ele ainda inclui *pixels* fora da bacia (porque a bacia é irregular), mas já reduz bastante o volume de dados.

8.2.2 RECORTE POR POLÍGONO (MÁSCARA PELA BACIA)

Agora fazemos o recorte preciso: mantemos apenas os *pixels* dentro do polígono da bacia. Para isso, usamos `mask()`, que corta e mascara ao mesmo tempo.

```
from rasterio.mask import mask
geoms = gdf_bacia.geometry.values
with rio.open(mde_path) as src:
    mde_clip, transform_clip = mask(src, geoms, crop=True)
    # mask() devolve (bandas, linhas, colunas)
    mde_clip = mde_clip[0].astype("float32")
    # Em memória, trabalhamos com NaN
```

```

if src.nodata is not None:
    mde_clip[mde_clip == src.nodata] = np.nan
fig, ax = plt.subplots(1, 1, figsize=(8, 7))
show(mde_clip, transform=transform_clip, ax=ax)
gdf_bacia.boundary.plot(ax=ax, linewidth=2, color="black")
ax.set_title("MDE recortado por polígono (bacia)")
plt.show()

```

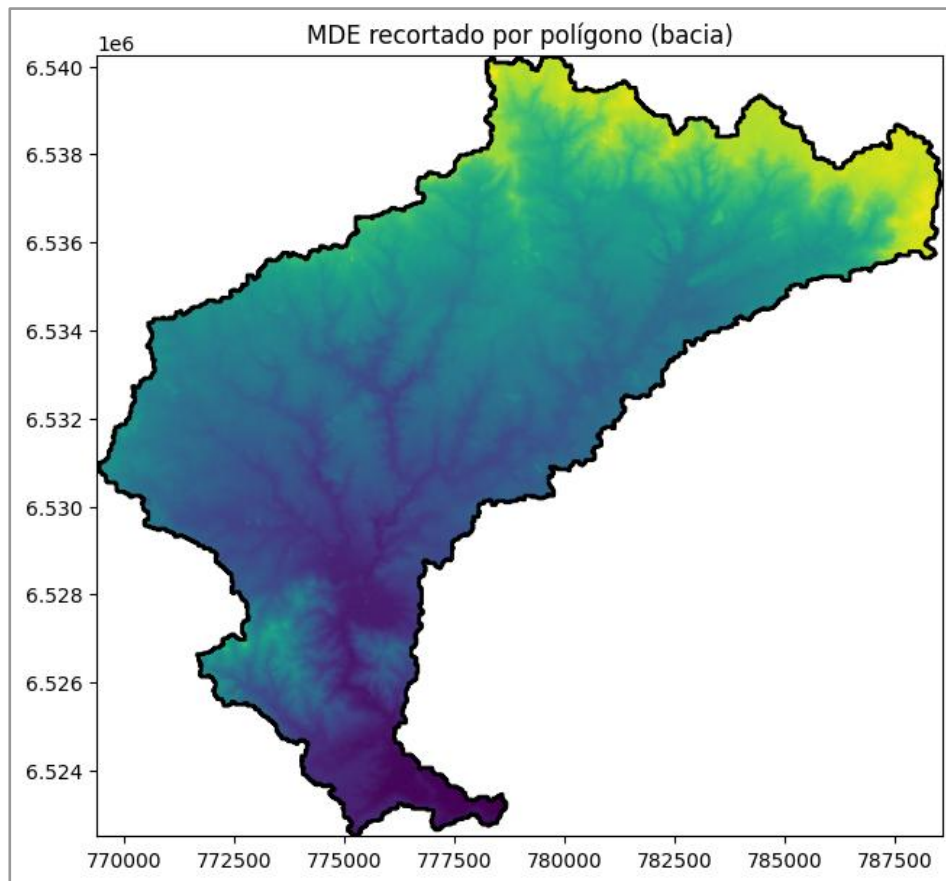


Figura 8.3 — Recorte do MDE por polígono da bacia (máscara espacial).

A partir daqui o *raster* já está ajustado à bacia — um formato muito mais conveniente para análises posteriores.

8.3 SALVANDO O RASTER RECORTADO PELA BACIA

Depois de recortar, o passo natural em um fluxo real é salvar o resultado em disco. Isso evita repetir o recorte e permite reutilizar o *raster* em outros notebooks ou em um SIG.

Para salvar, copiamos o profile do *raster* original e atualizamos o que mudou: dimensões (height, width) e transform. Também garantimos que exista um NoData explícito no arquivo de saída. A regra prática deste livro é simples:

Em memória: usamos NaN para facilitar estatísticas.

Em disco (GeoTIFF): usamos um NoData numérico (por exemplo, -9999.0).

```
mde_bacia_path = dados_dir / "mde_bacia.tif"
```

```
nodata_out = -9999.0
```

```
with rio.open(mde_path) as src:
```

```
    profile_out = src.profile.copy()
```

```
    profile_out.update(
```

```
        height=mde_clip.shape[0],
```

```
        width=mde_clip.shape[1],
```

```
        transform=transform_clip,
```

```
        count=1
```

```
    )
```

```
    if profile_out.get("nodata") is None:
```

```
        profile_out.update(nodata=nodata_out)
```

```
# Converte NaN -> NoData numérico para escrita
```

```
mde_to_save = mde_clip.copy()
```

```
mde_to_save = np.where(np.isnan(mde_to_save), profile_out["nodata"],  
mde_to_save).astype("float32")
```

```
with rio.open(mde_bacia_path, "w", **profile_out) as dst:
```

```
    dst.write(mde_to_save, 1)
```

```
print("Arquivo salvo:", mde_bacia_path)
```

Abrimos e visualizamos rapidamente o arquivo recém-criado para confirmar que tamanho, posição e recorte ficaram corretos:

with rio.open(mde_bacia_path) as chk:

```
fig, ax = plt.subplots(1, 1, figsize=(8, 7))
```

```
show(chk, ax=ax)
```

```
gdf_bacia.boundary.plot(ax=ax, linewidth=2, color="black")
```

```
ax.set_title("MDE recortado pela bacia (arquivo salvo)")
```

```
plt.show()
```

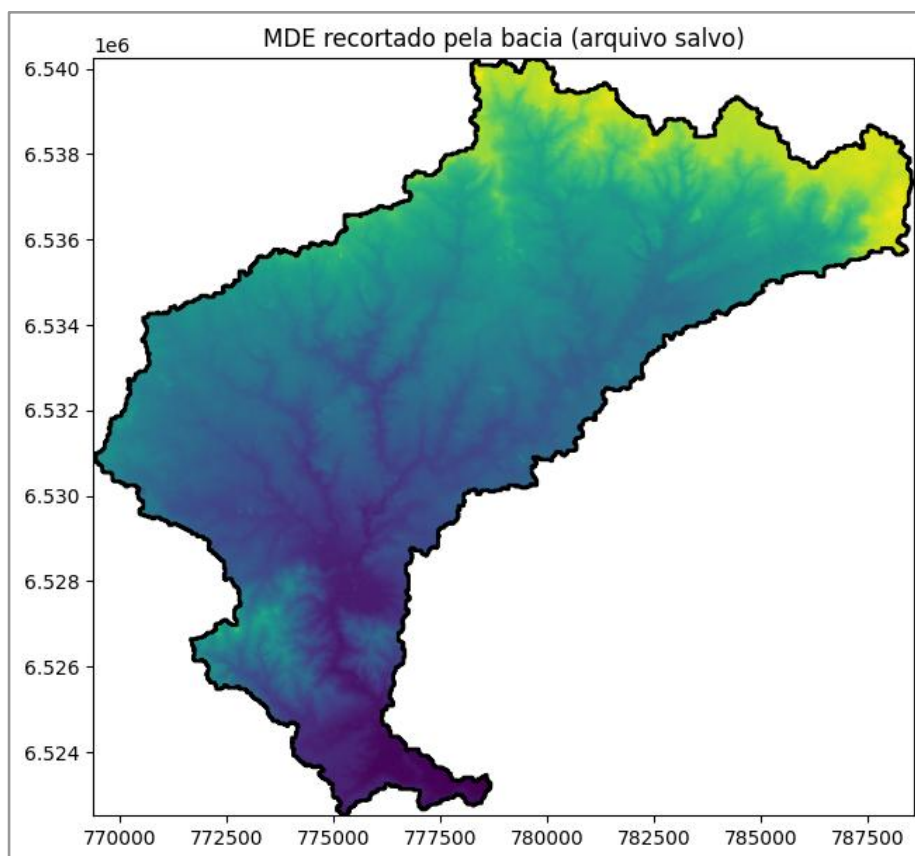


Figura 8.4 — Verificação visual do MDE recortado (arquivo salvo).

8.4 REPROJEÇÃO DE RASTERS E ESCOLHA DO MÉTODO DE REAMOSTRAGEM

Em alguns fluxos de trabalho, é necessário gerar versões do *raster* em outros sistemas de referência. Diferentemente do vetor, reprojetar *raster* exige reconstruir a grade de *pixels* e redistribuir valores. Esse processo é a reamostragem.

Uma regra prática resolve a maior parte dos casos:

- Variáveis contínuas (altitude, temperatura, precipitação): use métodos interpolativos (bilinear ou cubic).
- Variáveis categóricas (classes de uso do solo): use nearest (vizinho mais próximo).

Como altitude é contínua, vamos reprojetar o mde_bacia.tif para EPSG:4326 usando reamostragem bilinear.

```
from rasterio.warp import calculate_default_transform, reproject, Resampling

dst_crs = "EPSG:4326"
mde_bacia_4326_path = dados_dir / "mde_bacia_4326.tif"

with rio.open(mde_bacia_path) as src:
    src_arr = src.read(1).astype("float32")
    nodata_src = src.nodata if src.nodata is not None else -9999.0

    # Garante array numérico com NoData explícito (sem NaN) para reprojeção
    src_arr = np.where(np.isnan(src_arr), nodata_src, src_arr)

    bounds = rio.transform.array_bounds(src.height, src.width, src.transform)

    transform_dst, width_dst, height_dst = calculate_default_transform(
        src.crs, dst_crs, src.width, src.height, *bounds)

    profile_dst = src.profile.copy()
    profile_dst.update(crs=dst_crs, transform=transform_dst, width=width_dst,
        height=height_dst, nodata=nodata_src, dtype="float32")

    dst_arr = np.full((height_dst, width_dst), nodata_src, dtype="float32")

    reproject(source=src_arr, destination=dst_arr, src_transform=src.transform,
        src_crs=src.crs, dst_transform=transform_dst, dst_crs=dst_crs,
        resampling=Resampling.bilinear, src_nodata=nodata_src, dst_nodata=nodata_src)

with rio.open(mde_bacia_4326_path, "w", **profile_dst) as dst:
    dst.write(dst_arr, 1)

print("Raster reprojetado salvo em:", mde_bacia_4326_path)
```

Fazemos uma checagem rápida para confirmar o CRS e a resolução (agora em graus):

```
with rio.open(mde_bacia_4326_path) as r4326:  
    fig, ax = plt.subplots(1, 1, figsize=(8, 7))  
    show(r4326, ax=ax)  
    ax.set_title("MDE reprojetoado para EPSG:4326")  
    plt.show()  
  
    print("CRS:", r4326.crs)  
    print("Resolução (em graus):", r4326.res)
```

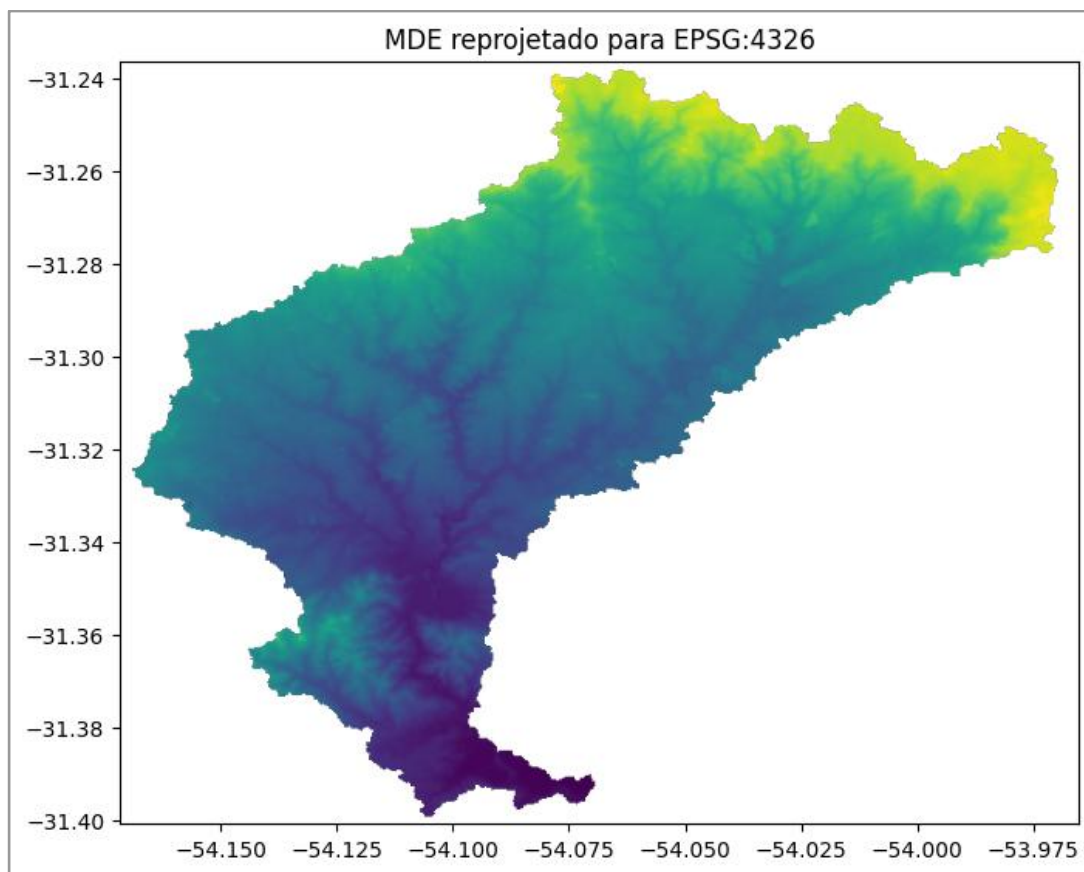


Figura 8.5 — MDE da bacia reprojetoado para EPSG:4326 com reamostragem bilinear.

CRS: EPSG:4326

Resolução (em graus): (0.00029648608964429296, 0.00029648608964429296)

Repare que, em EPSG:4326, a resolução passa a ser expressa em graus. Esse CRS é útil para interoperabilidade e visualização, mas análises métricas (distância, área, declividade) continuam sendo mais adequadas em CRS projetados (como UTM).

8.5 RASTERS MULTIBANDA: CONCEITO E PRIMEIROS CONTATOS

Até aqui, trabalhamos com *raster* de uma banda. Em imagens de satélite, é comum ter várias bandas no mesmo arquivo: várias matrizes empilhadas, perfeitamente alinhadas na mesma grade espacial. Neste ponto, a ideia não é discutir sensor ou comprimentos de onda, e sim entender como o Rasterio representa e acessa um arquivo multibanda.

Começamos verificando quantas bandas existem e qual o CRS do arquivo Sentinel:

with `rio.open(sentinel_path)` as `src`:

```
print("Número de bandas:", src.count)
print("Resolução:", src.res)
print("CRS:", src.crs)
```

#Saída:

Número de bandas: 4

Resolução: (0.00010244727807748734, 9.278019586507233e-05)

CRS: EPSG:4326

Como este *raster* está em EPSG:4326, ele não está no mesmo CRS do polígono da bacia (EPSG:31981). Quando formos recortar a imagem pela bacia, vamos reprojetar o vetor para o CRS do *raster*.

No Rasterio, o índice das bandas começa em 1 (e não em 0). Além disso, o número usado no `read()` indica a posição da banda dentro do arquivo, não o nome original no catálogo do sensor. No nosso GeoTIFF preparado, as quatro bandas armazenadas são B02, B03, B04 e B08, nessa ordem. Assim:

```
read(1) → primeira banda do arquivo (B02)
read(2) → segunda banda (B03)
read(3) → terceira banda (B04)
read(4) → quarta banda (B08)
```

Vamos visualizar uma banda isolada para confirmar a leitura.

```
with rio.open(sentinel_path) as src:
    b02 = src.read(1).astype("float32")

fig, ax = plt.subplots(figsize=(6, 6))
show(b02, ax=ax)
ax.set_title("B02 (banda 1 do arquivo)")
plt.show()
```

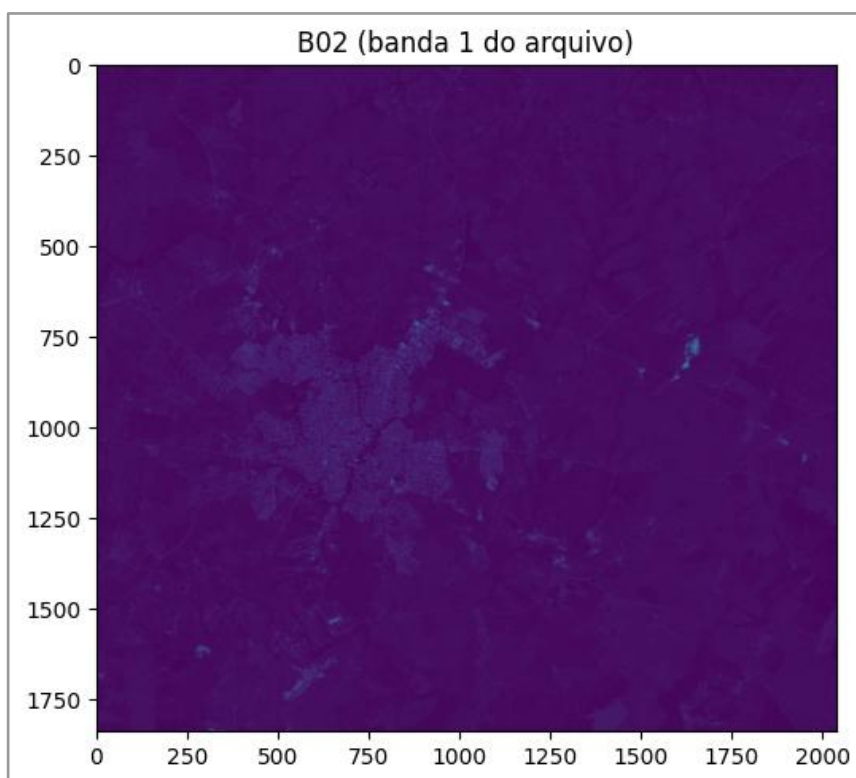


Figura 8.6 — Visualização de uma banda individual do *raster* Sentinel-2 (B02).

Uma visualização muito comum é a composição RGB: colocamos três bandas nos canais vermelho (R), verde (G) e azul (B). Neste arquivo, usamos:

R = B04 → read(3)

G = B03 → read(2)

B = B02 → read(1)

```
def normalizar(banda):
    p2, p98 = np.percentile(banda, (2, 98))
```

```
banda = np.clip(banda, p2, p98)
return (banda - p2) / (p98 - p2 + 1e-9)

with rio.open(sentinel_path) as src:
    r = src.read(3).astype("float32")
    g = src.read(2).astype("float32")
    b = src.read(1).astype("float32")

rgb_norm = np.dstack([normalizar(r), normalizar(g), normalizar(b)])

fig, ax = plt.subplots(figsize=(7, 7))
ax.imshow(rgb_norm)
ax.set_title("Composição RGB (B04, B03, B02)")
ax.set_axis_off()
plt.show()
```

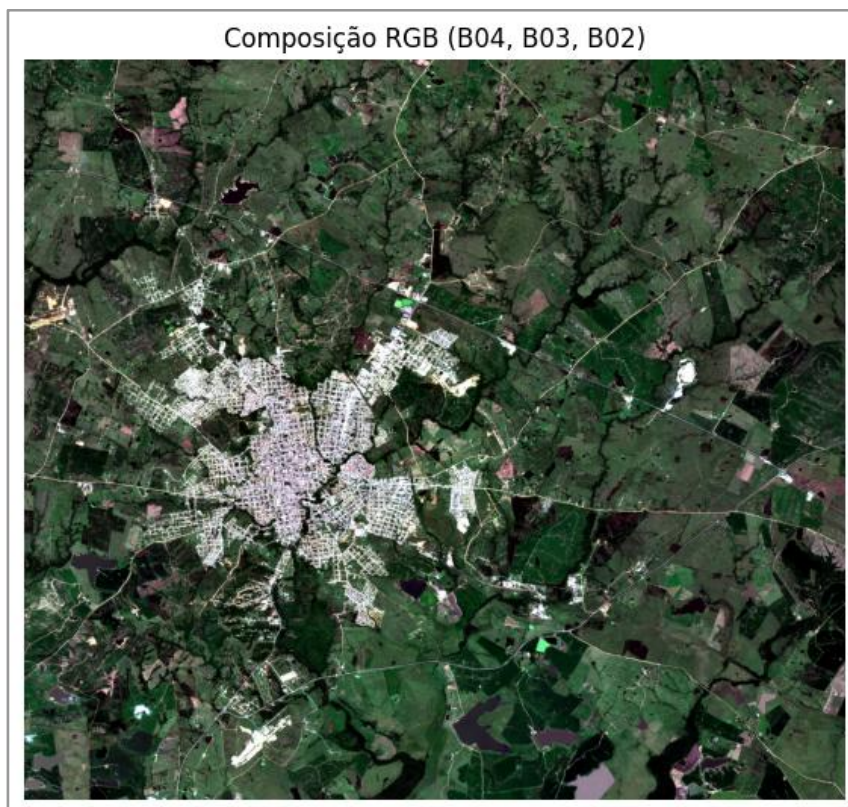


Figura 8.7 — Composição RGB da imagem Sentinel-2 (B04, B03, B02).

Agora repetimos a integração *raster-vetor*: recortamos a imagem Sentinel usando o polígono da bacia. Como o Sentinel está em EPSG:4326, primeiro reprojeta a bacia para esse CRS.

```
from rasterio.mask import mask
with rio.open(sentinel_path) as src:
    gdf_bacia_4326 = gdf_bacia.to_crs(src.crs)
    out_img, out_transform = mask(src, gdf_bacia_4326.geometry, crop=True)

# out_img tem forma (bandas, linhas, colunas)
r = out_img[2].astype("float32") # banda 3 do arquivo (B04)
g = out_img[1].astype("float32") # banda 2 do arquivo (B03)
b = out_img[0].astype("float32") # banda 1 do arquivo (B02)

rgb_clip = np.dstack([normalizar(r), normalizar(g), normalizar(b)])

fig, ax = plt.subplots(figsize=(7, 7))
ax.imshow(rgb_clip)
ax.set_title("Composição RGB recortada pela bacia")
ax.set_axis_off()
plt.show()
```

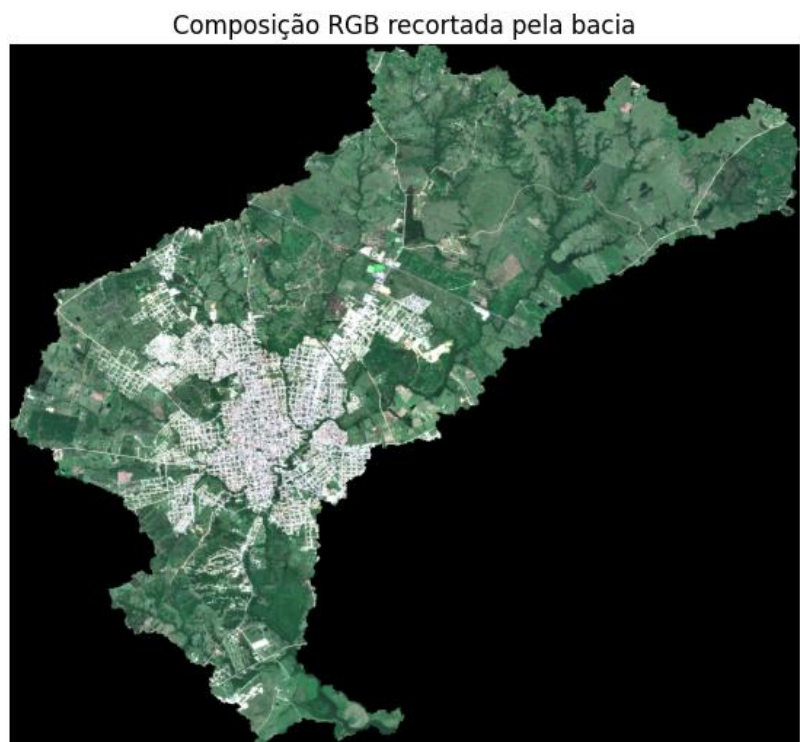


Figura 8.8 — Composição RGB da imagem Sentinel-2 recortada pela bacia.

Esse passo fecha o capítulo mostrando um fluxo completo e coerente: abrir dados, recortar por polígono, salvar produtos e, em seguida, aplicar a mesma lógica a um *raster* multibanda — mantendo o alinhamento entre bandas e restringindo tudo à área de interesse.

INTEGRAÇÃO RASTER + VETOR: ESTATÍSTICA ZONAL

Abrir no Colab

https://colab.research.google.com/github/Alexandrogschafer/geoprocessamento-python-livro/blob/main/notebooks/geo_python_capitulo9.ipynb

Nos capítulos anteriores, aprendemos a abrir, inspecionar e transformar *rasters*. Vimos que esse tipo de dado descreve o espaço *pixel a pixel*, sendo ideal para representar fenômenos contínuos como altitude, temperatura ou precipitação. Em muitas análises, porém, trabalhar diretamente no nível do *pixel* não é o objetivo final. O que normalmente queremos é resumir essa informação dentro de unidades territoriais bem definidas — como bacias hidrográficas, bairros, municípios ou talhões — para comparar áreas e apoiar decisões.

É exatamente nesse ponto que entra a estatística zonal. Em vez de analisar milhares (ou milhões) de *pixels* individualmente, calculamos estatísticas — como média, mínimo, máximo e desvio padrão — dentro de cada polígono de interesse. O resultado é uma tabela enxuta e interpretável: cada linha representa uma zona e cada coluna representa uma medida-resumo. Essa mudança de escala marca uma virada importante no fluxo de trabalho: saímos do mundo do *pixel* e passamos para o mundo do território.

Neste capítulo, vamos aplicar esse método de forma progressiva e prática, usando dados reais do repositório do livro. O objetivo é dominar o fluxo completo: carregar os dados, verificar o alinhamento espacial, calcular estatísticas zonais primeiro para uma única área e, em seguida, automatizar o procedimento para todas as sub bacias.

9.1 DADOS UTILIZADOS E PREPARAÇÃO INICIAL

Vamos trabalhar com dois arquivos:

- um Modelo Digital de Elevação (MDE) já corrigido (`mde_corrigido.tif`);
- um arquivo vetorial com quatro subbacias (`subbaciasbg.gpkg`).

Como nos capítulos anteriores, vamos baixar os dados diretamente do GitHub para garantir reprodutibilidade em qualquer ambiente (local ou Google Colab®).

```
from pathlib import Path
import geopandas as gpd
import rasterio as rio
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from rasterio.plot import show
from rasterio.mask import mask
```

Definimos as URLs e os caminhos locais:

```
url_mde = (
    "https://raw.githubusercontent.com/"
    "Alexandrogschafer/geoprocessamento-python-livro/main/"
    "dados/mde_corrigido.tif"
)
```

```
url_subbacias = (
    "https://raw.githubusercontent.com/"
    "Alexandrogschafer/geoprocessamento-python-livro/main/"
    "dados/subbacias.gpkg"
)
```

```
dados_dir = Path("dados")
dados_dir.mkdir(exist_ok=True)
```

```
mde_path = dados_dir / "mde_corrigido.tif"
subbacias_path = dados_dir / "subbacias.gpkg"
```

Fazemos o download, se necessário:

```
!wget -q -O "{mde_path}" "{url_mde}"
!wget -q -O "{subbacias_path}" "{url_subbacias}"
```

```
print("Arquivos baixados em:", dados_dir.resolve())
```

Agora abrimos o vetor das subbacias e o raster do MDE:

```
gdf_sub = gpd.read_file(subbacias_path)
```

```
with rio.open(mde_path) as src_mde:
```

```
    print("Número de subbacias:", len(gdf_sub))
```

```
    print("CRS subbacias:", gdf_sub.crs)
```

```
    print("CRS MDE:", src_mde.crs)
```

#Saída:

Número de subbacias: 4

CRS subbacias: EPSG:31981

CRS MDE: EPSG:31981

Ambos estão em EPSG:31981, o que garante que podemos integrá-los diretamente, sem reprojeção prévia.

Antes de qualquer cálculo, fazemos uma verificação visual simples, mas indispensável: visualizar o *raster* com o contorno das subbacias sobreposto. Esse mapa não é decorativo — ele serve para confirmar rapidamente que as camadas se sobrepõem corretamente.

```
with rio.open(mde_path) as src_mde:
```

```
    fig, ax = plt.subplots(1, 1, figsize=(8, 7))
```

```
    show(src_mde, ax=ax)
```

```
    gdf_sub.boundary.plot(
```

```
        ax=ax,
```

```
        linewidth=2,
```

```
        color="black"
```

```
)
```

```
ax.set_title("MDE e subbacias")
```

```
plt.show()
```

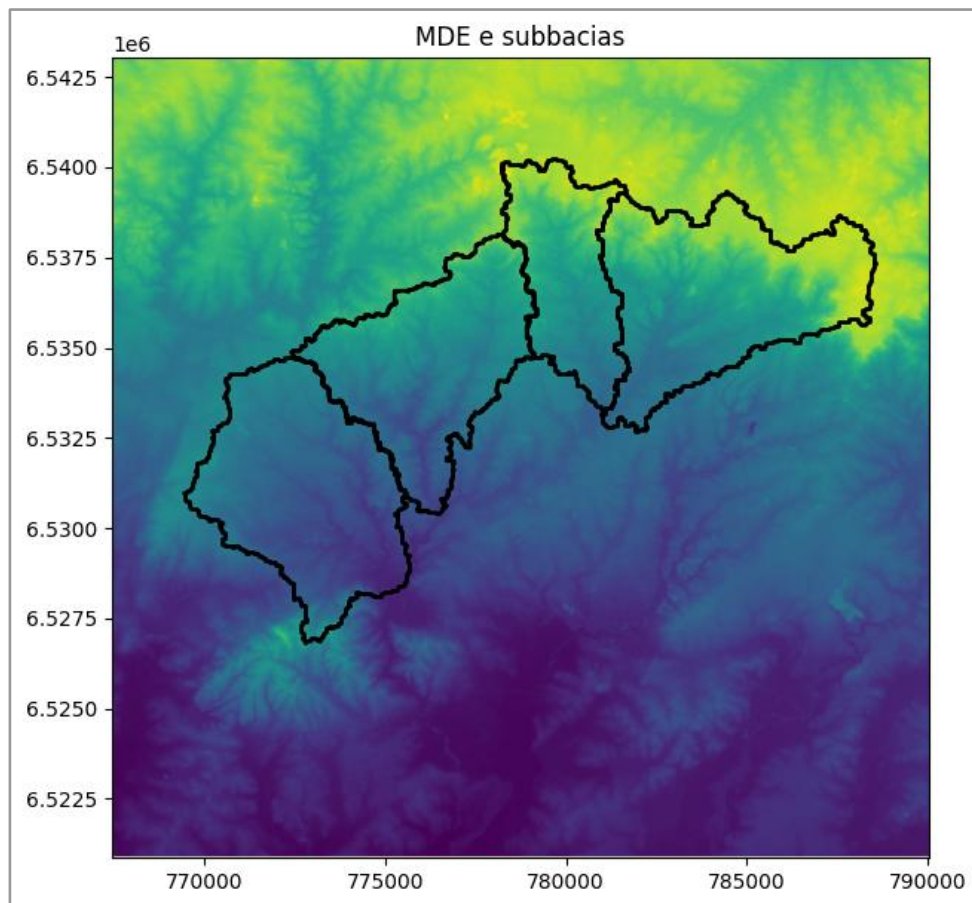


Figura 9.1 — MDE e subbacias sobrepostos para verificação espacial.

9.2 ESTATÍSTICA ZONAL: ENTENDENDO O PROCESSO EM UMA ÚNICA SUB BACIA

Antes de automatizar o cálculo para todas as áreas, é fundamental entender com clareza o que acontece em uma estatística zonal individual. Trabalhar primeiro com uma única sub-bacia permite visualizar o recorte do *raster*, compreender quais *pixels* entram no cálculo e interpretar os números obtidos. Esse passo evita erros conceituais e torna a automatização posterior muito mais segura.

Vamos começar selecionando apenas uma sub-bacia do conjunto:

```
sub = gdf_sub.iloc[[0]]
```

```
sub
```

	nome	id	geometry
0	bacia arroio tábua	1	MULTIPOLYGON (((778104.925 6538133.882, 778254...

Em seguida, recortamos o MDE usando o polígono dessa sub-bacia. O recorte é feito com `mask()`, que extrai apenas os *pixels* do *raster* que intersectam a geometria do polígono.

```
from rasterio.mask import mask
```

```
with rio.open(mde_path) as src_mde:
```

```
    mde_clip, transform_clip = mask(src_mde, sub.geometry, crop=True)
```

```
    # seleciona a banda e converte para float
```

```
    mde_clip = mde_clip[0].astype("float32")
```

```
    # converte NoData para NaN em memória
```

```
    if src_mde.nodata is not None:
```

```
        mde_clip[mde_clip == src_mde.nodata] = np.nan
```

Antes de calcular qualquer estatística, visualizamos o *raster* recortado. Esse mapa permite verificar se a área recortada corresponde de fato à sub-bacia escolhida.

```
fig, ax = plt.subplots(1, 1, figsize=(6, 6))
```

```
show(mde_clip, transform=transform_clip, ax=ax)
```

```
sub.boundary.plot(ax=ax, linewidth=2, color="black")
```

```
ax.set_title("MDE da subbacia selecionada")
```

```
plt.show()
```

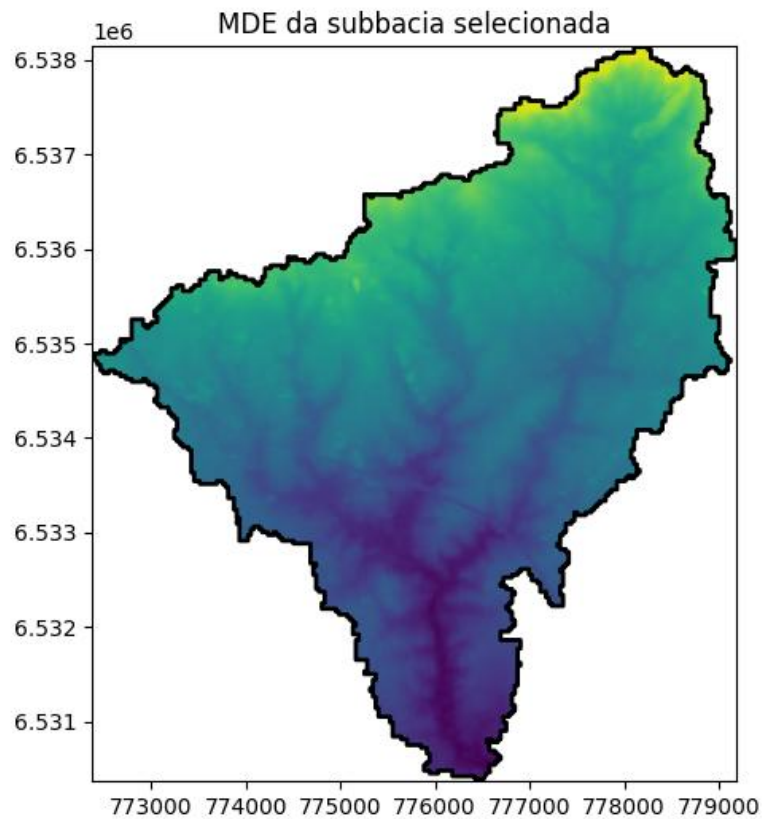


Figura 9.2 — MDE recortado da subbacia selecionada (checagem visual).

Agora, com os *pixels* da sub bacia isolados, calculamos estatísticas simples usando as funções `nan*` do NumPy, que ignoram automaticamente valores inválidos.

```
print("Altitude média:", np.nanmean(mde_clip))
print("Altitude mínima:", np.nanmin(mde_clip))
print("Altitude máxima:", np.nanmax(mde_clip))
print("Desvio padrão:", np.nanstd(mde_clip))
```

#Saída:

Altitude média: 261.6809

Altitude mínima: 194.22308

Altitude máxima: 355.89337

Desvio padrão: 30.277122

Nesse ponto, já realizamos uma estatística zonal completa, ainda que manualmente. Partimos de um *raster* contínuo e o resumimos em poucos números que representam o relevo de uma unidade territorial específica.

9.3 AUTOMATIZANDO A ESTATÍSTICA ZONAL PARA TODAS AS SUB BACIAS

Depois de entender o procedimento para uma única sub bacia, automatizar o cálculo é um passo natural. Em projetos reais, raramente trabalhamos com apenas uma zona; o mais comum é ter várias áreas e precisar calcular as mesmas estatísticas de forma consistente. A lógica, no entanto, não muda: para cada sub bacia, recortamos o *raster*, isolamos os *pixels* válidos e calculamos medidas-resumo que representem o relevo daquela área.

A automatização consiste basicamente em repetir esse processo para cada feição do GeoDataFrame das sub bacias e armazenar os resultados em uma estrutura organizada. Para isso, começamos criando uma lista vazia, que irá receber os valores calculados em cada iteração.

```
resultados = []
```

Em seguida, percorremos o GeoDataFrame linha a linha. Em cada passo do laço, utilizamos a geometria da subbacia para recortar o MDE com a função `mask()`. Como o *raster* pode conter *pixels* inválidos, convertemos o valor NoData para NaN, garantindo que as estatísticas sejam calculadas apenas sobre valores válidos. Por fim, armazenamos os resultados em um dicionário, que é adicionado à lista.

```
with rio.open(mde_path) as src_mde:
    for idx, row in gdf_sub.iterrows():
        geom = [row.geometry]
        mde_clip, _ = mask(src_mde, geom, crop=True)
        mde_clip = mde_clip[0].astype("float32")
        if src_mde.nodata is not None:
            mde_clip[mde_clip == src_mde.nodata] = np.nan
        resultados.append({
            "id_subbacia": row.get("id", idx),
            "nome": row.get("nome", f"subbacia_{idx}"),
            "alt_media": np.nanmean(mde_clip),
            "alt_min": np.nanmin(mde_clip),
            "alt_max": np.nanmax(mde_clip),
            "alt_std": np.nanstd(mde_clip)
        })
```

Após percorrer todas as subbacias, transformamos a lista de dicionários em uma tabela. Cada linha passa a representar uma subbacia, e cada coluna corresponde a uma estatística altimétrica.

```
df_mde = pd.DataFrame(resultados)
df_mde
```

	id_subbacia	nome	alt_media	alt_min	alt_max	alt_std
0	1	bacia arroio tábua	261.680908	194.223083	355.893372	30.277122
1	4	bacia alto bagé direita	302.592285	225.110870	381.506195	42.188854
2	3	bacia alto bagé esquerda	297.501007	226.554626	381.659943	37.569023
3	2	bacia arroio gontan	227.618958	174.057785	305.614105	23.023233

Essa tabela é o produto central da estatística zonal. Em vez de trabalhar com um *raster* inteiro, passamos a lidar com informações agregadas por unidade territorial, prontas para comparação direta, visualização em gráficos ou integração com outros dados tabulares.

Antes de interpretar os valores numéricos, é útil reforçar a ligação entre esses números e o espaço geográfico que eles representam. Para isso, vale gerar uma visualização sintética mostrando, lado a lado, o MDE recortado para cada sub bacia. Esse painel ajuda a consolidar a ideia de que cada linha da tabela corresponde a um subconjunto específico de *pixels* do *raster* original.

O código a seguir gera um painel com duas linhas e duas colunas, exibindo o MDE recortado de cada sub bacia, com o respectivo nome indicado acima de cada figura.

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.ravel()
```

```
with rio.open(mde_path) as src:
```

```
    base = src.read(1).astype("float32")
```

```
    if src.nodata is not None:
```

```
        base[base == src.nodata] = np.nan
```

```
    vmin, vmax = np.nanmin(base), np.nanmax(base)
```



```
for i, (_, row) in enumerate(gdf_sub.iterrows()):
    geom = [row.geometry]
    out_img, out_transform = mask(src, geom, crop=True)
    arr = out_img[0].astype("float32")
    if src.nodata is not None:
        arr[arr == src.nodata] = np.nan
    ax = axes[i]
    show(arr, transform=out_transform, ax=ax, vmin=vmin, vmax=vmax)
    ax.set_title(row.get("nome", f"subbacia_{i}"), fontsize=12)
    ax.set_axis_off()
for j in range(i + 1, len(axes)):
    axes[j].set_axis_off()
plt.tight_layout()
plt.show()
```

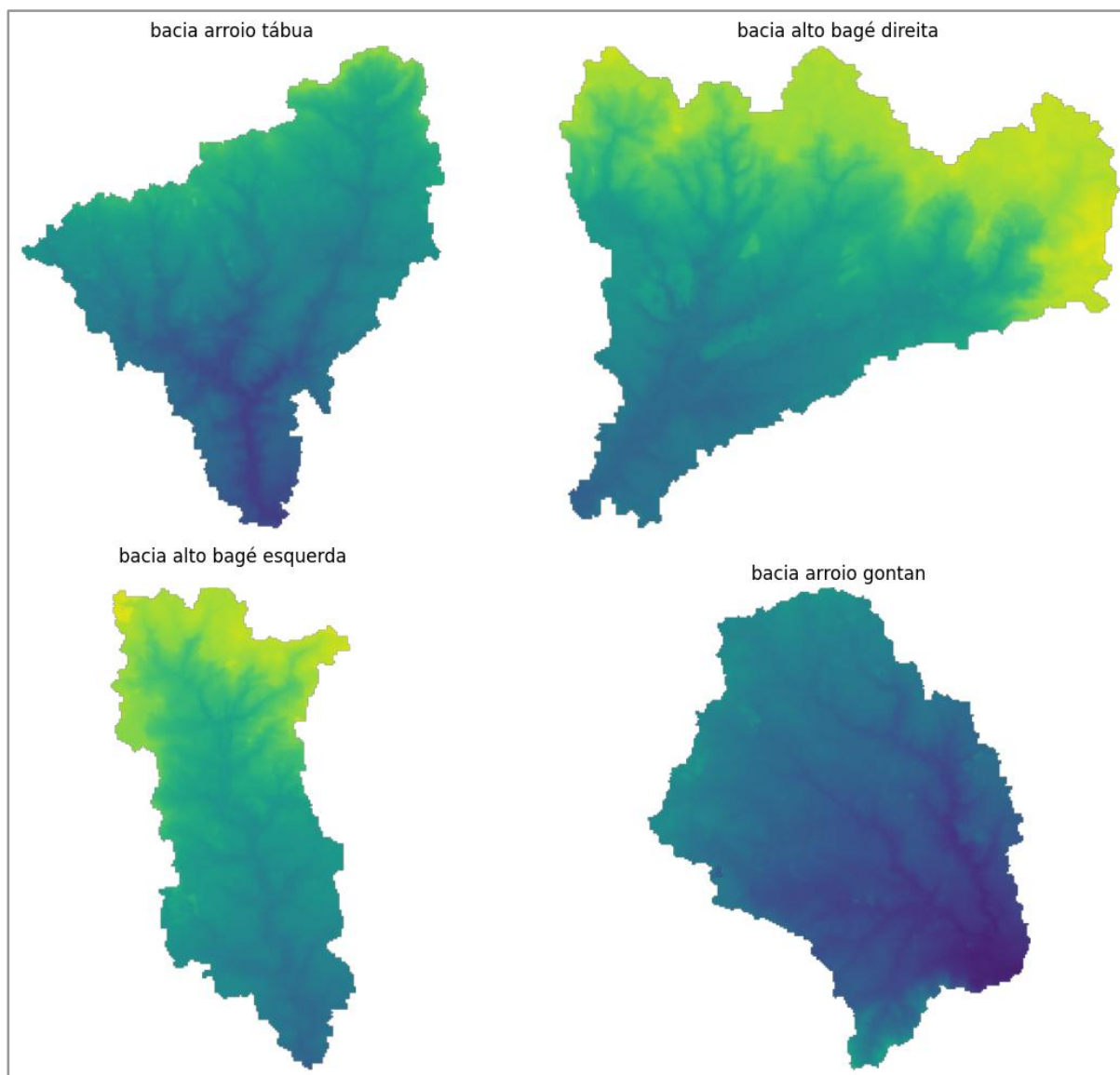


Figura 9.3 — MDE recortado por subbacia (painel comparativo).

Esse tipo de visualização não substitui a tabela de estatísticas, mas cumpre um papel didático importante: ele deixa claro que cada conjunto de números calculados corresponde a uma área espacial concreta, com forma, extensão e padrão altimétrico próprios.

9.4 INTERPRETANDO OS RESULTADOS DA ESTATÍSTICA ZONAL

A Tabela apresentada na Figura 9.3 sintetiza o comportamento altimétrico das quatro sub bacias a partir do Modelo Digital de Elevação. Cada linha representa uma sub bacia específica, e os valores numéricos resultam da agregação dos *pixels* do MDE contidos em sua geometria.

As diferenças nas altitudes médias já indicam contrastes claros no relevo geral. As sub bacias *Alto Bagé Direita* e *Alto Bagé Esquerda* apresentam as maiores altitudes médias (≈ 303 m e ≈ 298 m, respectivamente), o que sugere que essas áreas concentram os setores mais elevados do terreno. Em contraste, a sub bacia do *Arroio Gontan* possui a menor altitude média (≈ 228 m), caracterizando um relevo mais baixo no conjunto analisado.

Os valores de altitude mínima e máxima reforçam essa leitura. Enquanto o *Arroio Gontan* apresenta mínimos em torno de 174 m e máximos próximos de 306 m, as sub bacias do *Alto Bagé* atingem cotas máximas superiores a 380 m. Esses extremos ajudam a identificar áreas potencialmente associadas a divisores de água, encostas mais íngremes ou setores de cabeceira.

O desvio padrão da altitude fornece uma medida simples da variabilidade interna do relevo em cada sub bacia. Valores mais elevados indicam maior heterogeneidade altimétrica. Nesse sentido, a sub bacia *Alto Bagé Direita* apresenta o maior desvio padrão (≈ 42 m), sugerindo um relevo mais variado, com maior amplitude entre áreas altas e baixas. Já o *Arroio Gontan* possui o menor desvio padrão (≈ 23 m), o que indica um relevo relativamente mais homogêneo.

Essas informações numéricas ganham significado quando associadas ao painel comparativo do MDE recortado por sub bacia (Figura 9.3). O painel evidencia que cada linha da tabela corresponde a um subconjunto espacial concreto do *raster* original, com forma, extensão e padrão altimétrico próprios. Assim, a estatística zonal não substitui a análise espacial visual, mas a complementa, permitindo resumir padrões complexos em indicadores territoriais claros.

Em conjunto, os resultados mostram como a estatística zonal permite sair do “nível do *pixel*” e passar para uma leitura comparativa entre unidades espaciais bem definidas. Esse procedimento é fundamental em estudos hidrológicos, ambientais e territoriais, nos quais o interesse está menos em valores pontuais e mais no comportamento agregado de áreas inteiras.

CAPÍTULO 10

ORGANIZAÇÃO, CUIDADOS ESSENCIAIS E PRÓXIMOS PASSOS

Depois de percorrer o caminho que vai da leitura de dados vetoriais e *rasters* ao cálculo de estatísticas por área, este capítulo final reúne orientações para transformar o que você aprendeu em um fluxo de trabalho mais maduro e sustentável. Aqui, o objetivo não é apresentar novos comandos de Python™, mas amarrar as ideias centrais do livro: como organizar projetos de geoprocessamento de forma clara e reproduzível, quais cuidados técnicos merecem atenção constante ao trabalhar com *rasters* e vetores, e que direções você pode seguir para aprofundar as análises a partir daqui.

A ideia é simples: consolidar o que já foi visto e mostrar como esse conjunto de práticas pode ser levado para problemas reais, fora do contexto controlado dos exemplos do livro.

10.1 ORGANIZAÇÃO E FLUXO DE TRABALHO EM PROJETOS GEOESPACIAIS

Trabalhar com dados espaciais significa lidar com muitos arquivos diferentes ao mesmo tempo: camadas vetoriais, *rasters* frequentemente grandes, tabelas auxiliares, notebooks, scripts de teste e figuras de saída. Sem um mínimo de organização desde o início, um projeto rapidamente se torna difícil de entender, reproduzir ou compartilhar. Uma boa prática básica é separar claramente dados brutos de dados processados. Os dados brutos são aqueles obtidos diretamente de fontes externas — portais oficiais, sensores, repositórios públicos — e devem permanecer intocados, servindo como referência original. Sempre que você fizer qualquer transformação, como reprojeção, recorte, cálculo de índices ou alinhamento de resolução, o resultado deve ser salvo como um novo arquivo, em uma pasta separada.

Uma estrutura simples costuma ser suficiente: uma pasta para dados brutos, outra para dados processados, uma para resultados finais (tabelas, mapas, figuras) e outra para os notebooks de análise. No Google Colab®, essa organização se torna ainda mais importante, pois os arquivos geralmente ficam montados a partir do Google Drive e podem ser reutilizados por

vários notebooks. Nomes consistentes e uma hierarquia clara de diretórios reduzem o tempo gasto procurando arquivos e diminuem o risco de sobrescrever resultados importantes.

Outro aspecto fundamental é a documentação dentro do próprio notebook. Células de texto explicando o que será feito a seguir ajudam a dar sentido à sequência de códigos, enquanto comentários curtos nas células de Python™ tornam mais legíveis trechos que envolvem reprojeções, máscaras ou estatísticas por área. Com o tempo, também é útil separar notebooks por função: um para preparação e pré-processamento dos dados, outro para as análises principais e um terceiro apenas para a geração de figuras finais. Isso mantém cada arquivo mais limpo e facilita retomar um passo específico do fluxo sem se perder em dezenas de células.

Mais importante do que a estrutura exata é a coerência interna. Se você definiu uma convenção de nomes para arquivos e pastas — por exemplo, incluir CRS ou datas nos nomes —, procure mantê-la ao longo de todo o projeto. Essa disciplina simples torna o trabalho mais profissional e facilita tanto o seu próprio retorno ao projeto no futuro quanto o entendimento por parte de colegas ou alunos.

10.2 CUIDADOS ESSENCIAIS COM RASTERS E VETORES

Ao longo dos capítulos anteriores, alguns temas apareceram repetidamente: sistema de referência de coordenadas, valores NoData, máscaras, resolução e alinhamento. Essa repetição não é acidental. Pequenos descuidos nesses pontos podem comprometer completamente uma análise, muitas vezes sem gerar mensagens de erro claras.

O primeiro cuidado indispensável é sempre verificar o CRS. Antes de cruzar qualquer *raster* com qualquer camada vetorial, confirme se ambos estão no mesmo sistema de referência. Um *raster* em coordenadas geográficas (graus) e um vetor em coordenadas projetadas (metros) não podem ser integrados diretamente. É fundamental reprojetar uma das camadas e só então aplicar recortes, estatísticas por área ou amostragem de valores. Como regra prática, sempre que o objetivo envolver cálculo de área, distância ou densidade, trabalhe em um CRS projetado adequado à região de estudo.

Outro ponto crítico é o tratamento do NoData em *rasters*. Dados reais quase sempre contêm áreas sem informação, como bordas de cena ou falhas de sensor, codificadas com valores especiais. Se esses *pixels* forem incluídos em médias ou operações matemáticas, os resultados serão distorcidos. Ler *rasters* como *arrays* mascarados ou substituir explicitamente valores NoData por `np.nan` são estratégias seguras para garantir que estatísticas e gráficos representem apenas dados válidos.

Quando trabalhamos com múltiplos *rasters*, a resolução espacial e o alinhamento das grades também exigem atenção. Bandas com resoluções diferentes não devem ser combinadas diretamente. É necessário definir uma grade de referência e reprojeter os demais *rasters* para a mesma resolução, extensão e alinhamento. A escolha do método de reamostragem depende do tipo de dado: *rasters* categóricos pedem métodos como nearest, enquanto *rasters* contínuos, como MDE ou NDVI, se beneficiam de interpoladores como o bilinear.

Do lado vetorial, problemas geométricos também podem surgir. Polígonos com auto interseções ou geometrias inválidas podem causar erros em operações de recorte e estatística por área. Funções simples de correção geométrica ajudam a resolver esses problemas antes que eles impactem a análise. Além disso, o tamanho dos dados não deve ser ignorado: *rasters* multibanda de alta resolução consomem muita memória. Sempre que possível, recorte os dados para a área de estudo logo nas primeiras etapas e evite manter na memória conjuntos muito maiores do que o necessário, especialmente em ambientes como o Colab®.

Em resumo, boas práticas com *rasters* e vetores giram em torno de quatro pilares: CRS consistente, NoData bem tratado, *rasters* alinhados e geometrias válidas. Quando esses pontos estão sob controle, o restante da análise tende a fluir com muito menos surpresas.

10.3 CAMINHOS PARA SEGUIR ALÉM DESTES LIVRO

Com os conceitos e exemplos apresentados até aqui, você já tem uma base sólida para construir análises geoespaciais completas em Python™: leitura e inspeção de dados, manipulação de geometrias e *rasters*, cálculo de índices, reprojeções, recortes e estatísticas por área. Este capítulo encerra o livro, mas também aponta caminhos naturais de aprofundamento.

Um desses caminhos é o sensoriamento remoto aplicado à classificação de uso e cobertura da terra. Os conceitos vistos sobre *rasters* multibanda e índices espectrais podem ser estendidos para fluxos de classificação supervisionada, envolvendo amostras de treinamento, ajuste de modelos e avaliação de acurácia. Outro avanço comum é o trabalho com séries temporais de imagens, em que, em vez de analisar uma única cena, você estuda a evolução de um fenômeno ao longo do tempo. Bibliotecas como xarray e rioxarray ajudam a lidar com esses conjuntos de dados mantendo a lógica de *arrays* já explorada no livro.

No campo da infraestrutura de dados, integrar Python™ a bancos de dados espaciais, especialmente o PostGIS, é um passo importante. Em vez de trabalhar apenas com arquivos soltos, você passa a armazenar dados em tabelas espaciais, executar consultas mais complexas

e compartilhar informações de forma estruturada. Há ainda espaço para avançar em análises espaciais mais sofisticadas, como interpolação, análise de vizinhança, redes e acessibilidade, todas apoiadas na base conceitual construída aqui.

Por fim, plataformas em nuvem voltadas a dados geoespaciais, como o Google Earth Engine®, oferecem acesso a grandes catálogos de imagens e capacidade de processamento integrada. Mesmo com sintaxe e ambiente diferentes, a lógica fundamental continua a mesma: *rasters*, vetores, índices e estatísticas por área. O conhecimento adquirido neste livro se transfere diretamente para esses contextos.

Independentemente do caminho que você escolha seguir, alguns hábitos permanecem essenciais: organizar bem os dados, documentar os passos, validar resultados com senso crítico e manter fluxos de trabalho reproduzíveis. Com isso, você encerra este livro com um conjunto de práticas e conceitos prontos para serem aplicados em problemas reais de geoprocessamento. A partir daqui cada novo projeto será uma continuação natural do que foi aprendido, agora com mais autonomia e confiança no uso do Python™ para análise espacial.

CURRICULUM DO AUTOR



Alexandro Gualarte Schafer: Professor da Universidade Federal do Pampa (Unipampa), Engenheiro Civil (FURG), Mestre e Doutor em Engenharia Civil (UFSC). Atua na interface entre tecnologia geoespacial, ciência de dados espaciais e engenharia, com foco na gestão do ambiente urbano e rural, infraestruturas e recursos hídricos. Suas atividades concentram-se em soluções de engenharia orientadas por dados, baseadas em geoinformática e ciência da informação geográfica, para diagnóstico, monitoramento e apoio à decisão em escalas local e regional.

alexandro.schafer@unipampa.edu.br

 <https://orcid.org/0000-0001-8700-0860>

 <http://lattes.cnpq.br/0395790058174680>

 <https://github.com/Alexandrogschafer>

